Tanta University

Faculty of Computers & Information

Graduation Project 2025/2026

Trabot

A graduation project dissertation by:

AbdelMenem Mohamed AbdelMenem

Abdullah Ashraf Eldeb Mohamed

Abdullah Elsayed Abdullah Elgazzar

Ahmed Khaled Ahmed Zain

Doha Osman Mohamed Mahmoud

Eman Hesham Mohamed Aka

Salspil amin Mahdi Temraz

Samy Mohamed Fathallah Abdelaal

Shahd Nabil Ahmed Mohamed

Shahd Ayman Saeed Mohamed

Supervised by:
Dr. Shimaa Hagras

2026

Table of Contents

..... 1

Acknowledgment..... 1

Abstract 2

CHAPTER 1 — INTRODUCTION..... 3

1.1 Background of the study..... 3

1.2 Significance of the Project..... 4

1.3 Project Objectives..... 4

1.4 Project Scope..... 5

CHAPTER 2 — RELATED WORK 7

2.1 Related Systems Review **..... 7**

2.2 Limitations of Existing Platforms 9

2.3 TRABOT — How It Is Different **..... 10**

CHAPTER 3— INTERVIEW..... 12

3.1 Stakeholder Interview – MobileFix Store **..... 12**

3.2 Stakeholder Interview – Sports Supplements Store **..... 15**

3.3 Stakeholder Interview – Pharmacist**..... 18**

3.4 Stakeholder Interview – Auto Spare Parts Store **..... 20**

3.5 Stakeholder Interview – Toshiba (Electrical & Household Appliances Store) **..... 22**

3.6 Stakeholder Interview – Clothing Store **..... 24**

3.7 Stakeholder Interview – Cosmetics Shop**..... 26**

CHAPTER 4— SYSTEM ANALYSIS..... 28

4.1 Software Requirements Specification 28

4.1.1 Functional Requirements**..... 28**

4.1.2 Non-Functional Requirements**..... 29**

4.2 Use Case Diagram..... 30

4.3 Sequence Diagram..... 38

4.4 Activity Diagram..... 39

CHAPTER 5— SYSTEM DESIGN..... 40

5.1 Introduction 40

5.2 Architectural design..... 40

5.3 Object design..... 42

5.4 User Interface Design..... 47

CHAPTER 6— IMPLEMETATION 71

6.1 Introduction 71

6.2 Tools and technologies 71
CHAPTER 7— CONCLUSION 103
7.1 conclusion..... **103**
7.2 future features..... **103**
7.3 Project links..... **107**
7.4 References **107**

Acknowledgment

We would like to extend our deep appreciation to everyone who played a part in the creation and progress of our graduation project, **Trabot**— a community platform designed to bring together all the shops and essential services a community needs, in one convenient place.

Our sincere gratitude goes to our supervisor, **Dr. Shimaa Hagras**, whose guidance, encouragement, and valuable feedback consistently pushed us to improve our work. Her support throughout the project was essential, and we are truly grateful for her time and dedication.

We are also thankful to our university, our faculty members, and all those who provided the knowledge, resources, and learning environment that enabled us to bring this idea to life. Their efforts helped shape Trabot into a practical and impactful solution.

We would like to give special recognition to the entire development team for their hard work, collaboration, and persistence. Their determination and technical skill were key to turning this project from a concept into a functional platform.

Thank you to everyone who contributed in any way—your support made this journey possible and meaningful.

Abstract

Trabot is a community-based platform designed to connect local residents with nearby service providers, shops and communication within a trusted and verified environment. The platform aims to strengthen local communities by integrating e-commerce, service booking, and social interaction into a single unified system.

Trabot enables citizens to discover local shops, book services such as doctors, teachers, and professionals, and interact through a dedicated community social feed where real experiences, recommendations, and local announcements are shared. Unlike traditional e-commerce or social platforms, Trabot focuses on verified users within the same neighborhood ensuring authenticity, trust, and relevance of content.

The system adopts a role-based architecture with dynamic dashboards tailored to different user roles. Each role has controlled access and personalized features, allowing efficient management of services, products, bookings and community moderation through a single backend infrastructure.

Trabot also leverages artificial intelligence techniques to enhance user experience through smart recommendations, data. These features help users make informed decisions, support small local businesses, and promote meaningful engagement within the community.

By transforming local services and transactions into trusted social experiences, **Trabot** aims to reduce uncertainty in decision-making, encourage local economic growth, and become an essential digital hub for everyday community life.

CHAPTER 1 — INTRODUCTION

1.1 Background of the study

The rapid growth of digital platforms has significantly changed the way people access services, shop for products, and communicate with others. Traditional e-commerce platforms and social networks have successfully connected users on a global scale; however, they often fail to address the specific needs of local communities. These platforms usually lack trust, local relevance, and real-life verification, which leads to uncertainty, unreliable recommendations, and limited support for small local businesses.

In many local communities, especially in villages and neighborhoods, residents rely heavily on word-of-mouth recommendations to choose doctors, teachers, shops, and service providers. Despite this reliance on trust-based interactions, there is no unified digital platform that effectively combines local services, verified community members, and real experiences into one system. Existing solutions either focus solely on online transactions or social interaction, without integrating both aspects in a meaningful and reliable way.

Trabot is proposed to bridge this gap by introducing a community platform that connects citizens, service providers, store owners within the same local environment. The platform emphasizes verified users, role-based access, and community interaction built around real services. By linking social engagement with actual transactions, **Trabot** ensures that recommendations, reviews, and content are based on genuine experiences.

Furthermore, the growing demand for convenience, transparency, and personalized digital experiences highlights the need for intelligent systems that support decision-making. **Trabot** addresses this need by incorporating smart recommendation mechanisms and data-driven insights that enhance service discovery, improve user satisfaction, and support small businesses in understanding community needs.

By focusing on trust, locality, and real-world interaction, **Trabot** aims to transform how local communities engage with services and commerce, providing a secure, interactive, and sustainable digital ecosystem tailored to everyday life.

1.2 Significance of the Project

Most existing e-commerce platforms are primarily designed for large-scale markets and global transactions, offering limited personalization, weak community interaction, and insufficient support for small local businesses.

Trabot addresses these limitations by providing a community-centered platform. The platform enables shops, service providers, and professionals to enter the digital market with minimal cost and technical complexity.

This approach enhances trust, reduces uncertainty in decision-making, and encourages continuous community engagement. Additionally, the platform's role-based system and smart recommendation features help users access relevant services efficiently while supporting business owners with data-driven insights.

Trabot also encourages residents to buy from nearby shops, use local services, and share authentic experiences, which contributes to increased sales, business sustainability, and job creation. Ultimately, this project demonstrates how technology can be leveraged to create a secure, inclusive, and socially connected e-commerce ecosystem that prioritizes local communities over mass-market competition.

1.3 Project Objectives

The objectives of the **Trabot** project is improving the overall digital experience for users within the same locality. These objectives include:

1. **Develop a community-centered digital platform**
   Trabot aims to create a localized e-commerce ecosystem that connects residents with nearby shops, service providers, and professionals. By focusing on a shared geographic community, the platform strengthens social ties and promotes neighborhood-based economic activity.
2. **Support and empower small businesses and service providers**
   The platform provides small shops and local service providers with an accessible and low-cost digital presence, enabling them to showcase products and services without advanced technical requirements or competition with large-scale retailers.
3. **Enable meaningful and direct communication**
   Trabot facilitates direct interaction between users and sellers or service providers through messaging and community engagement, helping build trust, improve transparency, and deliver personalized customer support.
4. **Provide efficient and transparent transaction management**
   The system offers clear order and booking tracking, real-time updates, and well-

defined processes that enhance reliability, reduce confusion, and improve user confidence.

5. **Establish trust through verified reviews and ratings**
   Trabot implements a structured and experience-based rating system to ensure feedback accuracy, support informed decision-making, and maintain service quality across the platform.
6. **Deliver a smooth and intuitive user experience**
   The platform is designed with simplicity and usability in mind, featuring unified search, smart filtering, and clear navigation to encourage frequent use and long-term engagement.

1.4 Project Scope

The platform is divided into the main following modules:

1. **User Mobile Application**
   The user mobile application represents the main access point to the Trabot platform. It allows users to browse nearby shops, service providers, and professionals within their community. Users can search, filter, and view detailed product and service information, place orders or bookings, complete secure payments, track deliveries or appointments in real time, and receive notifications.
2. **Seller & Service Provider Dashboard**
   This dashboard enables shop owners and service providers to manage their digital operations efficiently. Users can add and update products or services, manage pricing, availability, and inventory, upload images and descriptions, and track orders or bookings. Service providers can also manage schedules, available time slots, and appointment confirmations, allowing better time management and service delivery.
3. **Admin Dashboard**
   The admin dashboard serves as the central control system of Trabot. Administrators manage users, roles, communities, shops, services, orders, bookings, and payments. The dashboard supports moderation, verification, dispute resolution, and policy enforcement. Analytical tools provide insights into user activity, platform growth, and community performance to support data-driven decisions.
4. **Community Social Module**
   This module enables verified community members to interact through posts,

questions, recommendations, local announcements, and alerts. All social interactions are linked to real profiles and services, ensuring authenticity and trust. The community feed strengthens engagement, supports informed decision-making, and transforms services into shared social experiences.

5. **Product and Service Management**
   This module ensures structured and accurate management of products and services. Sellers and providers can categorize items, define pricing, manage stock or availability, and highlight promotions or special offers. Maintaining updated information improves discoverability and enhances the overall user experience.
6. **Booking and Scheduling System**
   The booking module allows users to schedule appointments with doctors, teachers, and professionals. It includes availability calendars, booking confirmation, reminders, cancellation policies, and post-service notifications. This system reduces scheduling conflicts and improves efficiency for both users and service providers.
7. **Ordering and Tracking**
   This module manages the full order lifecycle, including order placement, processing and real-time tracking. Delivery companies can update order statuses, calculate delivery costs, and manage service areas. Users receive continuous updates, ensuring transparency and reliability throughout the process.
8. **AI-Powered Recommendation System**
   The recommendation engine analyzes user behavior, search history, bookings, and community interactions to suggest relevant products, services and professionals. This intelligent feature enhances personalization, reduces search effort, and increases user satisfaction.
9. **Role-Based Access Control and Permissions**
   Trabot implements a role-based permission system that controls feature access based on user roles, such as citizen, service provider, store owner or administrator. A unified dashboard layout dynamically adapts to each role, ensuring security, clarity, and efficient interaction with platform features.

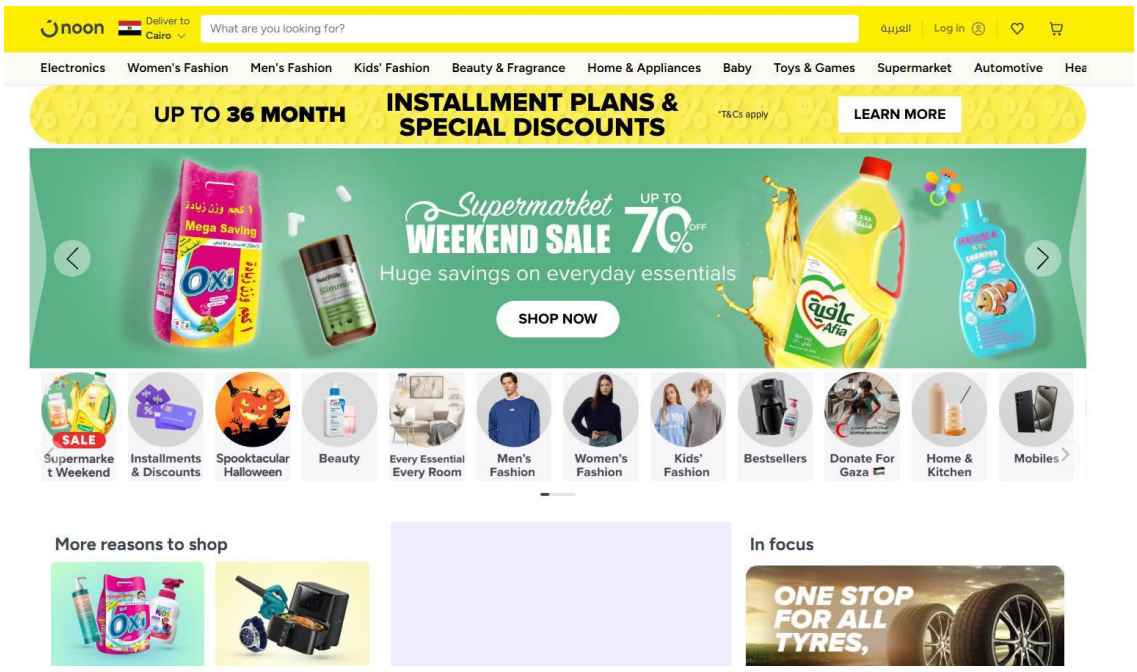
CHAPTER 2 — RELATED WORK

2.1 Related Systems Review

E-commerce platforms have evolved significantly in recent years, offering extensive features that connect buyers and sellers and streamline online transactions. Several well-known platforms—such as **Amazon, Noon, and Jumia**—have shaped the online shopping experience worldwide. However, these platforms still have limitations that motivate the need for a more community-focused system.

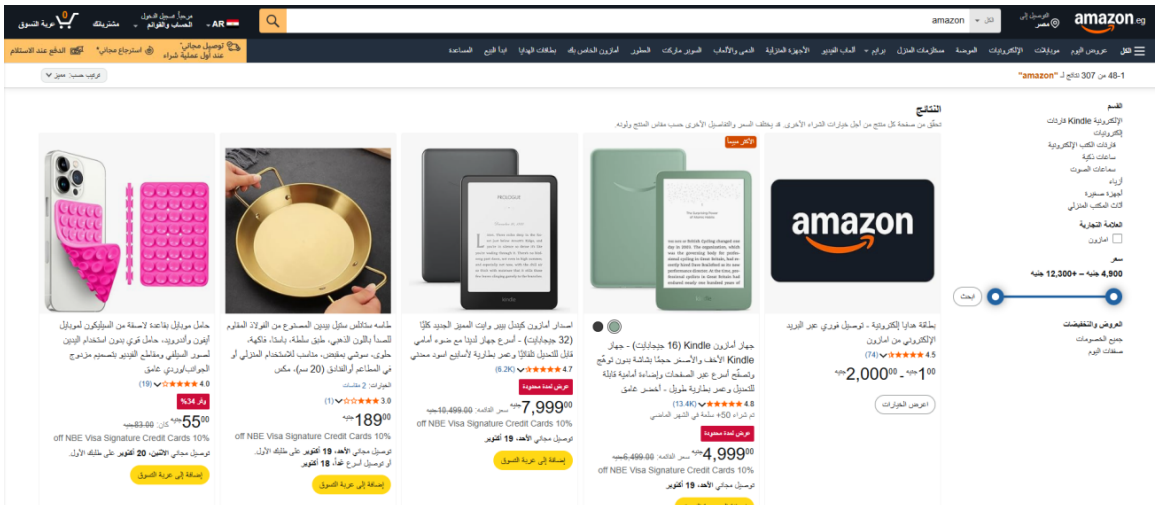
2.1.1 Noon

Noon is among the most popular e-commerce platforms in the Arab world. It provides an Arabic-friendly interface, multiple payment options, and frequent deals that attract a wide customer base.
Despite these strengths, Noon—like most large platforms—does not offer direct communication between buyers and sellers. The relationship remains transactional and impersonal, with no emphasis on building trust or long-term connections within communities.



2.1.2 Amazon

Amazon is one of the largest global e-commerce platforms, known for its clean interface, strong customer service, and sophisticated recommendation system powered by artificial intelligence. The platform offers fast delivery and an enormous variety of products.
However, Amazon poses challenges for small sellers, who struggle due to high competition and pricing pressures. While Amazon excels at large-scale global retail, it does not prioritize local communities or foster direct communication between buyers and sellers.



2.1.3 Vezeeta

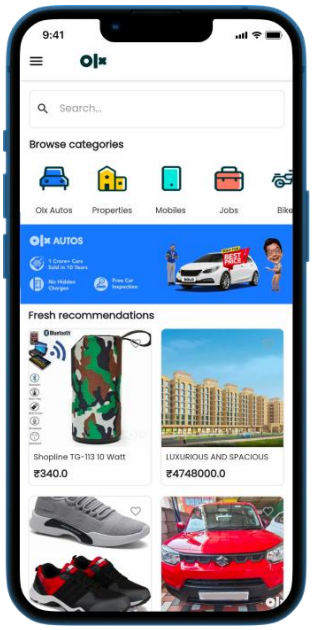
Vezeeta is one of the leading healthcare booking platforms in the Middle East. It enables users to search for doctors based on specialty, location, ratings, and consultation fees. Patients can book appointments online and receive reminders before their scheduled visits.

Although Vezeeta provides an efficient appointment management system, it is limited to healthcare services and does not support other professional categories such as private tutors or educational instructors. In addition, the platform focuses solely on appointment scheduling and lacks community interaction features that allow users to communicate, share experiences, or engage with service providers beyond the booking process.

2.1.4 olx

OLX is a widely used classified advertisements platform that allows users to buy and sell new or second-hand products. It provides a simple posting process, direct communication between buyers and sellers, and a wide range of product categories.

However, OLX primarily focuses on classified advertisements and does not provide integrated e-commerce functionality such as shopping carts, order management, or secure multi-vendor transactions. Additionally, the platform lacks service-booking capabilities and does not offer a unified ecosystem that combines shopping, service reservations, and social interaction.



2.2 Limitations of Existing Platforms

Despite the popularity of platforms such as Amazon, Noon, Vezeeta, and OLX, they still suffer from several limitations that prevent them from providing a fully integrated community-oriented experience. The most common issues include:

- Limited direct interaction and communication between users, sellers, and service providers.
- Separation of services across multiple platforms, requiring users to switch between different applications for shopping, booking appointments, and buying or selling used items.
- Limited support for local communities and neighborhood-based interactions.
- Lack of a unified platform that combines e-commerce, service booking, classified advertisements, and social networking features.
- Restricted opportunities for small businesses, tutors, and independent service providers to reach potential customers.
- Dependence on traditional rating and review systems without promoting ongoing relationships between users and providers.
- Insufficient personalization based on users' interests, preferences, and local environment.

These limitations highlight the need for a comprehensive platform that integrates all of them in one.

2.3 TRABOT — How It Is Different

Trabot is a community-based digital platform designed to overcome the limitations of traditional e-commerce systems, specific bookings and communication by combining them in specific geographic area. The system introduces a new approach that focuses on local communities and smart technology.

1. Community & Location-Based Focus

Trabot gathers all shops and service providers within a defined area or neighborhood, including retail stores, clinics, professionals, restaurants, gyms, and more, creating a unified local marketplace.

2. Integrated E-Commerce & Service Booking

Users can:

- Purchase products from nearby stores online
- Book services such as medical appointments, private lessons, or gym sessions
- Manage all orders and reservations through one platform

3. Real-Time Order Tracking

Users can track their orders during the delivery process step-by-step, ensuring transparency and improving customer satisfaction.

4. Community Social Hub

The platform includes a built-in social space where users can post content, share experiences, interact with neighbors, and engage in local discussions—transforming **Trabot** into more than just a marketplace.

5. Intelligent Features Using Artificial Intelligence

Trabot integrates AI-powered tools such as:

- Personalized recommendations for products based on user behavior, preferences.
- Smart suggestions for services and products relevant to the user's community.
- Suggesting tops rated services and products based on customer ratings.

6. Trustworthy Rating System

A structured and verified rating and review system ensures accuracy, transparency, and reliability for both products and services.

CHAPTER 3— INTERVIEW

There are some interviews with stakeholders to understand and solve their problems in our project:

3.1 Stakeholder Interview – MobileFix Store

Store: MobileFix Store (small shop selling mobile accessories)
Interviewee: Mohamed Sami, 29 years old, owner
Business Duration: 5 years
Purpose: To understand the challenges faced by small sellers on large e-commerce platforms and identify needs to develop Trabot, a community-based platform supporting local businesses.

Interview Questions and Answers

1. What is the most problem you encountered while selling products on platforms (Amazon - Noon - Jumia)?

- The most problem I faced was intense competition, especially from large sellers and the prices that are always lower than us, and this makes our products not appear to the customer easily.

2. Why did you start selling online and why did you choose the first platform?

- I started selling online because I feel that the online market is much bigger. From selling in the store and my first choice was (Jumia) because it was easier to register and less complicated.

3. Difficulties during registration?

- She was requesting a lot of papers such as the commercial register and the tax card, and also the audit of the account took a long time until I agreed.

4. Problems while lifting products?

- Yes, photo hours were rejected because they were not in high quality or a certain size, and also writing the description in an appropriate way takes a long time to be accurate.

5. What do you think about the sales and commission fees?

- Frankly, in (Amazon and Noon) the commission is high, and this affects the profit margin, especially for the small seller. But in (Jumia) it is a little lighter.

6. For inventory management, how do you track the number of pieces available? Does the system help you or do you manage it manually?

- I follow the stock manually and with the help of Excel, because the system is not always accurate, especially if there are many requests, sometimes a product can be finished and still visible.

7. Have you had a shipping issue or delivery delay? If it happens, does this affect the evaluation or reputation of your store?

- There were many problems with the delay, and this made some customers write negative reviews, and this affected the reputation of the store and the visibility rate on the platform.

8. How was your experience with the vendor support service in the platforms? They were effective and you don't have to wait too much?

- The support is very slow and takes a long time to respond, especially in (Amazon and Noon) a little faster, but I don't feel that they understand my problem well.

9. The seller's control panel, I think it's simple, easy, not complicated, and needs training to deal with it?

- Frankly, the Amazon control panel is complex and full of details, but Jumia is much easier and more organized.

10. When a customer leaves a negative review, how does the platform deal with him? Do you feel that the evaluations can sometimes oppress the seller?

- The evaluations are very affected by sales, even if the problem was from shipping, not from the product, and the platform does not have a very little negative evaluation, even if you are wronged.

11. Does the platform provide you with marketing or advertising tools that help you promote your products? Even ah, is it effective?

- Oh, it exists, but it is very expensive, especially paid ads on Amazon, and if the products don't do it, it won't appear to people.

12. Are you having trouble competing with big brands or big vendors on the same platform?

- Of course, large companies sell at lower prices, and they have a budget for advertising, and this makes it difficult for me to appear in the search before them.

13. If a customer returns the product, do you feel that your rights are reserved as a seller? Or are you financially damaged?

- Unfortunately, in returns, the hours of the product return open or damaged, and I return. I am the one who bears the cost, and my rights are not always reserved.

14. From your point of view, what are the missing needs in the current platforms that need to be present to help the small seller?

- Reduce commission for small sellers.
- Faster and clearer technical support.
- Easy tools for sales and inventory analysis.

15. Have you encountered problems withdrawing profits or delaying the arrival of financial dues? If it happens, can you explain?

- Sometimes profits are delayed by two weeks or more, especially in the season of offers, and this stops the capital and makes me not able to buy new goods.

16. In your opinion, if we design a new platform that supports local vendors, what are the 3 most important features that should be present in it?

- Small or fixed commission for the small seller.
- Reliable and fast shipping system.
- A simple control panel with clear reports of sales, inventory, and profits.

Main Problems

Small sellers on platforms like Amazon, Jumia, and Noon face these issues:

1. Difficult and slow seller registration process
2. Complicated dashboard for managing products, sales, and inventory
3. High commissions and fees that reduce profit
4. No smart alerts for low or out-of-stock products
5. Shipping problems that affect seller ratings even when it’s not their fault

6. Slow and ineffective customer support
7. Hard to promote products and compete with big brands
8. Delayed payout of earnings or money being held without explanation

---

User Needs

Seller's need:

1. Fast and easy registration process
2. A simple dashboard showing sales, orders, profit, and stock
3. Easy product upload (photos, prices, descriptions)
4. Smart notifications for low or almost out-of-stock items
5. Fair commission rates or flexible plans for small sellers
6. Reliable shipping that doesn't harm seller ratings
7. Quick support from real people, not just bots
8. Fast access to earnings with clear financial reports

3.2 Stakeholder Interview – Sports Supplements Store

Store: Sports Supplements Store
Interviewee: Owner
Purpose: To understand the main challenges small business owners face when selling online and how Trabot can help them transition into e-commerce with ease.

Interview Questions and Answers

1. Can you tell us about your store and how long you've been in business?

- I own a sports supplements store that has been running for about five years. We mainly rely on in-store sales.

2. Have you ever considered selling your products online?

- Yes, I have, but it always seemed complicated. I wasn't sure how to start or manage online payments.

3. At Trabot, we offer a full e-commerce solution. You can sell online using only your smartphone—no technical setup required. How would you feel about managing your business through an app?

- That sounds great! If it's simple and helps me reach more customers, I'd definitely use it.

4. Do you currently use any online platforms to sell, like Amazon or Jumia?

- Not yet, because those platforms require too many documents and the registration process is quite long.

5. What do you think are the main challenges for small sellers joining e-commerce platforms?

- The biggest challenges are high commissions, complicated setup steps, and lack of personal support.

6. Do you think managing stock and orders online would be easy for you?

- If the system is simple and has alerts for low stock, yes. Otherwise, it could get confusing.

7. How important is it for you to receive payments quickly?

- Very important. Delays in receiving money can stop me from restocking products on time.

8. What are your thoughts on shipping and delivery issues with online sales?

- Fast and reliable delivery is crucial. Late deliveries can cause bad reviews and harm the store's reputation.

9. Have you faced issues with customer reviews or product returns before?

- Not personally, but I know many sellers who lose money because of unfair return policies or negative feedback.

10. What kind of support do you expect from an e-commerce platform?

- Quick and human customer support that actually solves problems, not just automated replies.

11. If you could improve one thing in current online marketplaces, what would it be?

- Lower commissions for small sellers and a simpler dashboard for managing everything.

12. Would you be interested in promotional tools or ads to boost your sales?

- Yes, if they're affordable. Paid ads on big platforms are usually too expensive for small businesses.

Main problems

Currently, small sellers face several difficulties when trying to join large e-commerce platforms, including:

- Complex registration steps and long approval times
- Unclear dashboards for managing products and sales
- High commission rates that reduce profits
- Slow customer support that delays problem resolution
- Shipping delays and unfair return policies that can harm reputation and financial stability

---

User Needs

Small business owners need a platform that provides:

- Simple and fast registration with minimal documentation
- A clear, easy-to-use dashboard for managing sales, inventory, and payments
- Fair and transparent commission plans
- Reliable and fast shipping options
- Human customer support with quick responses
- Marketing and analytics tools to help grow their business
- Flexible payment systems allowing quick withdrawal of earnings

---

3.3 Stakeholder Interview – Pharmacist

Business Type: Pharmacy (sells medicines and medical products)
Purpose: To understand the challenges and expectations of pharmacists regarding online sales and digital pharmacy platforms, in order to guide the development of an online pharmacy ordering system.

Interview Questions and Answers

1. What is your business type?

- A pharmacy – sells medicines and medical products.

2. Did you try online selling through platforms like Jumia or your own website?

- No, because of fear of scams, fake products, or poor-quality items that don't match the description.

3. What are the main difficulties that you face as a pharmacist?

- Sometimes prescriptions are hard to read.
- Some products are missing from the store (out of stock), so alternatives must be offered to customers.

4. Did you face difficulty in tracking stock or missed products?

- Yes. For example, when a customer asks for four items and one is missing, they take a deposit and order the missing product from the supplier.

5. Did you face problems with returns?

- Yes. Some customers open or damage products and then want to return them, which causes a loss because the item can't be sold again. The return checker should ensure the item is in the same condition.

6. What is your opinion about an online platform/app for local pharmacies and stores?

- It's a great idea. It will make it easier for people to order trusted products from multiple nearby shops in one place.

7. In your point of view, what are disadvantages of online selling?

- Hard to tell if a product is real or good quality, especially cosmetics (like skin cleansers).
- Some websites sell products at unrealistically low prices, making customers think they're fake or used.

8. What is your opinion about showing videos and photos for products?

- Yes, very useful—especially for cosmetics—to show the product's serial number and package size.

9. What is your preferred payment method: cash or online?

- Both "cash on delivery" and "online payment" should be available. Customers should choose freely.

10. What are the desired features in an online platform for your business?

- Fast delivery
- Products always available (never out of stock)
- Customer reviews and detailed product information
- Easy to use on both mobile and computer
- Ability to upload prescriptions and chat with a pharmacist about them

---

Main Problems

1. Hard to read some prescriptions
2. Frequent stock shortages
3. Difficulty tracking what's available or out of stock
4. Issues with damaged product returns
5. Customers' distrust of online product authenticity or quality
6. Lack of clear visuals or product details online

User Needs

1. Reliable online system showing only verified, original products
2. Easy stock tracking for the pharmacy
3. Option for pharmacist–customer communication (for prescriptions)
4. Secure and flexible payment options
5. Fast delivery service
6. Clear photos and videos of products (with serial numbers)
7. Customer reviews and full product descriptions

3.4 Stakeholder Interview – Auto Spare Parts Store

Business Type: Shop selling spare parts for cars and rims
Purpose: To understand the challenges of selling auto spare parts online and gather insights for building a local e-commerce platform like Trabot.

Interview Questions and Answers

1. What type of business do you operate?

- A shop that sells spare parts for cars and rims.

2. Do you have any previous experience selling online, whether through a platform or a personal page?

- Yes, I had transactions before through a page. I had a Facebook page where I sold, displayed products, and interacted with customers.

3. What are the problems you face when selling products through Facebook?

- Pricing the products is difficult because prices are always increasing. I stopped posting the price on products because it's hard to update after people have interacted with the post based on the old price.
- Customers may ask about a product that is already sold out, requiring me to update posts constantly.
- Following up with customers during the shipping period is challenging.

4. What are the problems you face when shipping orders or communicating with customers?

- There's no tracking of orders because I rely on a shipping company. Once the delivery person takes the product, communication happens only between the customer and the delivery agent. Problems sometimes arise during delivery, such as unclear addresses or the delivery person going to the wrong location. Without tracking, I cannot identify the issue or the order's location, especially for customers living outside Tanta, such as in Alexandria or Fayoum.

5. What payment method do you use?

- All transactions are online, done exclusively through Vodafone Cash or InstaPay.

6. What do you think of creating a platform where you can track products?

- It would be a great idea. It would make the buying and selling process easier for both customers and me, and increase credibility, as some people hesitate to buy from unknown or unreliable sellers.

7. What do you think of displaying the products in photos and videos?

- It would be very useful to make the product clearer. The QR code should be visible so customers can verify that it is original.

8. If you're using the platform, can you accept cash payments?

- Of course, if the transaction is through a trusted platform.

9. What features would you like to see?

- Fast delivery with tracking until the product reaches the customer.
- A customer feedback section to follow up on product issues or special requests.
- Easy-to-use platform for both mobile and laptop.
- Simple product management, allowing modification of price, model, color, etc.

Main Problems

1. Frequent changes in product prices make it difficult to update listings.
2. Products are sometimes sold out, requiring constant updates.
3. Lack of order tracking and visibility during shipping.
4. Difficulties in communicating with customers during delivery.
5. Limited credibility and trust when selling online through informal channels.
6. Manual and time-consuming product management (editing price, model, color, etc.).
7. Customers may have difficulty verifying product authenticity.

User Needs

1. A platform that allows fast and easy product updates, including price, model, and availability.
2. Real-time tracking for orders during the delivery process.
3. Reliable and fast shipping options.
4. Ability to communicate with customers for inquiries or issues.
5. Secure and trustworthy environment for cash and online payments.
6. Clear product visuals, including photos, videos, and QR codes for authenticity.
7. Customer feedback section for monitoring satisfaction and handling complaints.

8. Easy-to-use interface on both mobile and desktop for product management.

3.5 Stakeholder Interview – Toshiba (Electrical & Household Appliances Store)

Business Type: Selling electrical and household appliances and their spare parts
Purpose: To identify challenges in selling appliances online and gather requirements for a multi-vendor e-commerce platform like Trabot.

Interview Questions and Answers

1. What business do you run?

- Selling electrical and household appliances and their spare parts.

2. Do you have previous experience selling online, whether through your own platform?

- Yes, there is an official Toshiba page listing products and spare parts, a section for customer reviews, and a section for offers and discounts.

3. What are the problems you face while selling online?

- Delivery and shipping issues: orders can be delayed, and products (especially large appliances) can get damaged during transit.
- Page slowness affects purchases, data entry, and online payments.

4. Have there been customer complaints about orders or communication?

- Customers want the ability to track their orders from dispatch until delivery. Out-of-stock items sometimes require postponement or refunds, depending on the customer's preference.

5. What payment method do you use?

- Only online payments through the National Bank of Egypt, Vodafone Cash, and Etisalat Cash.

6. Is it possible to display your products on another platform with multiple vendors?

- Possible if technical issues are resolved. Only a portion of products may be displayed to encourage visits to the official page.

7. Is it possible to accept cash payments through the platform?

- No, online payment is preferred as it is secure and guaranteed.

8. What features would you like to see added to the platform?

- Order tracking
- Solving slow-loading issues
- Real product photos and videos to ensure authenticity
- Faster shipping
- Multiple communication channels with customers

Main Problems

1. Delivery delays and product damage during shipping
2. Slow website/page performance affecting sales
3. Customers unable to track orders
4. Out-of-stock issues causing order delays or partial refunds

---

User Needs

1. Reliable order tracking system
2. Faster and more secure online platform
3. Clear product photos and videos for authenticity
4. Expedited and secure delivery
5. Multiple communication channels with customers

3.6 Stakeholder Interview – Clothing Store

Business Type: Physical clothing store (planning to start online sales)
Purpose: To understand challenges in selling clothing online and desired platform features.

Interview Questions and Answers

1. Do you have a physical store only, or do you also sell online?

- I have a physical store and plan to start selling online soon.

2. Have you tried selling online through platforms like Instagram, Jumia, or your own website?

- Tried Instagram; it was good initially but tiring due to manual customer communication and order tracking.

3. What are the main difficulties when displaying clothes online?

- Capturing true colors in photos and addressing size differences between brands.

4. Difficulties in delivering accurate information about fabric or size?

- Yes, especially for fabrics, as photos often don’t show material type clearly.

5. Have you received complaints about differences between photos and real products?

- Yes, especially due to strong lighting in photos.

6. What is the hardest part of managing orders?

- Following up on orders and handling returns is exhausting.

7. Do you find it difficult to track inventory or out-of-stock items?

- Yes, especially with many models and colors.

8. What features would you like in an online selling platform?

- Easy-to-use interface, quick product addition, stock synchronization with physical store

- Ability to design store interface (colors, images, logos)
- Manual pricing with suggested seasonal discounts
- Filtering by size, color, fabric type
- Display products with photos and short videos
- Customer reviews and ratings
- Reports showing best-selling products and most profitable seasons
- Mobile app management preferred
- Simple admin panel with sales and stock updates and alerts

9. Delivery and payment preferences:

- Cash on delivery preferred, with options for cards and e-wallets
- Prefer platform-provided delivery partners

Main Problems

1. Difficulty showing accurate colors, fabric, and size online
2. Inventory management is complex for many models and colors
3. Manual order tracking and returns are time-consuming
4. Shipping delays from third-party companies

User Needs

1. Easy-to-use platform for product management and stock synchronization
2. Mobile-friendly admin panel
3. Customer filtering options (size, color, fabric)
4. Customer reviews and ratings
5. Delivery partner integration
6. Reports for sales and profitability
7. Photos and short videos for accurate product presentation

3.7 Stakeholder Interview – Cosmetics Shop

Business Type: Gift and cosmetics shop
Purpose: To understand challenges and requirements for starting online sales.

Interview Questions and Answers

1. Can you tell us more about your business?

- I own a gift and cosmetics shop selling gifts (teddy bears, flowers, accessories) and original beauty products.

2. What's your main goal for creating an online store?

- To increase sales and reach more customers online instead of relying only on walk-ins.

3. Have you ever sold online before?

- Yes, through Facebook and WhatsApp. Sometimes this will be the first official online store.

4. What payment methods would you like to offer?

- Start with cash on delivery; later add card payments or Vodafone Cash.

5. Do you have your own delivery system, or use a shipping company?

- Mostly a shipping company; sometimes personal delivery for nearby areas.

6. Do you have professional photos of your products?

- Yes, but they need organizing or enhancement.

7. Would you prefer short or detailed product descriptions?

- Short and catchy descriptions are enough.

8. Who is your main target audience?

- Mostly women aged 18–35 buying for themselves or as gifts.

9. Do you offer discounts or special deals?

- Yes, during occasions like Mother's Day, Valentine's Day, New Year, etc.

10. Who will manage the store and products?

- Initially myself; later may hire someone if business grows.

11. How would you like customers to contact you?

- Through WhatsApp; it's easiest and fastest.

12. Do you plan to expand products or business in the future?

- Yes, adding personal care products and customized gifts.

Main Problems

1. Lack of organized and professional product presentation online
2. Manual handling of customer inquiries and orders
3. Delivery and payment options limited initially

User Needs

1. Easy-to-manage online store for gifts and cosmetics
2. Professional product photos and short descriptions
3. Multiple payment methods including cash on delivery and digital payments
4. Integration with shipping companies and own delivery options
5. Efficient customer communication channel (WhatsApp or similar)

CHAPTER 4— SYSTEM ANALYSIS

4.1 Software Requirements Specification

Software Requirements Specification (SRS) involves creating a complete and structured document that captures every requirement of the system from both the technical and user perspectives. In the following sections, we outline the full set of requirements for **Trabot**.

- **Functional Requirements** describe the features and operations the system must provide to fulfill user expectations.
- **Non-Functional Requirements** specify the overall qualities and standards the system should meet. These cover aspects like ease of use, system performance, security measures, and compatibility across different devices and web browsers.

4.1.1 Functional Requirements

1. **User Management**

- The system shall allow users to register and log in as customers, store owner, service provider or admin of the system.
- The system shall support management for all user types.

2. **Local Marketplace**

- The system shall display products available from shops .
- The system shall allow users to browse, search, and filter products by category, price, rating and search.
- The system shall allow sellers to add, update, and manage product listings.

3. **Service Booking**

- The system shall allow users to book services such as doctor appointments, private lessons and gyms.
- The system shall provide available time slots for each service.

4. **Order Management**

- The system shall allow users to place orders for products from local shops.
- The system shall provide order confirmation and status updates.

5. **Real-Time Order Tracking**

- The system shall allow users to track delivery status in real time.
- The system shall display delivery stages (e.g., processing, shipped, cancelled, delivered).

6. **Community Social Platform**

- The system shall allow users to create posts within the community.
- The system shall allow users to communicate with the post publisher.

7. **Analytics Dashboard**

- The system shall present visual analytics dashboard for admin, store owner, service provider and customer.

8. **AI-Powered Features**

- The system shall generate personalized recommendations based on user behavior, reviews and ratings.

4.1.2 Non-Functional Requirements

1. **Usability Requirements**

• The system shall provide a simple, intuitive interface for sellers with clear navigation.
• All pages must be responsive and accessible on different devices.
• The seller dashboard shall use icons, colors, and visual indicators for clarity.

2. **Scalability Requirements**

• The system shall scale horizontally to support increased traffic loads.
• The database shall support indexing to speed up queries as data grows.

3. **Performance Requirements**

• The system shall load main pages (Home, Products, Dashboard) in less than 3 seconds under normal network conditions.

4. **Security Requirements**

- The system shall Protect seller and customer data and passwords.

5. **Reliability**

   - The platform should be highly reliable and consistently available to users.
   - It must recover gracefully from errors and maintain stability.

6. **Maintainability**

   - The platform should be developed using a modular and clean architecture that allows for easy updates, bug fixes, and future feature additions

7. **Compatibility**

   - Trabot should function correctly across all major web browsers (Chrome, Firefox, Safari, Edge) and be responsive mobile devices. The experience should remain consistent across platforms.

4.2 Use Case Diagram

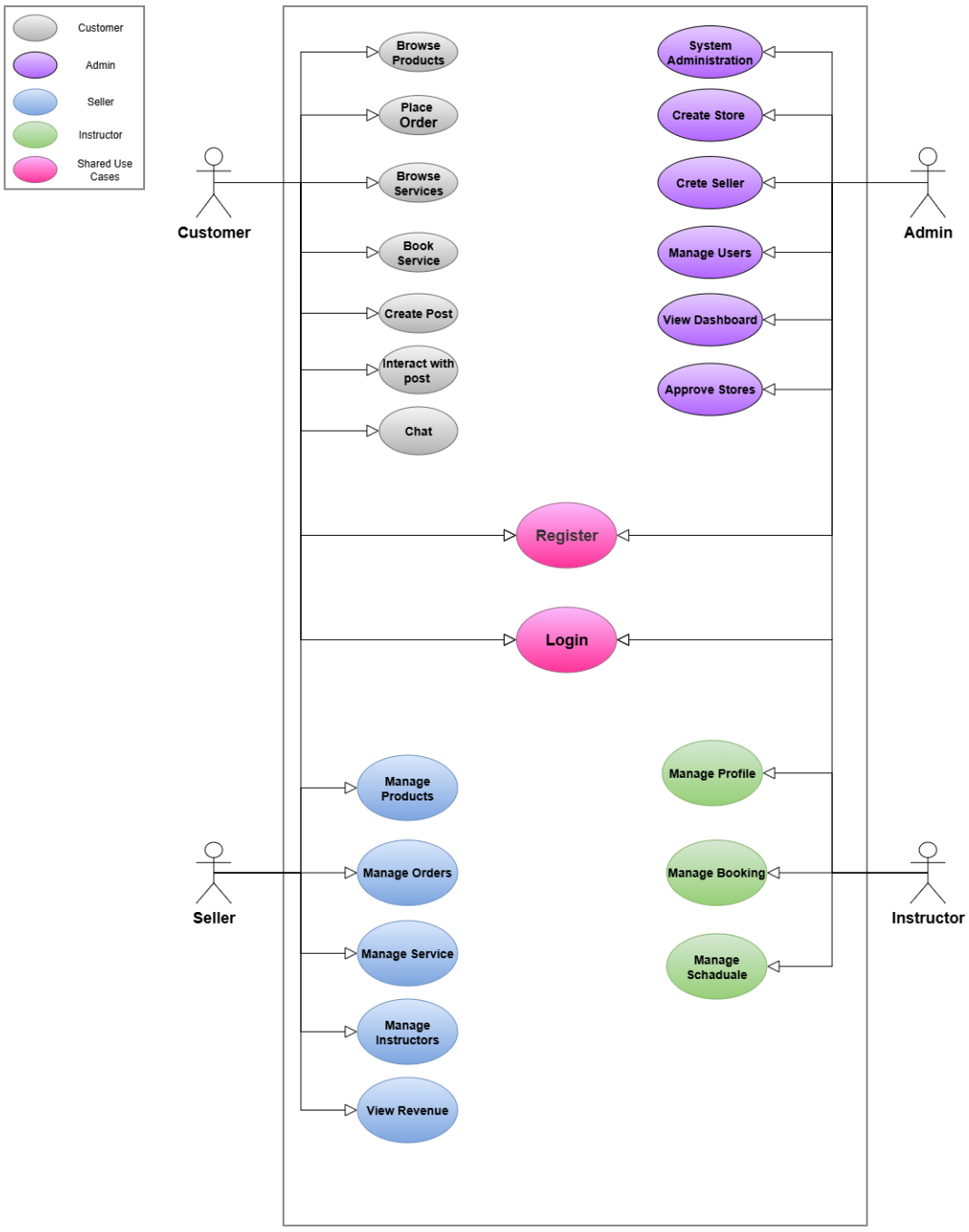
This diagram illustrates the main functionalities of the **Trabot** and how different users interact with the system.

It provides a high-level overview of the system's behavior from the perspective of its primary actors: **Customer**, **Seller**, **Admin and instructor**.

Users can be customers, sellers or instructors.

4.2.1 Use Case Diagram

The diagram below outlines the main use cases that define how each actor interacts with the system:


Customer
Admin
Seller
Instructor
Shared Use Cases
Customer
Admin
Seller
Instructor
Browse Products
Place Order
Browse Services
Book Service
Create Post
Interact with post
Chat
System Administration
Create Store
Crete Seller
Manage Users
View Dashboard
Approve Stores
Register
Login
Manage Products
Manage Orders
Manage Service
Manage Instructors
View Revenue
Manage Profile
Manage Booking
Manage Schaduale

4.2.2 Use Case Description

Use Case Name	Register
Actor	Customer, Seller, Instructor, Admin
Description	Allows a new user to create an account in the system.
Preconditions	User is not registered.
Main Flow	1. User opens registration page. 2. Enters required information. 3. Submits registration form. 4. System validates data. 5. Account is created successfully.
Postconditions	User account is created.

Use Case Name	Login
Actor	Customer, Seller, Instructor, Admin
Description	Allows users to access their accounts.
Preconditions	User has a valid account.
Main Flow	1. User enters credentials. 2. System validates credentials. 3. User is authenticated and redirected to dashboard.
Postconditions	User is logged in successfully.

Use Case Name	Browse Products
Actor	Customer
Description	Allows customers to browse available products.
Preconditions	Products exist in the system.
Main Flow	1. Customer opens products page.

	2. System displays available products. 3. Customer searches or filters products.
Postconditions	Customer views product information.

Use Case Name	Place Order
Actor	Customer
Description	Allows customers to purchase products.
Preconditions	Product is available.
Main Flow	1. Customer adds products to cart. 2. Proceeds to checkout. 3. Confirms order. 4. System creates order.
Postconditions	Order is recorded in the system.

Use Case Name	Browse Services
Actor	Customer
Description	Allows customers to browse available services.
Preconditions	Services exist.
Main Flow	Customer views and searches available services.
Postconditions	Service details are displayed.

Use Case Name	Book Service
Actor	Customer

Description	Allows customers to reserve a service.
Preconditions	Service is available.
Main Flow	1. Customer selects service. 2. Chooses date/time. 3. Confirms booking. 4. System records booking.
Postconditions	Booking is created.

Use Case Name	Create Post
Actor	Customer
Description	Allows customers to publish posts within the platform community.
Preconditions	User is logged in.
Main Flow	User creates and publishes a post.
Postconditions	Post becomes visible to other users.

Use Case Name	Interact with Post
Actor	Customer
Description	Allows customers to like, comment, or interact with posts.
Preconditions	Post exists.
Main Flow	User selects a post and performs an interaction.
Postconditions	Interaction is recorded.

Use Case Name	Manage Products
Actor	Seller
Description	Allows sellers to add, update, delete, and view products offered in their stores.

Use Case Name	Manage Orders
Actor	Seller
Description	Allows sellers to review, process, and update customer orders.

Use Case Name	Manage Services
Actor	Seller
Description	Allows sellers to create, update, and manage services offered through the platform.

Use Case Name	View Revenue
Actor	Seller
Description	Allows sellers to monitor sales performance and revenue statistics.

Use Case Name	Manage Profile
Actor	Instructor
Description	Allows instructors to update personal information and account details.

Use Case Name	Manage Booking
Actor	Instructor
Description	Allows instructors to view, accept, reject, and manage service bookings.

Use Case Name	Manage Schedule
Actor	Instructor
Description	Allows instructors to create and update their availability schedule.

Use Case Name	System Administration
Actor	Admin
Description	Allows administrators to oversee and manage overall system operations.

Use Case Name	Create Store
Actor	Admin
Description	Allows administrators to create new stores within the platform.

Use Case Name	Create Seller
Actor	Admin
Description	Allows administrators to create seller accounts and assign them to stores.

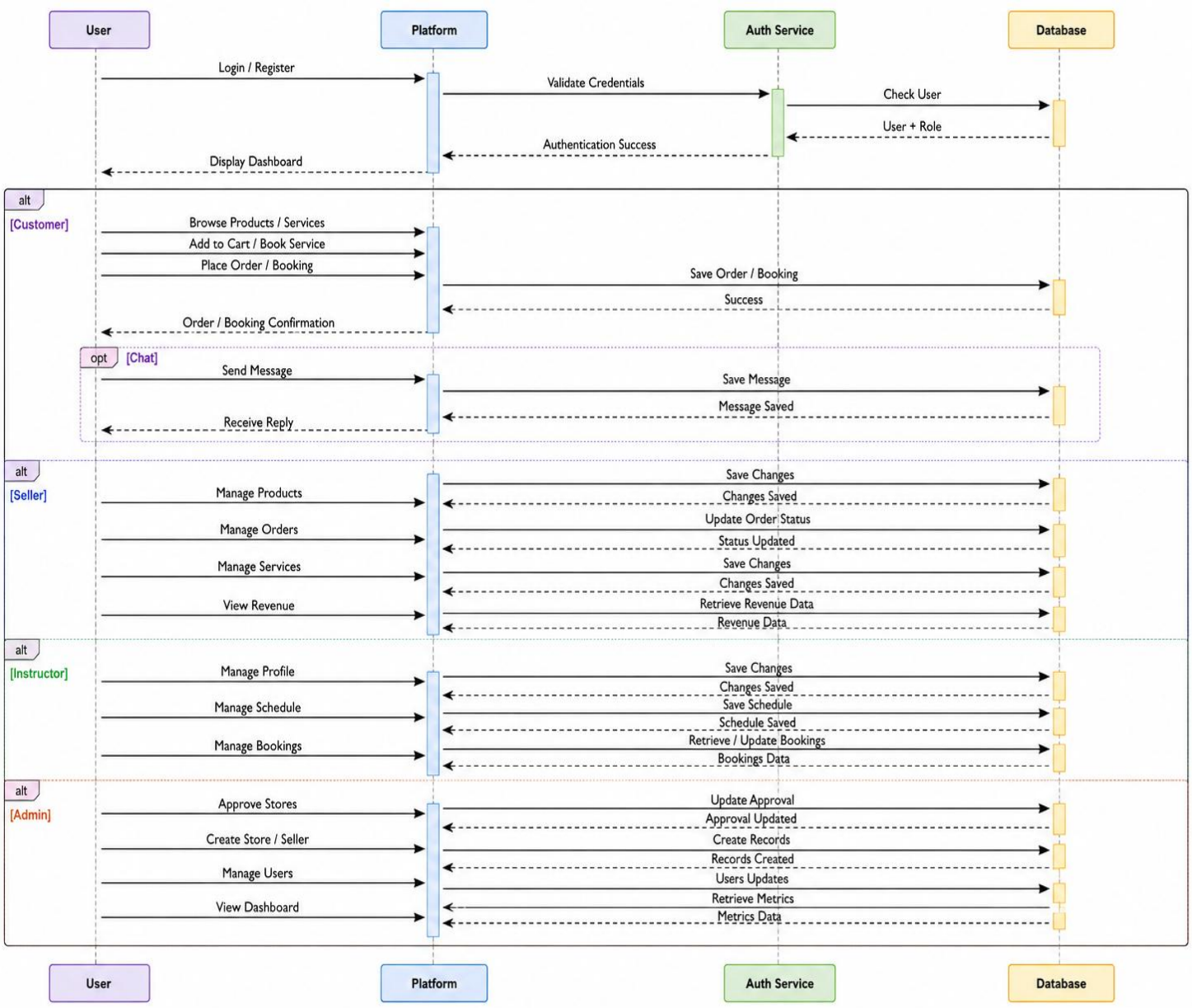
Use Case Name	Manage Users
Actor	Admin
Description	Allows administrators to manage user accounts, permissions, and system access.

Use Case Name	View Dashboard
Actor	Admin
Description	Allows administrators to monitor platform statistics and system performance.

Use Case Name	Approve Stores
Actor	Admin
Description	Allows administrators to review and approve store registration requests before activation.

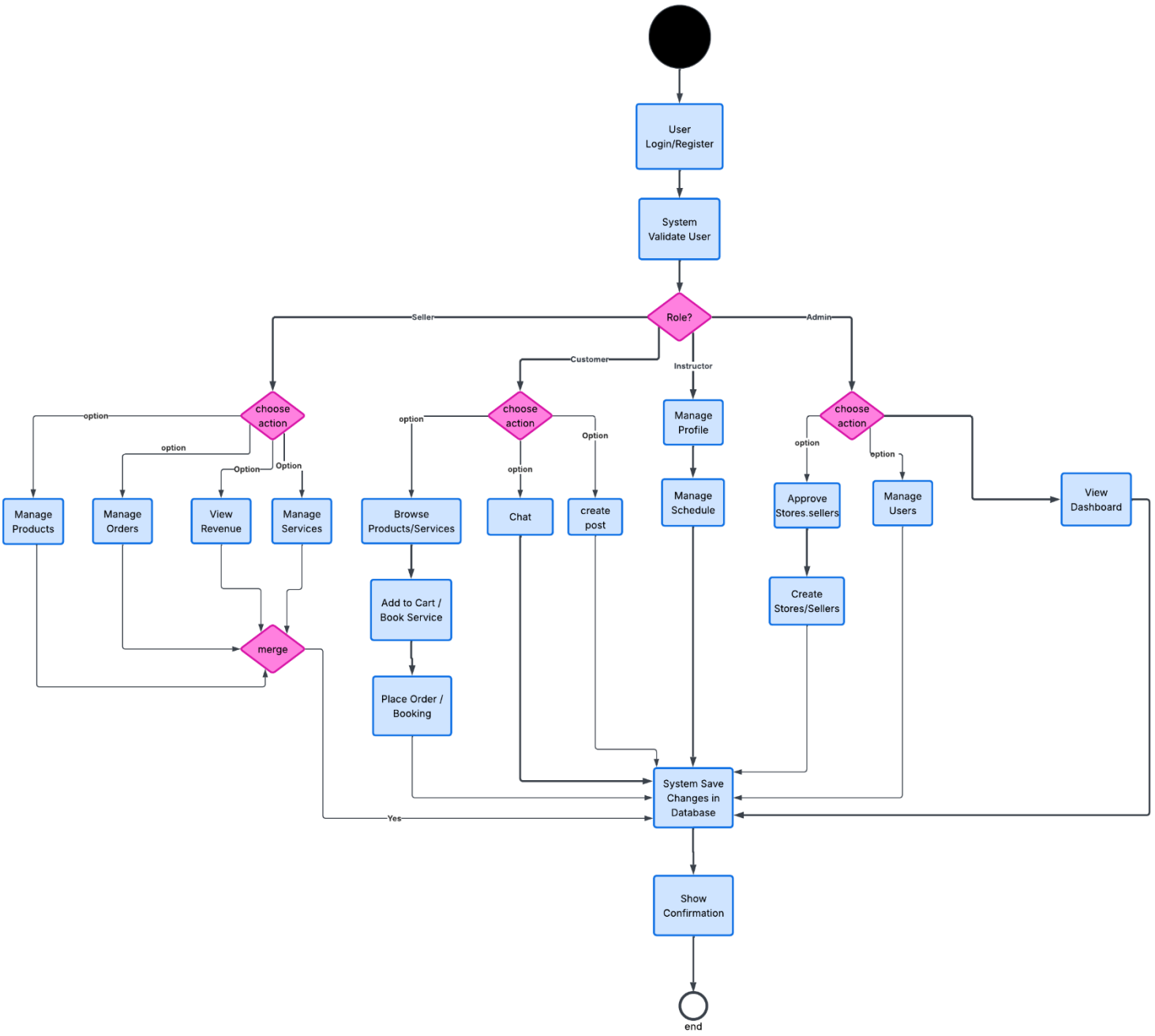
4.3 Sequence Diagram

The sequence diagram below illustrates the dynamic interaction between actors and the platform:



4.4 Activity Diagram

The activity diagram represents the flow of main activities or actions within a system:



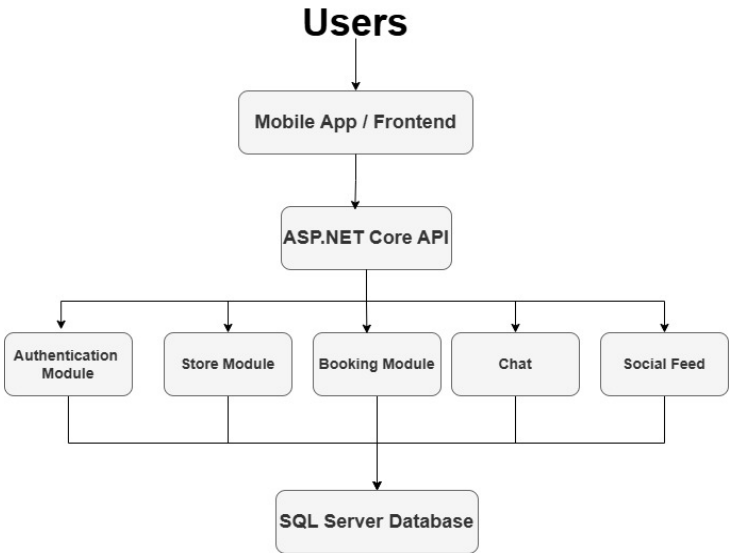
CHAPTER 5— SYSTEM DESIGN

5.1 Introduction

This chapter presents the overall system design of the Trabot. It focuses on the technical architecture and structural components that enable the platform to support efficient community based platform. The primary objective of this design is to ensure that the system is scalable, secure, modular, and easy to maintain, while delivering a seamless and reliable user experience across both web and mobile platforms.

5.2 Architectural design

The system architecture is designed to handle multiple user roles—Customer, Seller, service provider and Admin—and to support smooth interaction between the frontend, backend services, and database layers. It provides a clear blueprint for designing, developing, and maintaining the platform efficiently. Trabot follows a modular and layered architecture that ensures scalability, maintainability, and smooth integration between different system parts.



The Trabot system is built using a **three-tier architecture model**, separating the presentation layer, application logic, and data management. This approach supports multiple user roles—**Customer, Seller, service provider and Admin**—while enabling seamless interaction across web and mobile platforms.

5.2.1 Presentation Layer (Frontend)

- Responsible for delivering an intuitive and user-friendly interface for all users.
- Developed using **Flutter** for the mobile application and **React** for the web application.
- Fully responsive design to support multiple devices and screen sizes.
- Key features include:
    - Browsing and searching products across different categories.
    - Booking service that the user needs.
    - Interact with posts and communicate.
    - Managing customer accounts, orders, and bookings.
    - Providing admin with dashboard for monitoring sellers, markets, and services.
    - Interacting with the **AI** for help, suggestions and recommendations.
    - Enabling secure authentication and role-based access for customers, sellers, service provider and admins.

5.2.2 Application Layer (Backend)

Built using ASP.NET Core to handle all business logic and system operations.

• Acts as a bridge between the frontend and data layer.

• Responsibilities include:

- **User authentication and authorization**, supporting roles like admin and customer.
- **Shops and products management**
- **Booking operations**
- **Admin panel** for moderating content, handling reports, and managing system users.

5.2.3 Data Layer

The Data Layer is implemented using a secure and reliable **SQL Server** relational database to store, manage, and organize all application data. It ensures data consistency, integrity, and efficient querying to support system performance and analytics.

The data layer is designed with scalability in mind, allowing the system to efficiently support a growing user base and future analytical requirements.

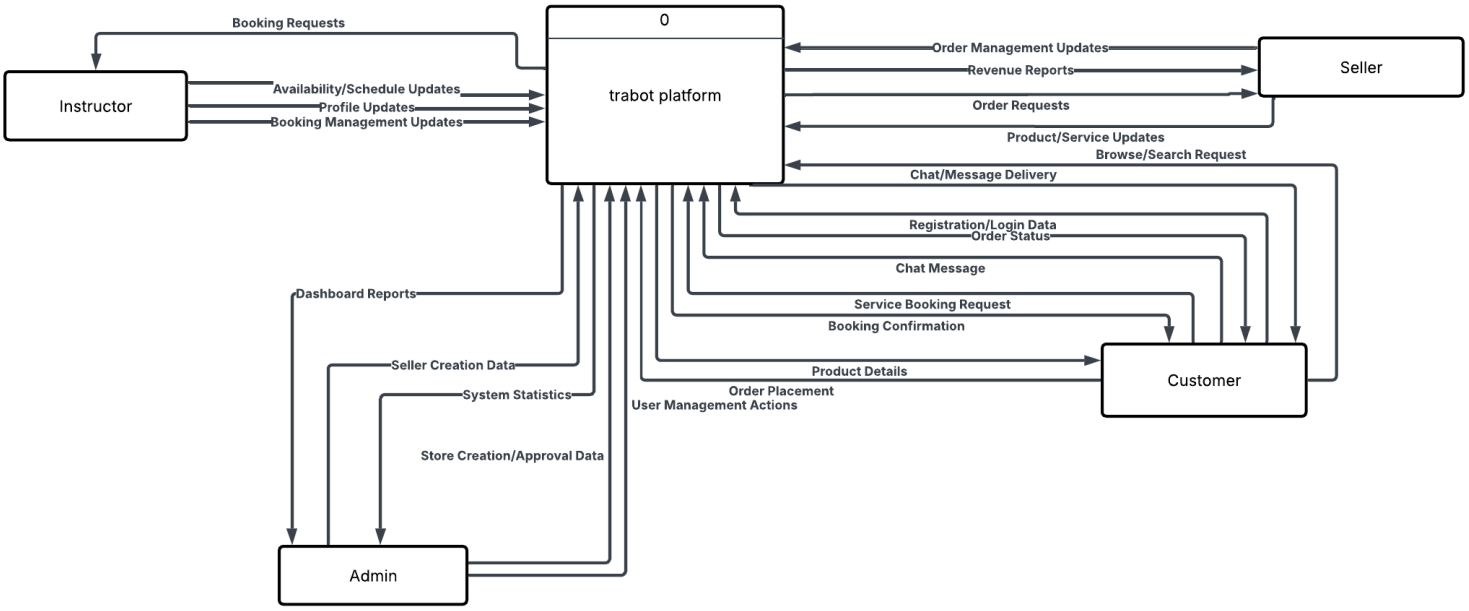
5.3 Object design

The object-oriented design of the **Trabot** focuses on defining internal system structures that support modularity, reusability, and clarity. This design phase translates functional requirements into structured components that interact to fulfill system behavior. The objective is to define how the system's objects (users, packages, bookings, etc.) collaborate to provide an efficient and scalable solution.

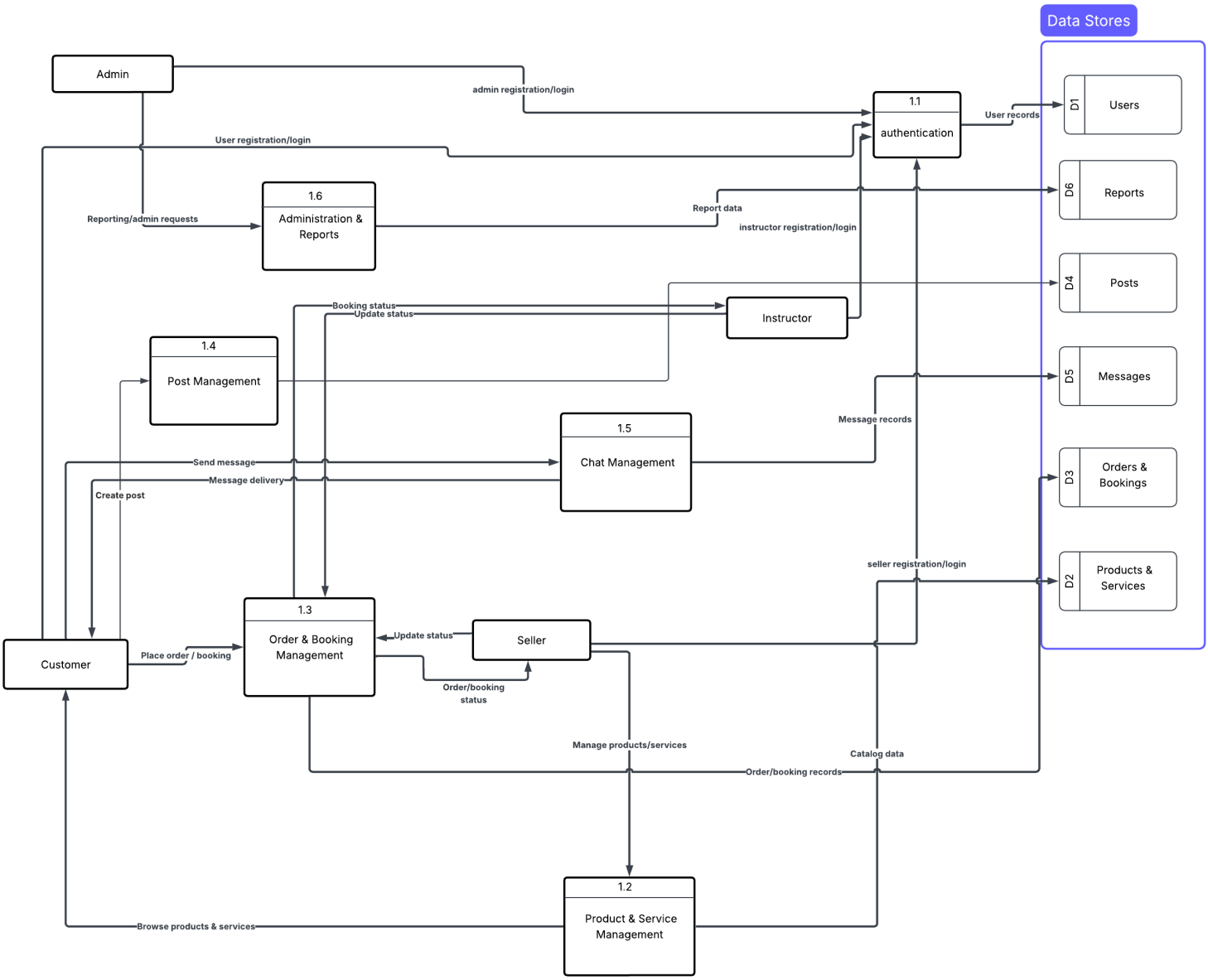
5.3.1 Data Flow Diagram (DFD)

The Data Flow Diagram illustrates how data moves through the community system.
It identifies data sources, processes, data stores, and data destinations.

- **Context diagram**

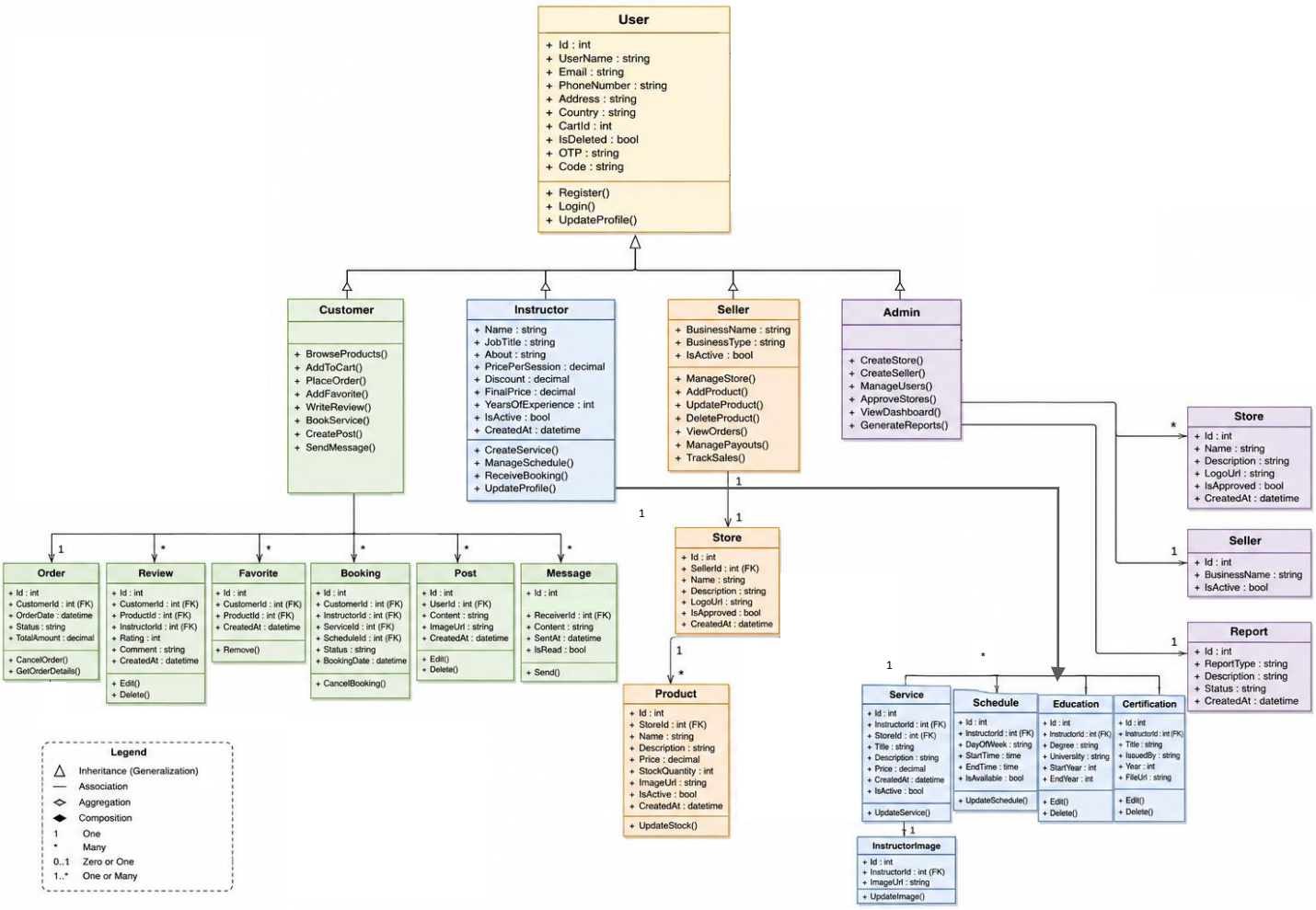


➢ Level 1


Data Stores
Admin
admin registration/login
1.1
authentication
User records
D1
Users
User registration/login
1.6
Administration & Reports
Reporting/admin requests
Report data
instructor registration/login
D6
Reports
D4
Posts
Booking status
Update status
Instructor
1.4
Post Management
D5
Messages
Message records
1.5
Chat Management
Send message
Message delivery
Create post
D3
Orders & Bookings
seller registration/login
D2
Products & Services
1.3
Order & Booking Management
Update status
Seller
Customer
Place order / booking
Order/booking status
Manage products/services
Catalog data
Order/booking records
1.2
Product & Service Management
Browse products & services

5.3.2 Class Diagram

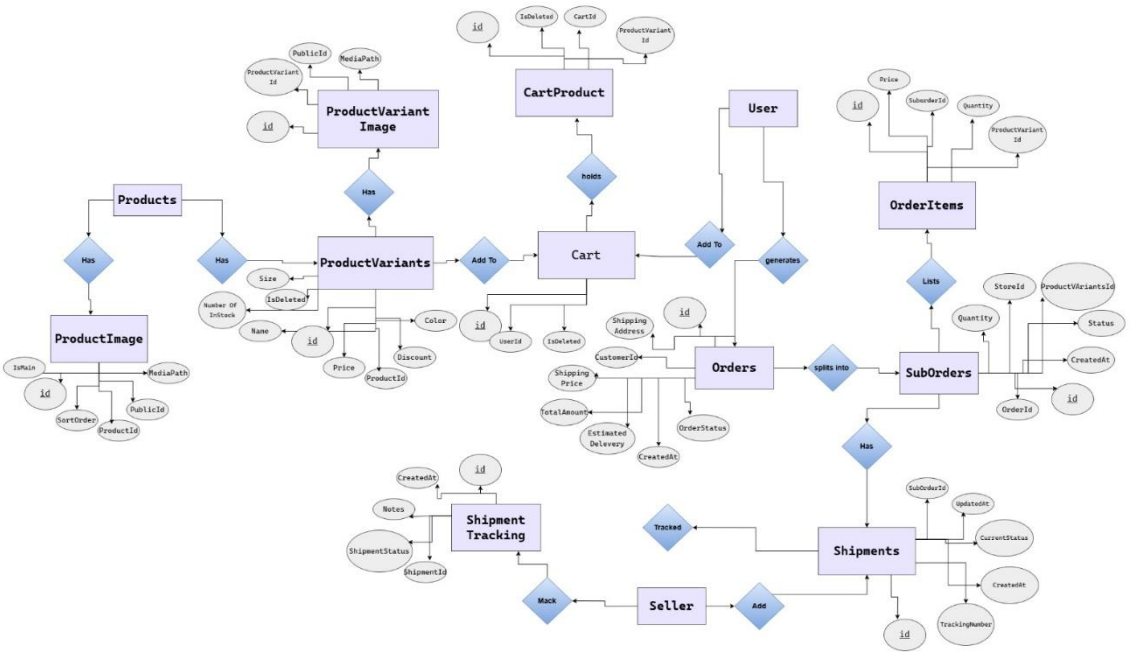
The Class Diagram represents the static structure of the community system.
It shows the system classes, their attributes, methods, and relationships.



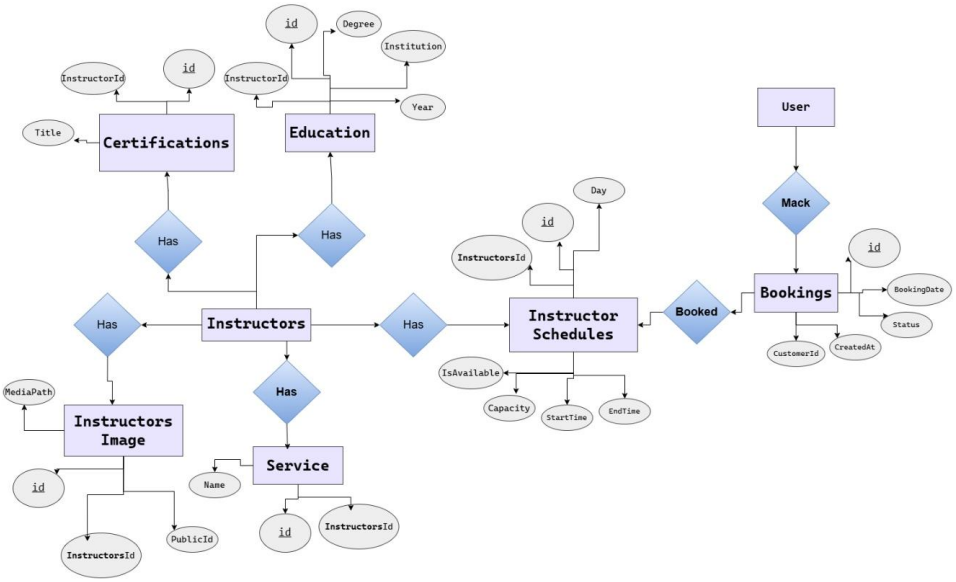
5.3.3 Entity Relationship Diagram (ERD)

This ERD represents a **Trabot system** that manages users, stores, products, orders, bookings and posts.

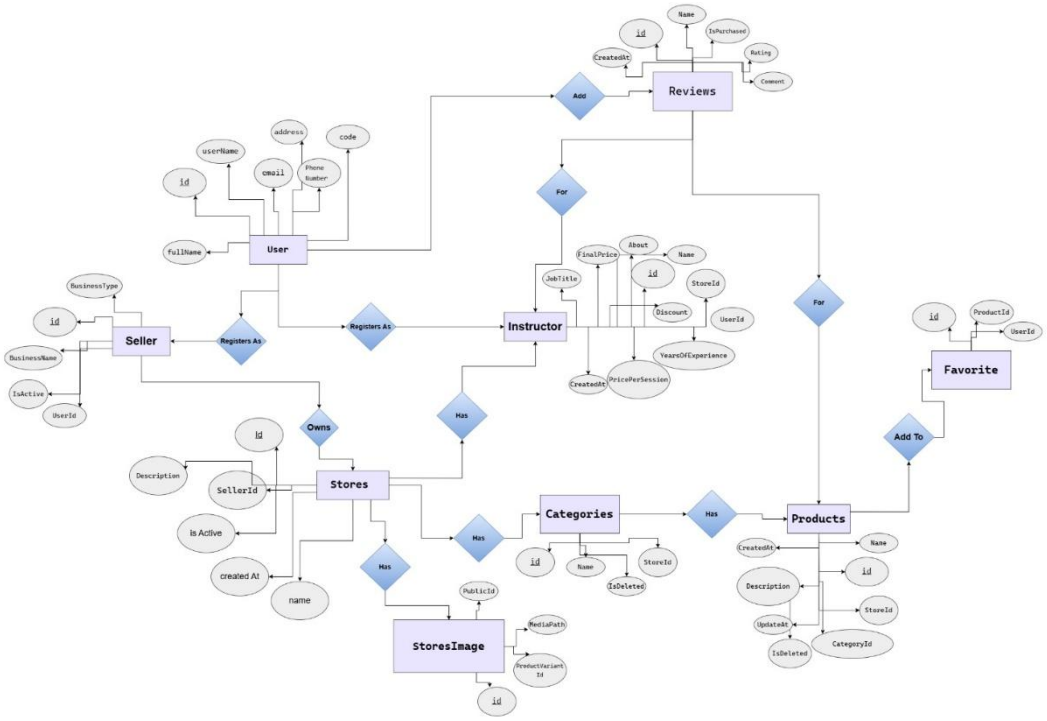
- **Product, Cart & Order Management**



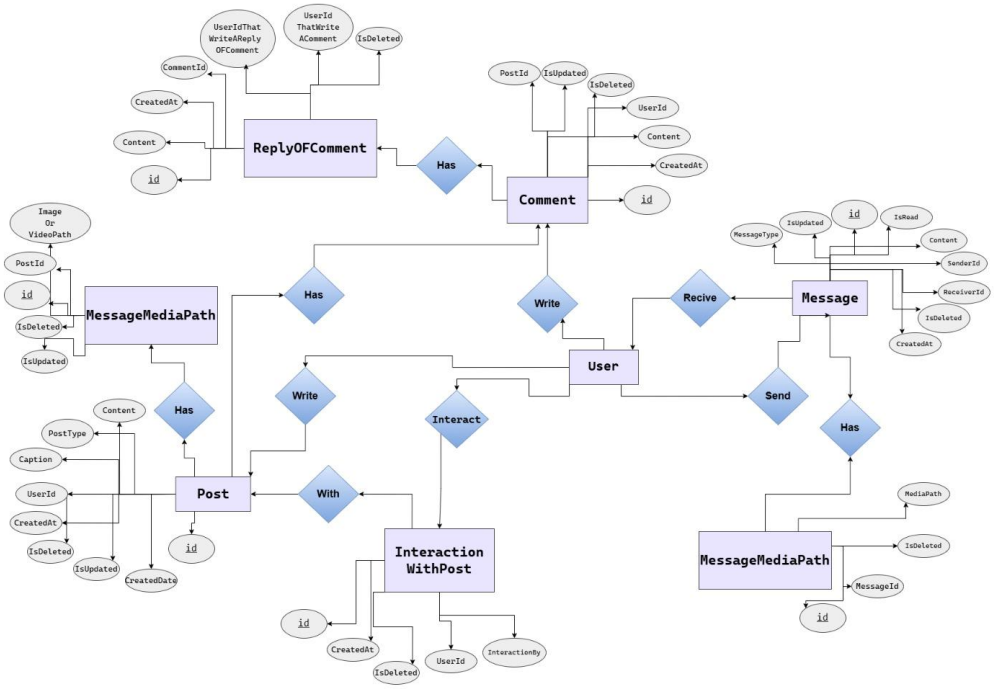
- **Booking & Scheduling Management**



- **User, Seller & Instructor Management**



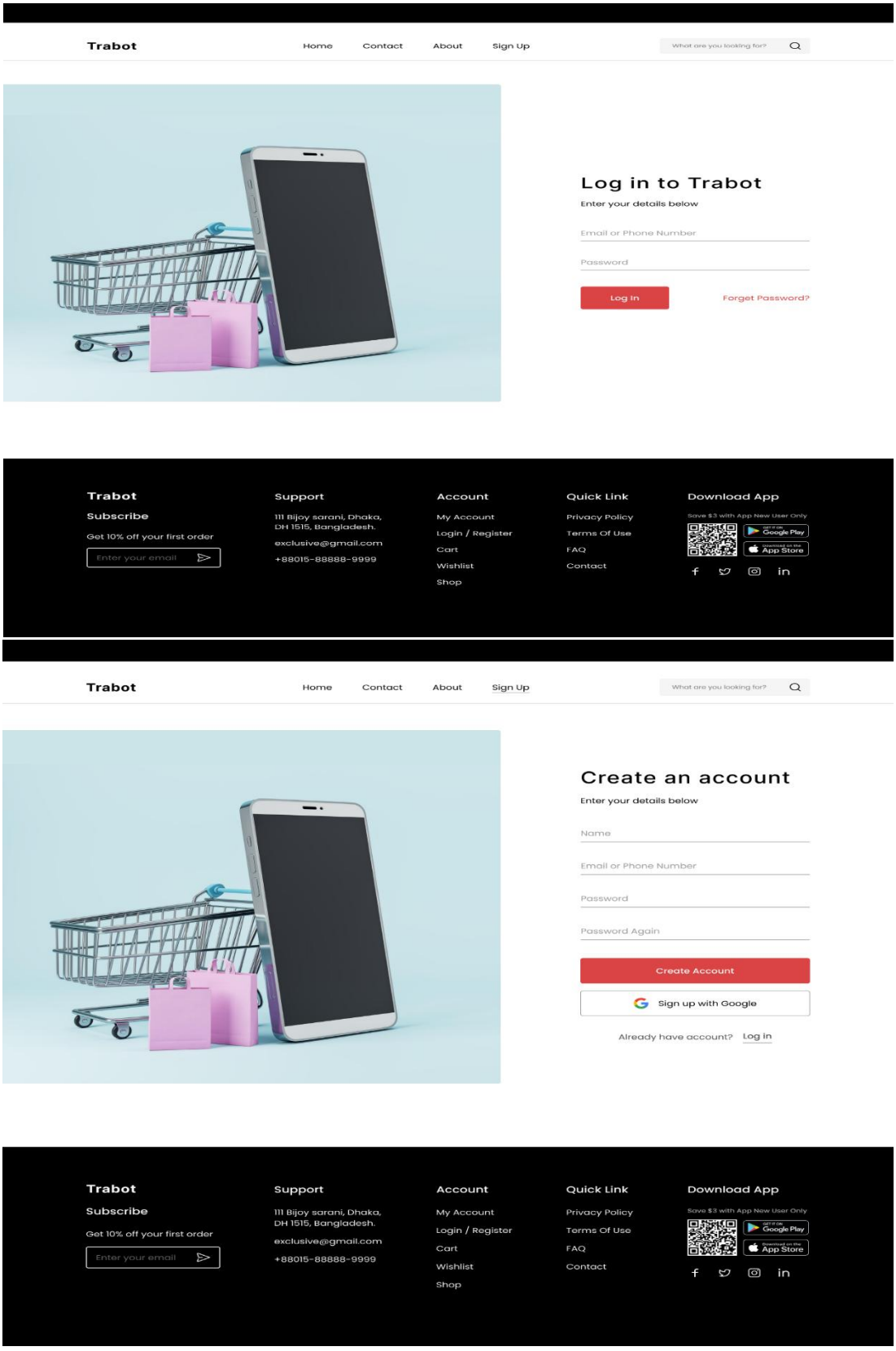
- **Social & Chat Management**



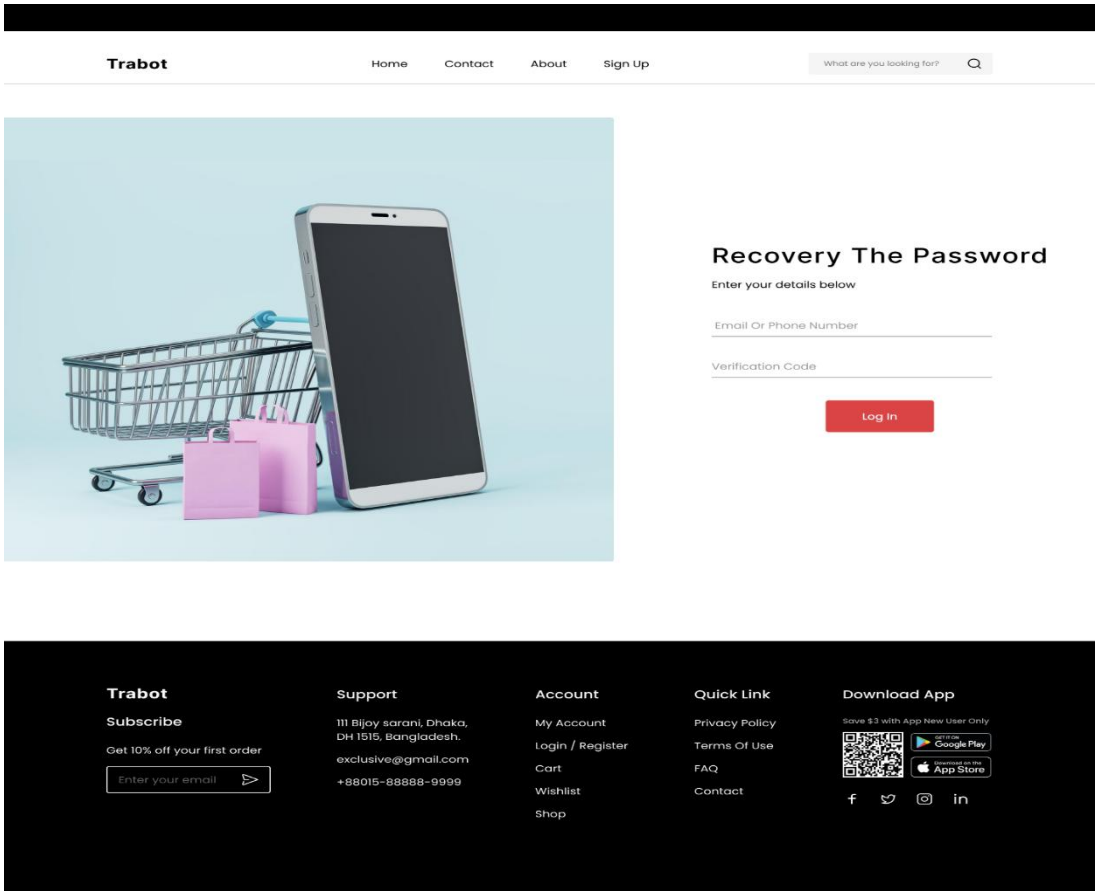
5.4 User Interface Design

5.4.1 Web Site

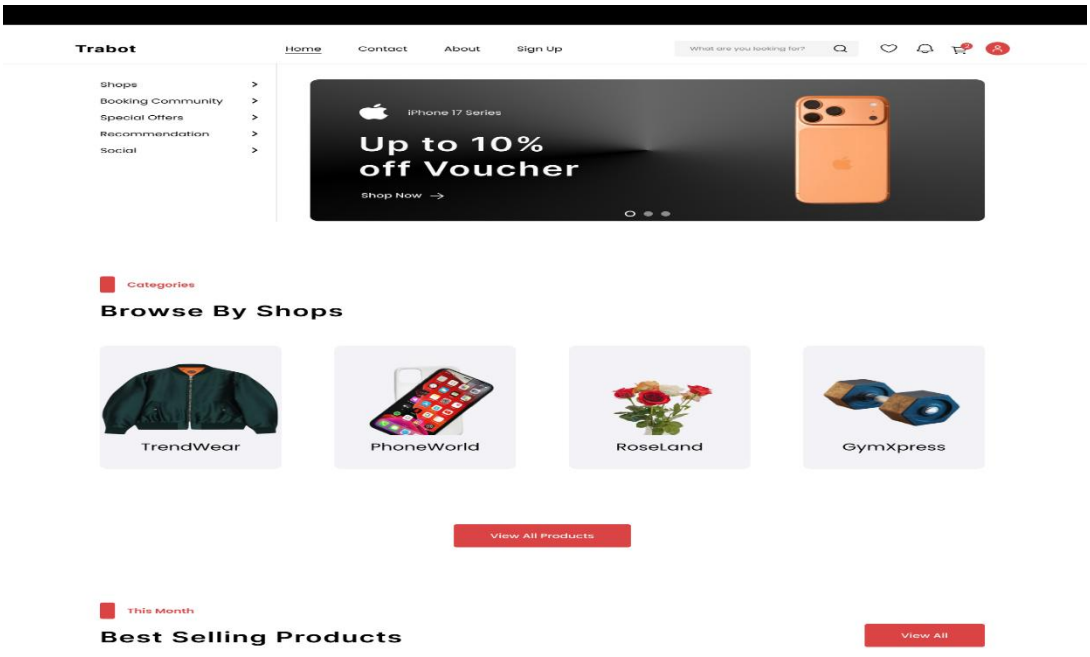
- **User Interface design for Authentication**



➢ User Interface design for Password recovery



➢ User Interface design Home page



The north coat
$260 $360
★★★★★ (65)

Apple Iphone 17
$500 $650
★★★★★ (65)

Fresh Cut Red Roses
$100 $150
★★★★★ (65)

HOMCOM Set of 2 x 10 kg
$250 $300
★★★★★ (65)

FREE AND FAST DELIVERY
Free delivery for all orders over $140

24/7 CUSTOMER SERVICE
Friendly 24/7 customer support

MONEY BACK GUARANTEE
We reurn money within 30 days

Booking Community

Book Your Appointment

Doctor Appointment
Consult with certified specialists
Select Service

Gym Session
Personal & group training sessions
Select Service

Online Courses
Learn anytime, anywhere
Select Service

Photography Session
Portraits, products & events
Select Service

50K+	5,000+	4.8 ★	98%
Total Bookings	Professionals	Average Rating	Satisfaction Rate

Special Offers

Limited Time Deals You Don't Want To Miss

Limited Time
New Year Health Package
Complete health checkup with 3 specialist consultations
$199 $270
Save 33% Off
Book Now

Popular
3-Month Gym Membership
Unlimited access + 2 personal training sessions per month
$299 $399
Save 34% Off
Book Now

New
Learn & Grow Package
Access to 3 courses of your choice + certificates
$247 $320
Save 32% Off
Book Now

Best Value
Photography Bundle
2-hour session + 50 edited photos + album
$345 $477
Save 33% Off
Book Now

Recommendation

Recommended For You

Most Booked This Week
- Personal Training Sessions
- Web Development Course
- General Health Checkup

View All

Highly Rated Services
- Wedding Photography
- Yoga Classes
- Data Science Course

View All

Quick Availability
- Group Fitness Today
- Portrait Photography
- Digital Marketing Course

View All

Why Book With Us?

Benefits you'll love

Flexible Scheduling
Book anytime that works for you

Instant Confirmation
Get confirmed in seconds

Top Rated Services
Quality guaranteed

Exclusive Deals
Save with special offers

Trabot
Subscribe

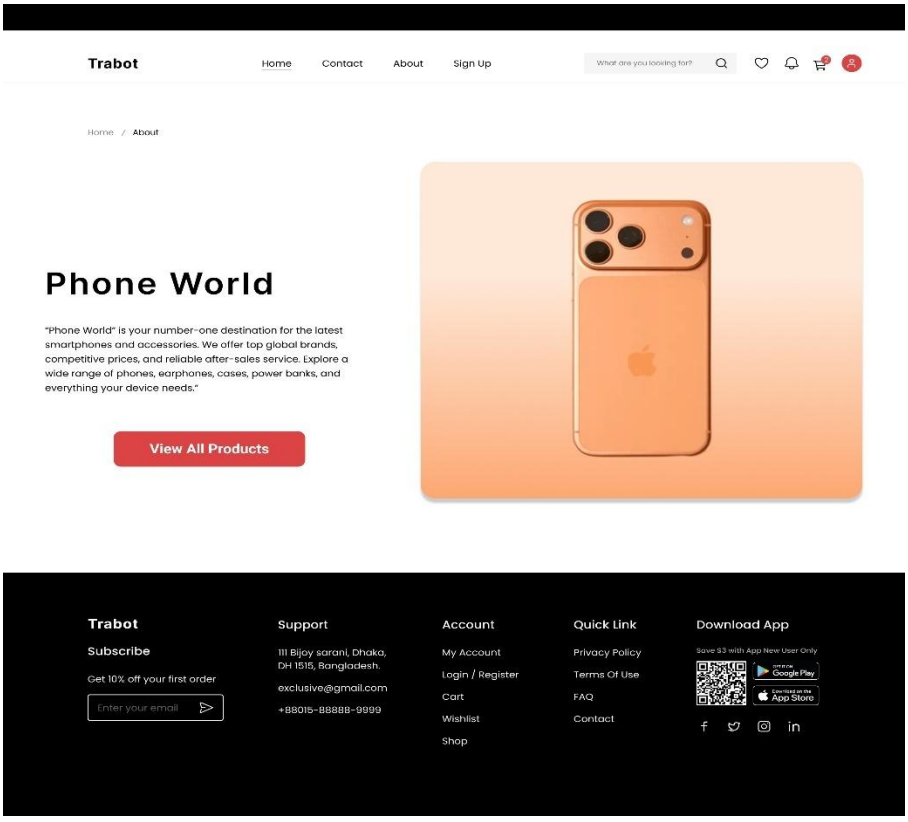
Support
111 Bijoy sarani, Dhaka,

Account
My Account

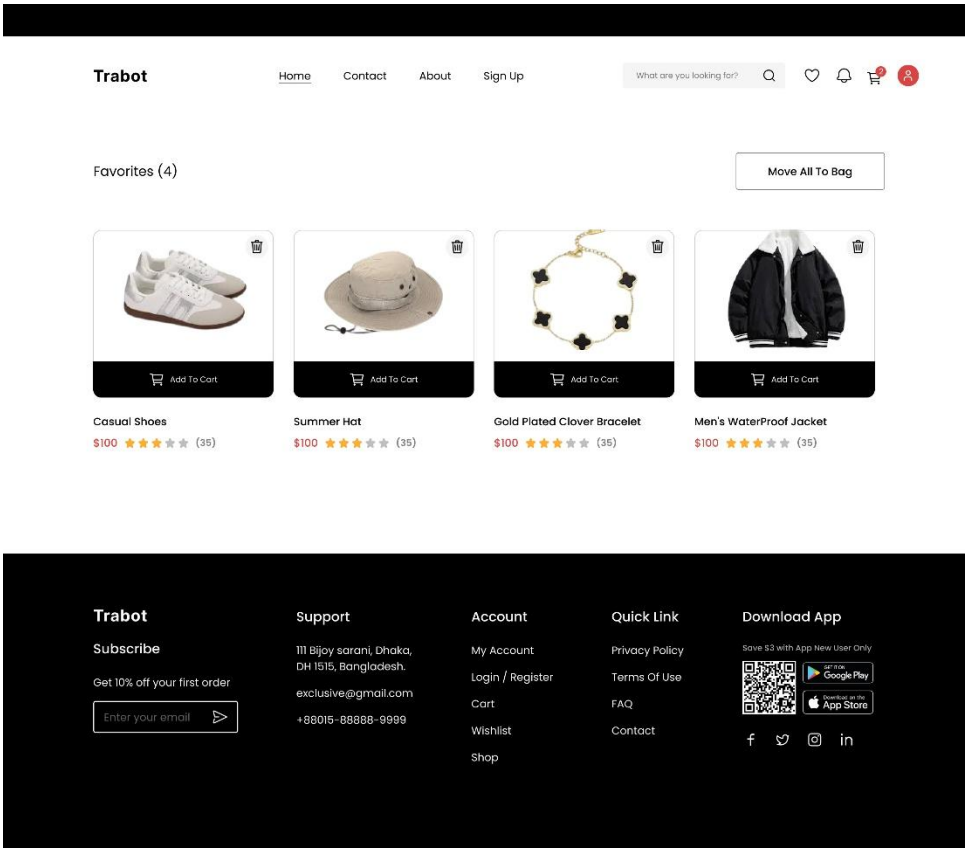
Quick Link
Privacy Policy

Download App
Save $3 with App New User Only

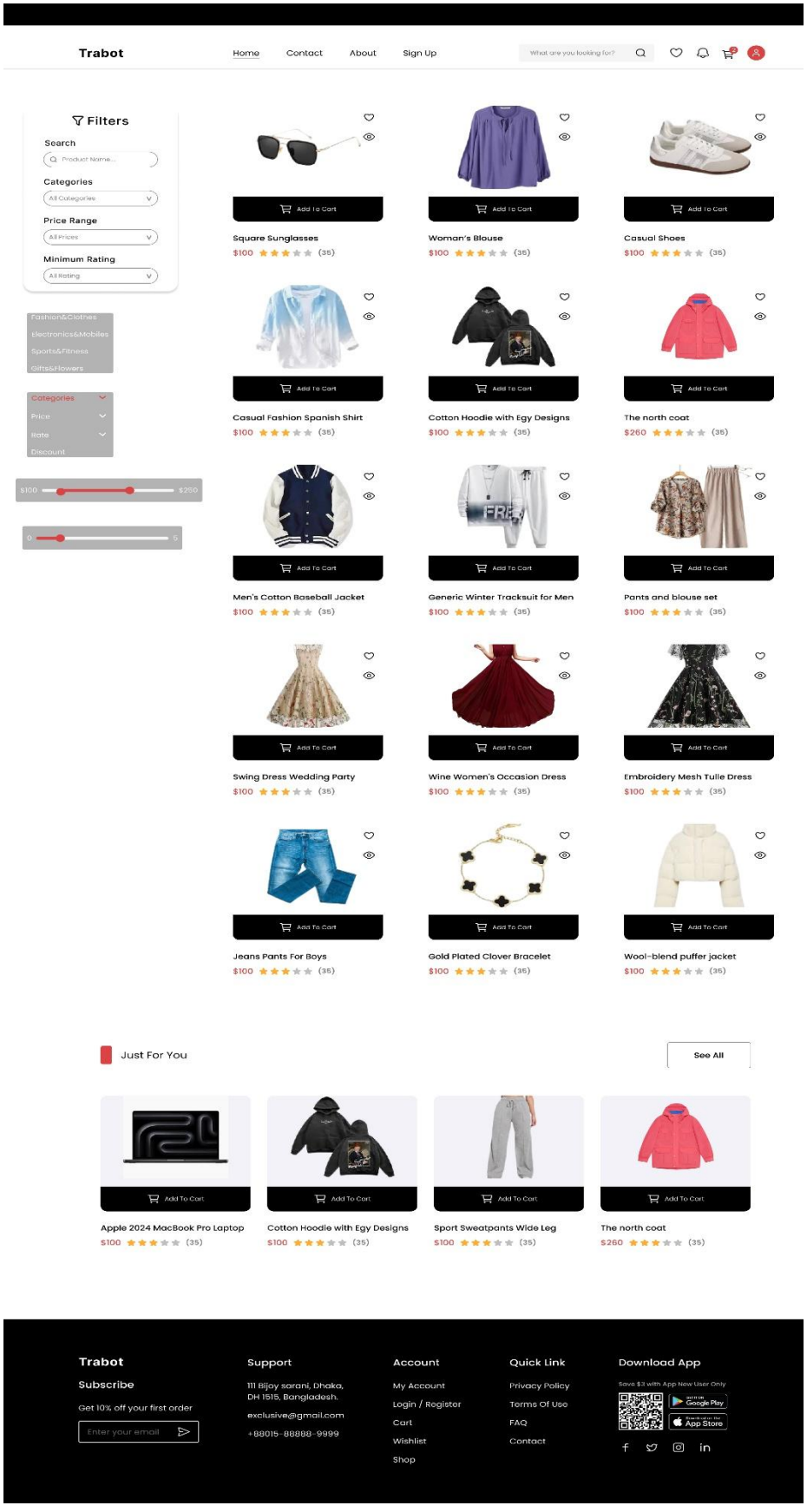
➢ User Interface design for shop description



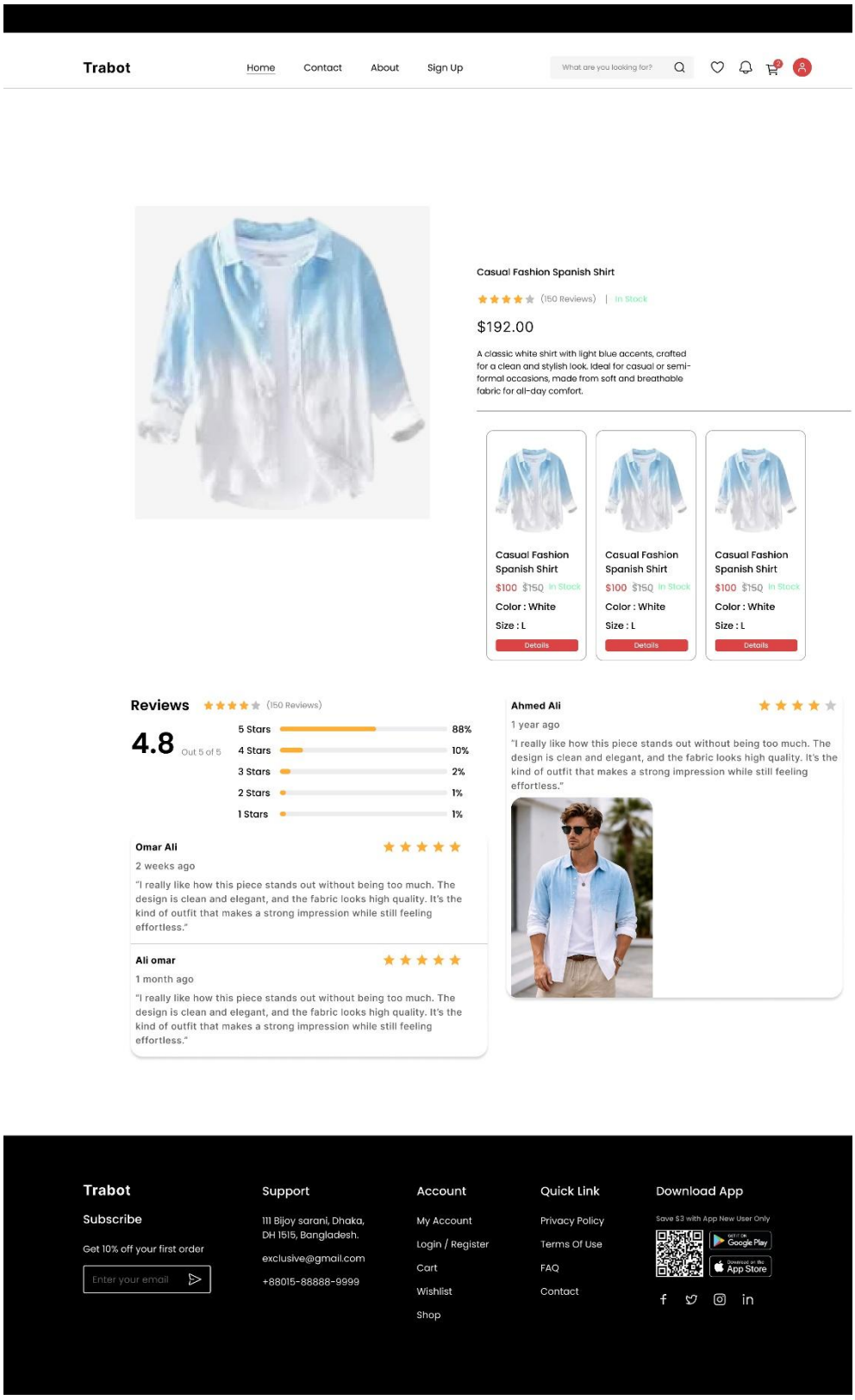
➢ User Interface design for favorites



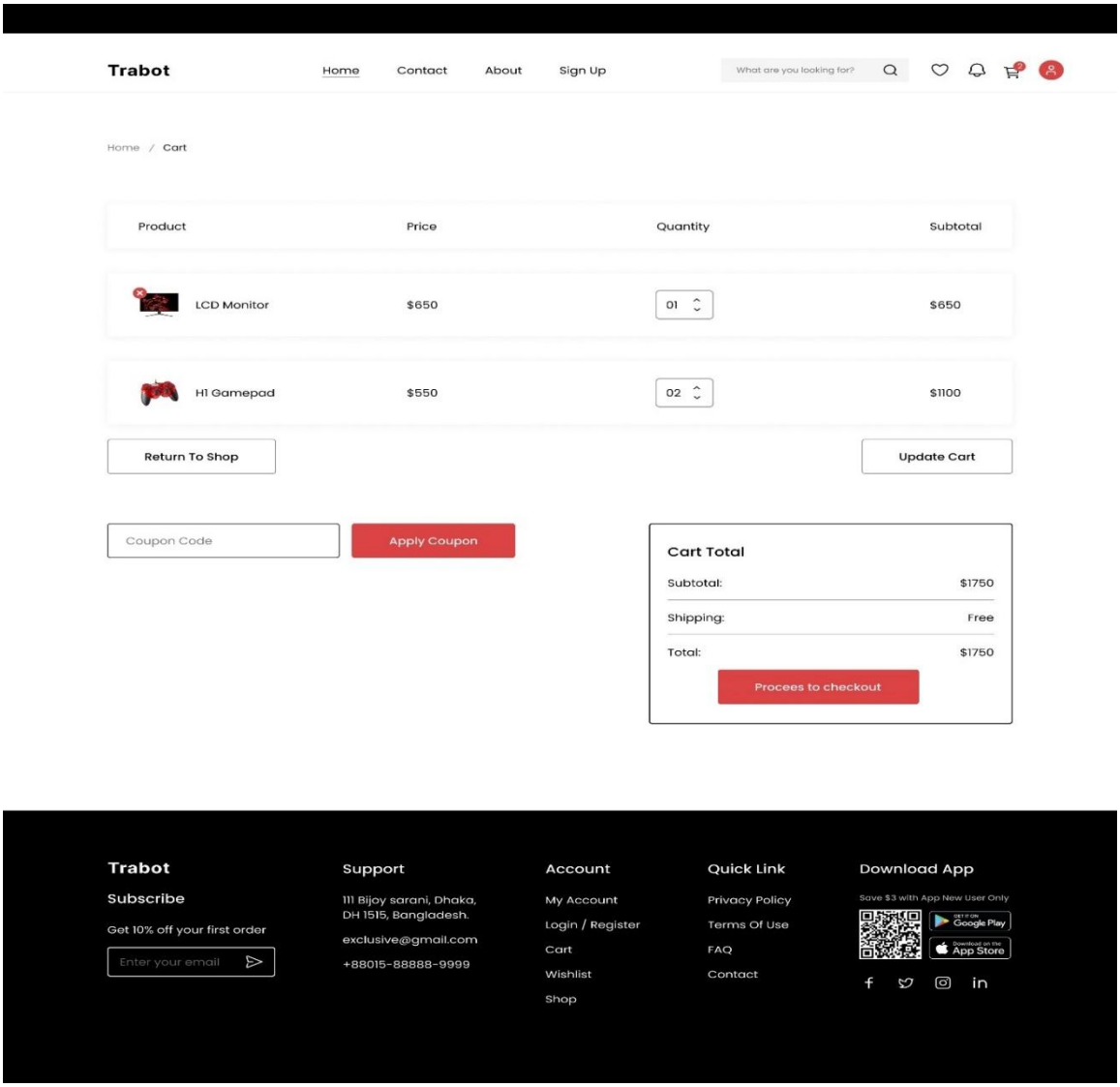
➢ User Interface design for shop products



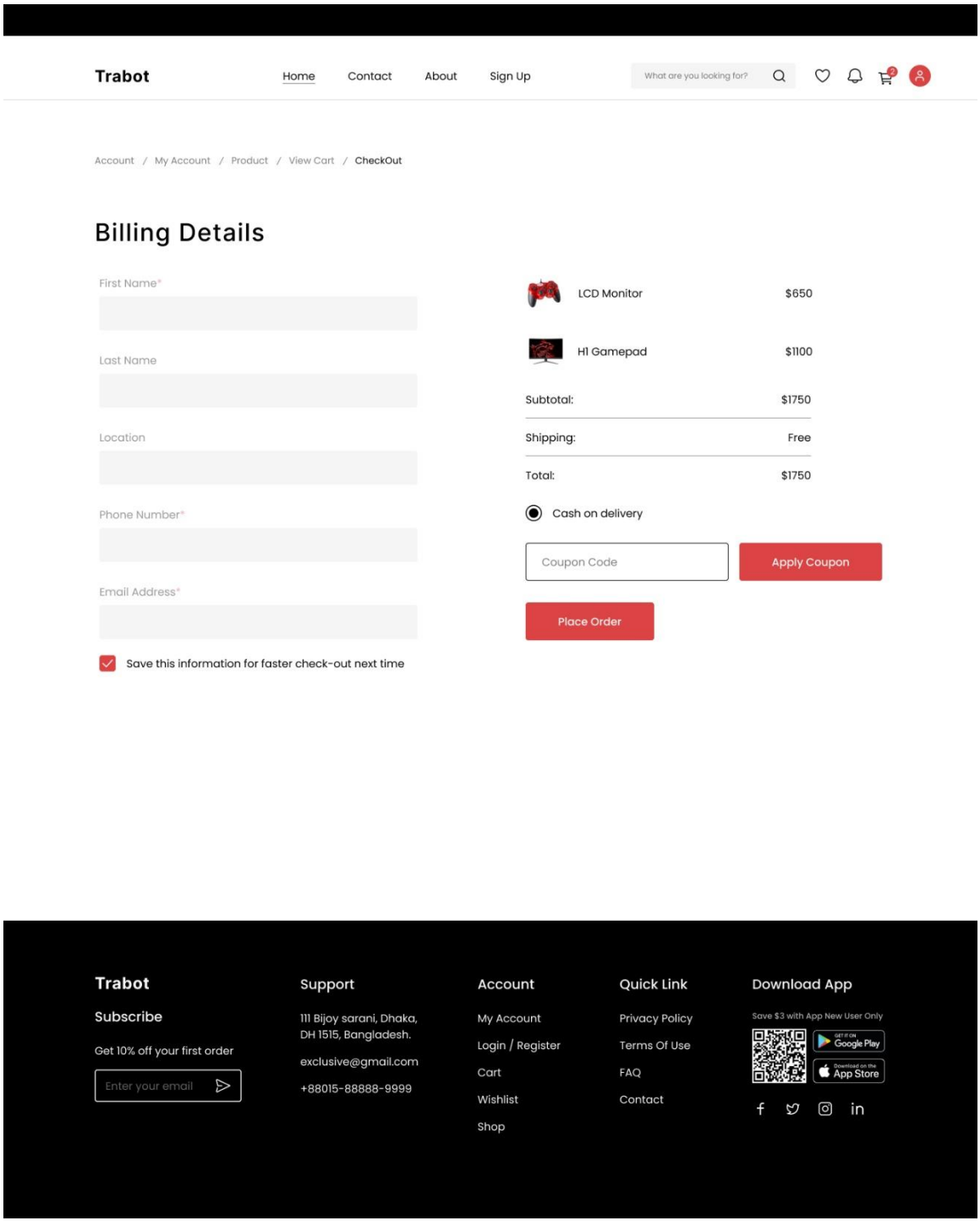
➢ User Interface design for shop product details



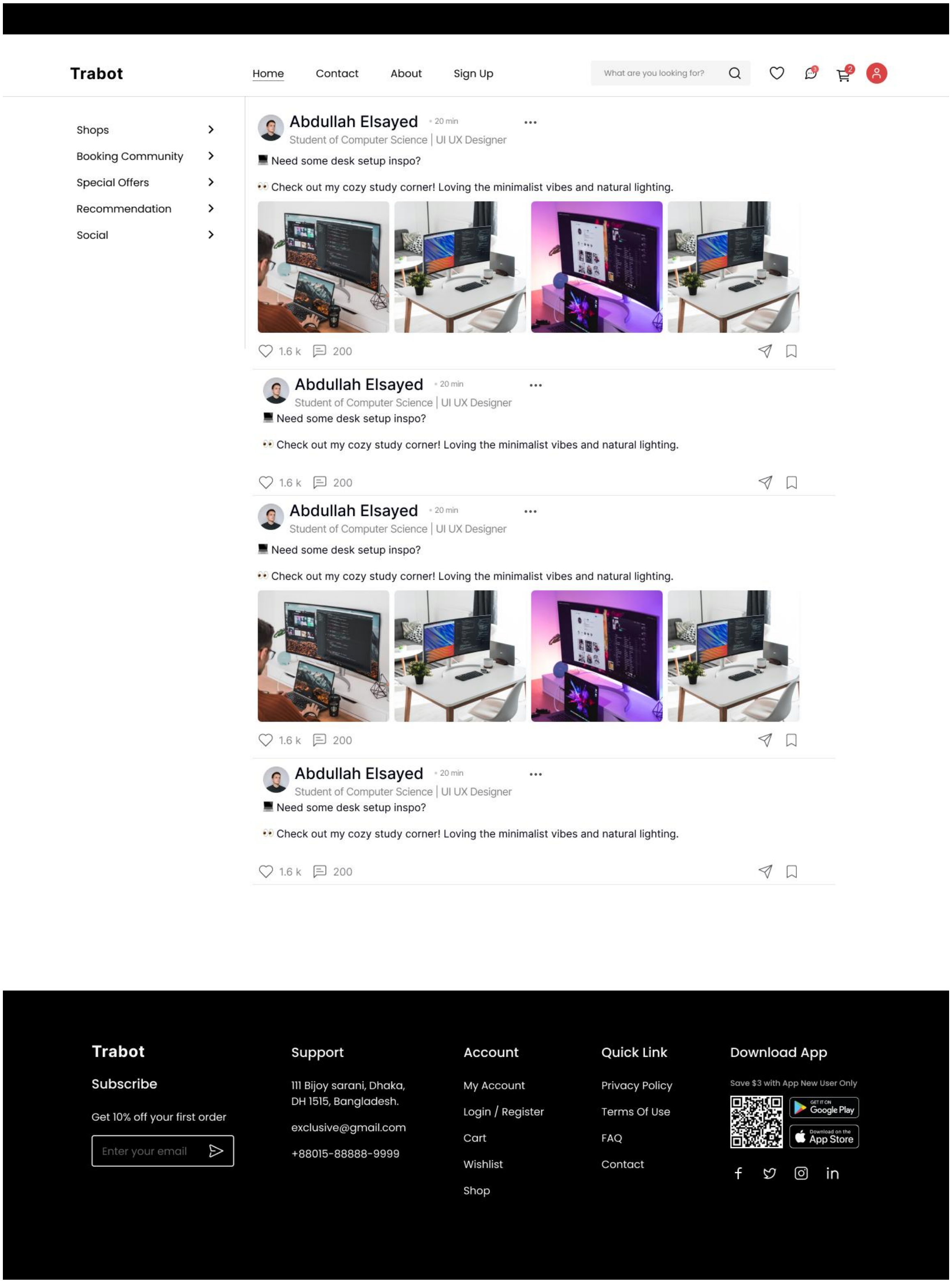
- **User Interface design for cart**



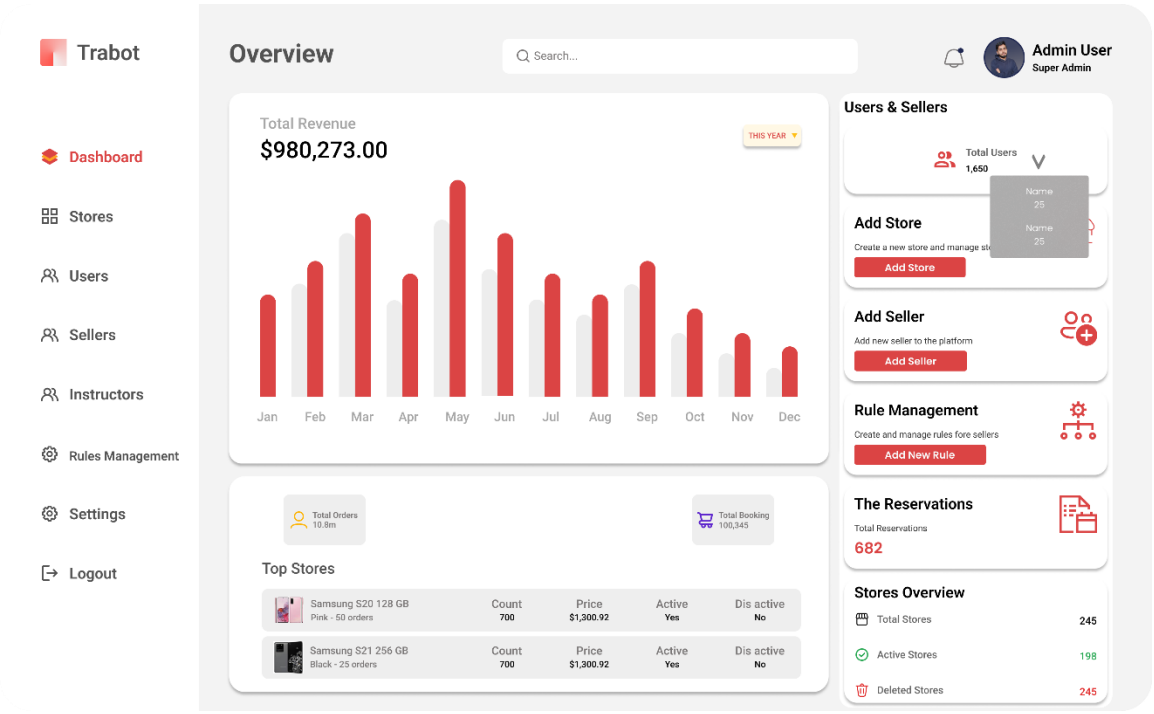
User Interface design for check out

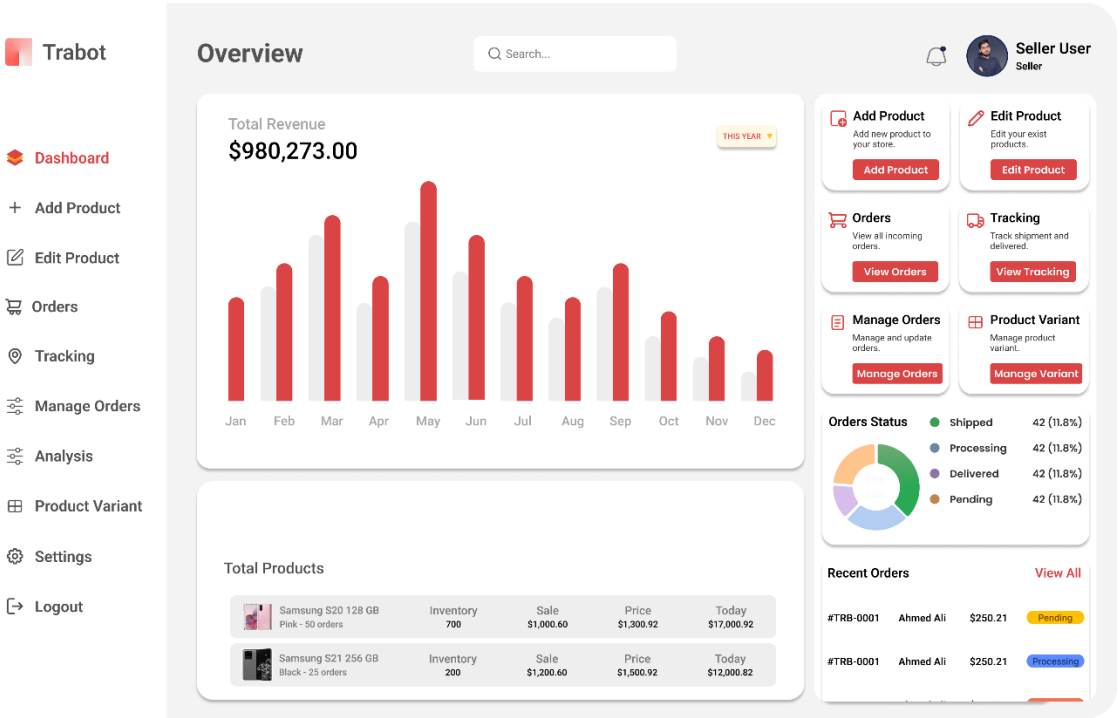


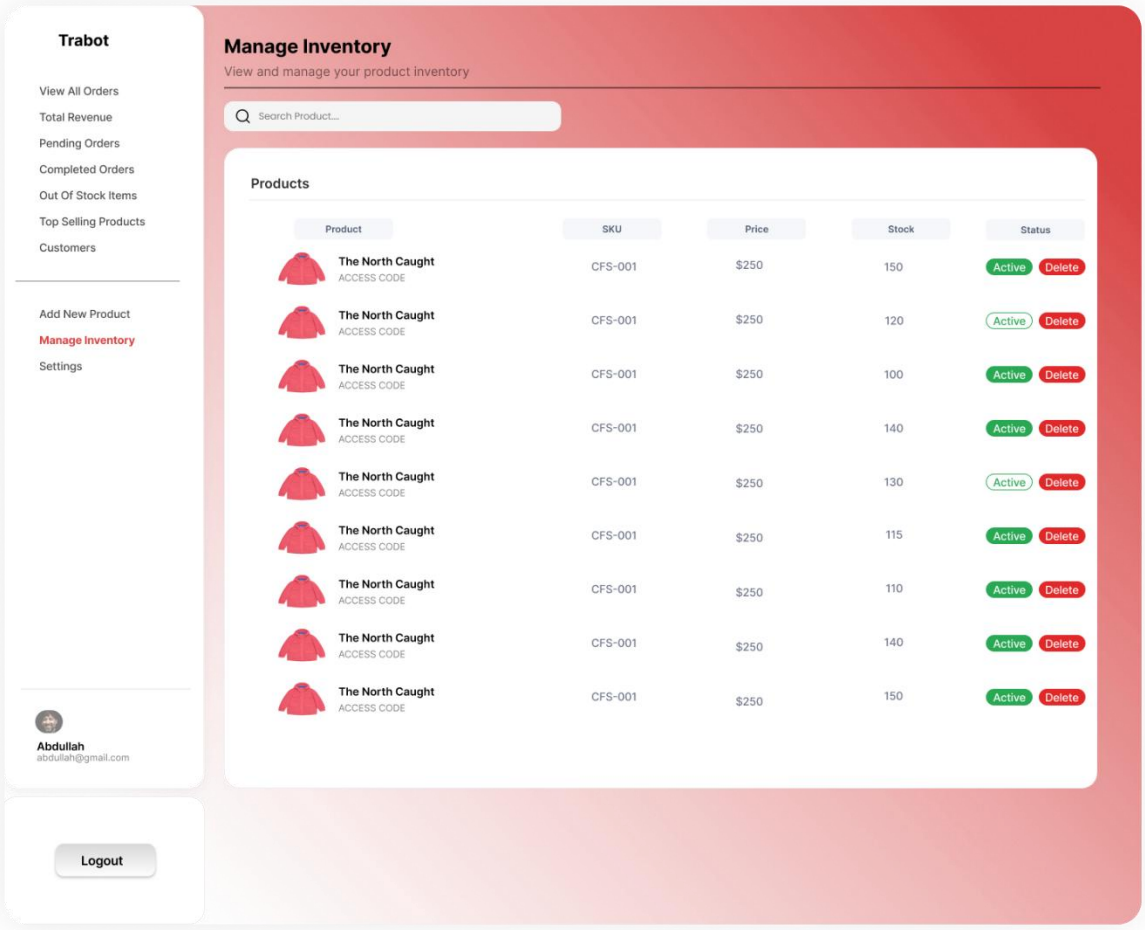
➢ <u>User Interface design for social</u>

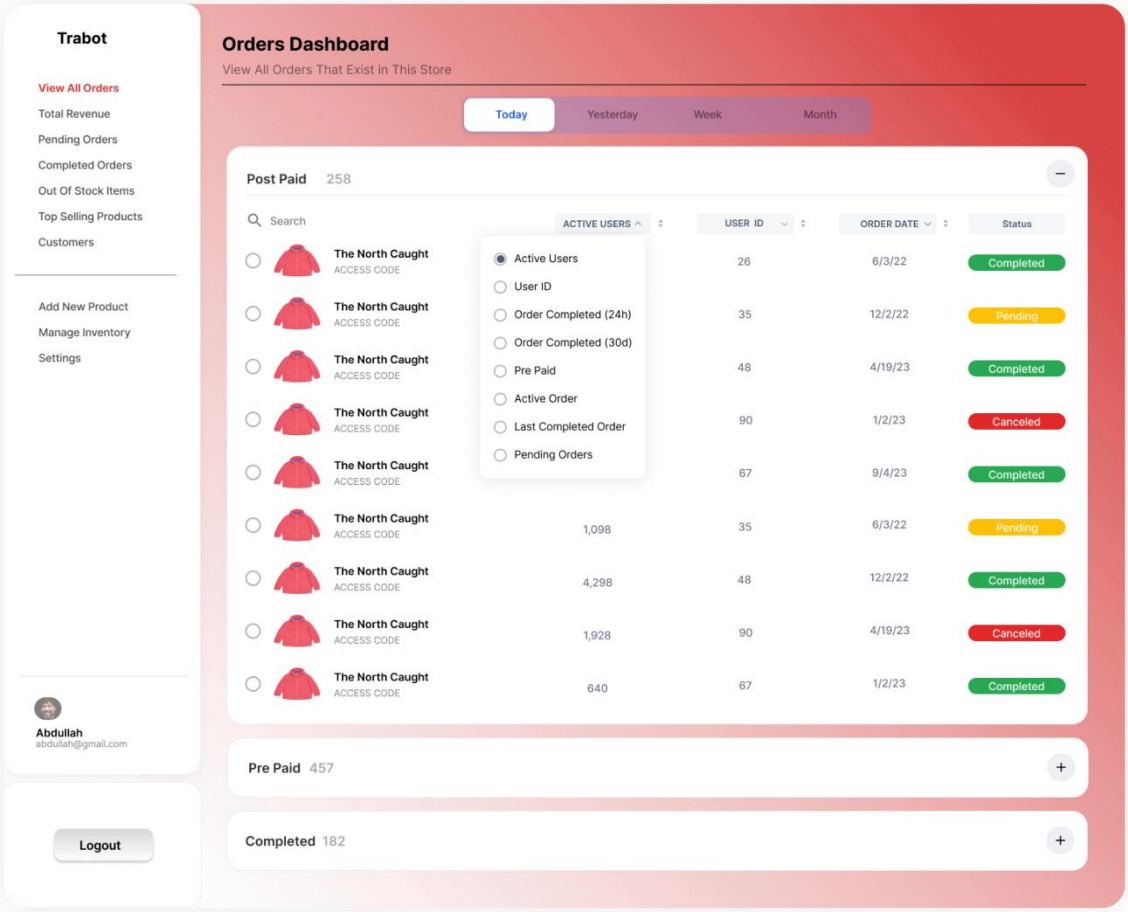


➢ User Interface design for Dashboards

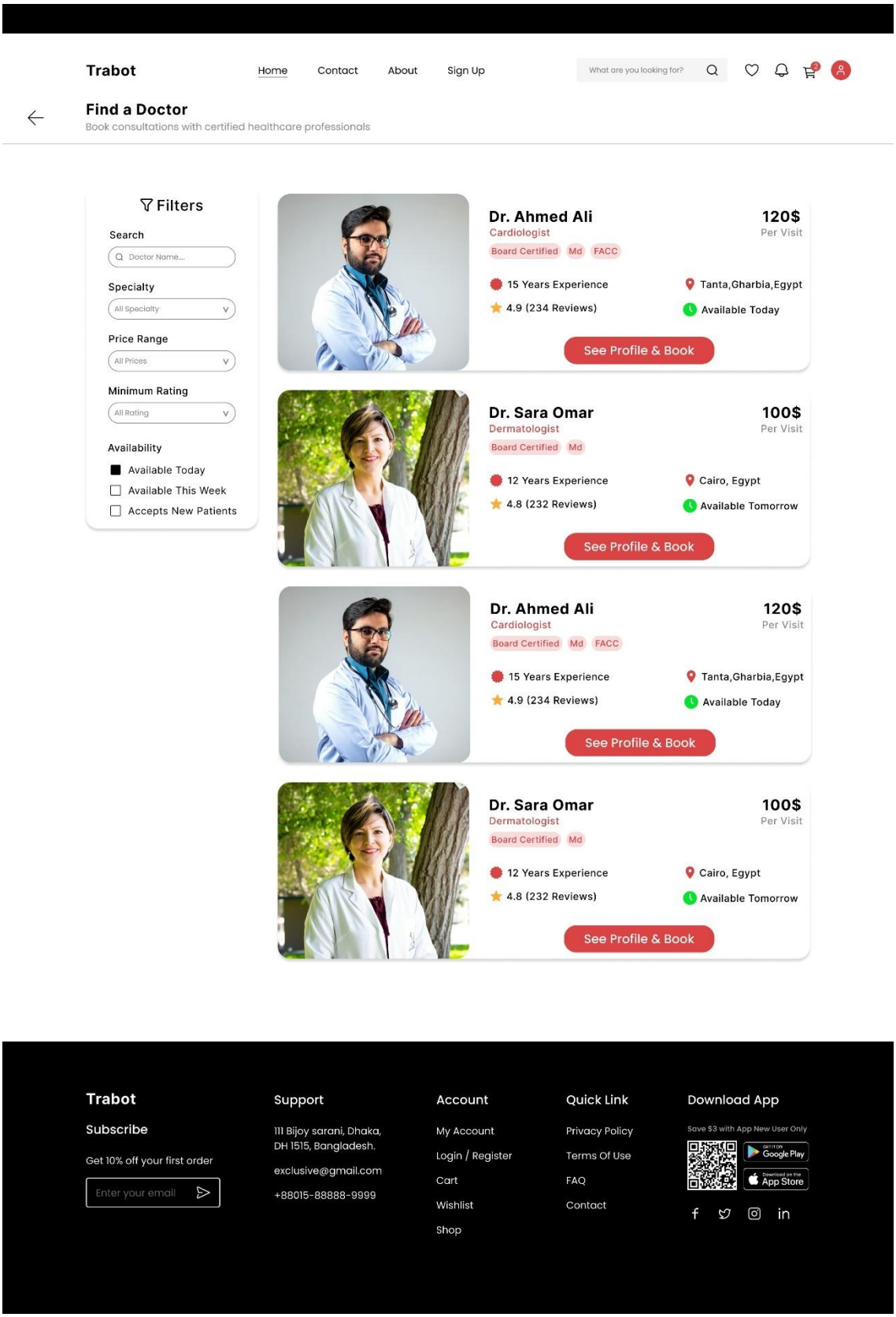




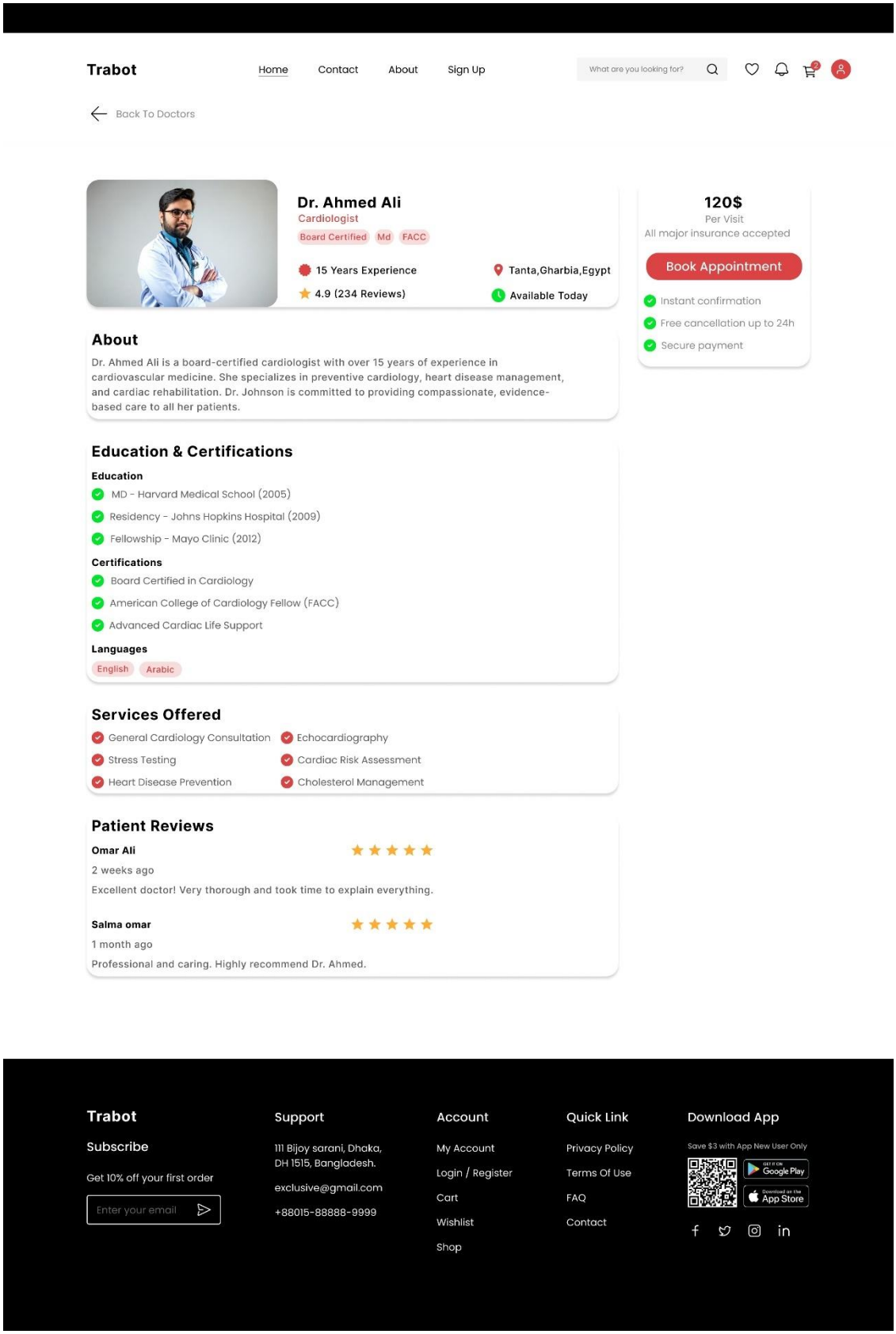

Trabot
View All Orders
Total Revenue
Pending Orders
Completed Orders
Out Of Stock Items
Top Selling Products
Customers
Add New Product
Manage Inventory
Settings
Abdullah
abdullah@gmail.com
Logout
Manage Inventory
View and manage your product inventory
Search Product...
Products
Product
SKU
Price
Stock
Status
The North Caught
ACCESS CODE
CFS-001
$250
150
120
100
140
130
115
110
Active
Delete


Trabot
View All Orders
Total Revenue
Pending Orders
Completed Orders
Out Of Stock Items
Top Selling Products
Customers
Add New Product
Manage Inventory
Settings
Abdullah
abdullah@gmail.com
Logout
Orders Dashboard
View All Orders That Exist in This Store
Today
Yesterday
Week
Month
Post Paid 258
Search
ACTIVE USERS
USER ID
ORDER DATE
Status
Active Users
User ID
Order Completed (24h)
Order Completed (30d)
Pre Paid
Active Order
Last Completed Order
Pending Orders
The North Caught
ACCESS CODE
1,098
4,298
1,928
640
26
35
48
90
67
6/3/22
12/2/22
4/19/23
1/2/23
9/4/23
Completed
Pending
Canceled
Pre Paid 457
Completed 182

➢ User Interface design for doctors booking service

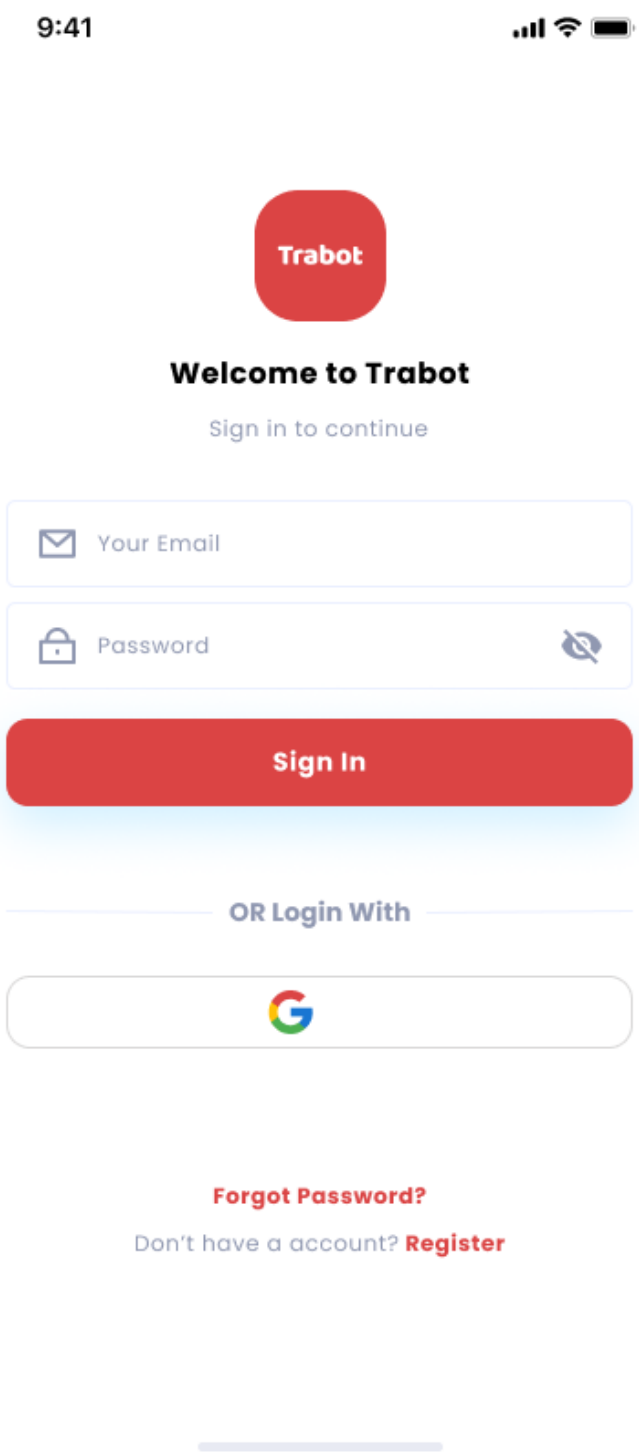


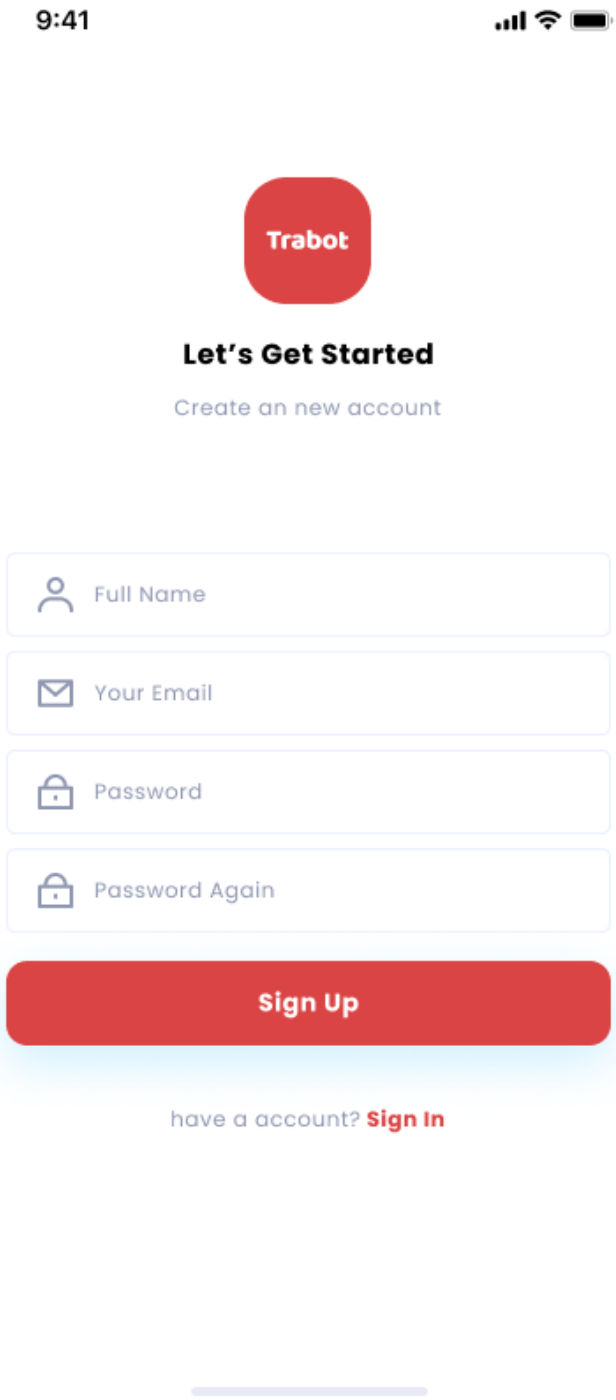
➢ User Interface design for Doctor Details page



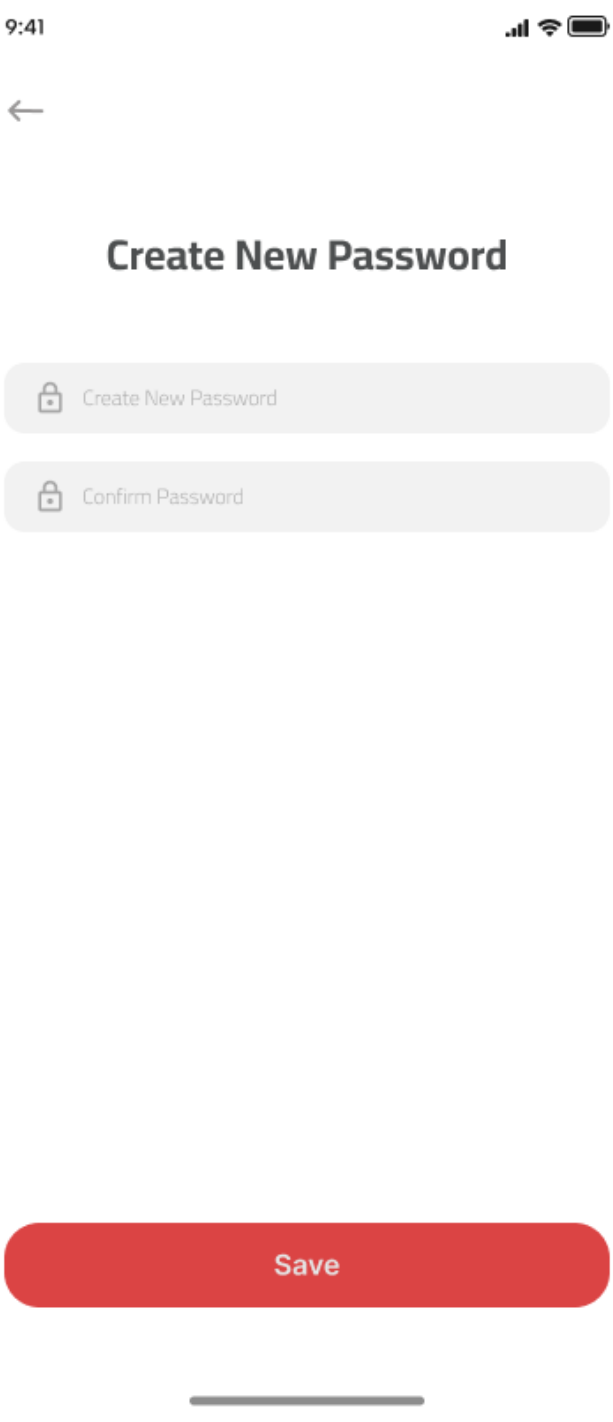
5.4.2 Mobile App

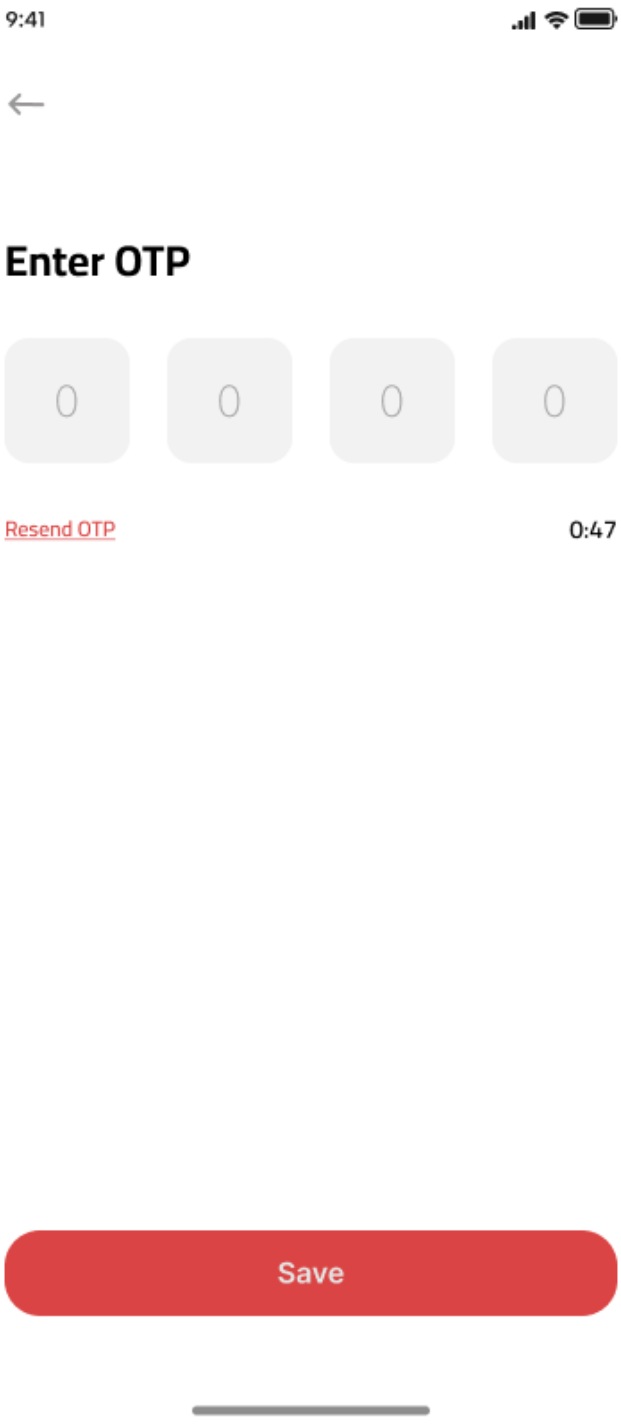
User Interface design for Authentication



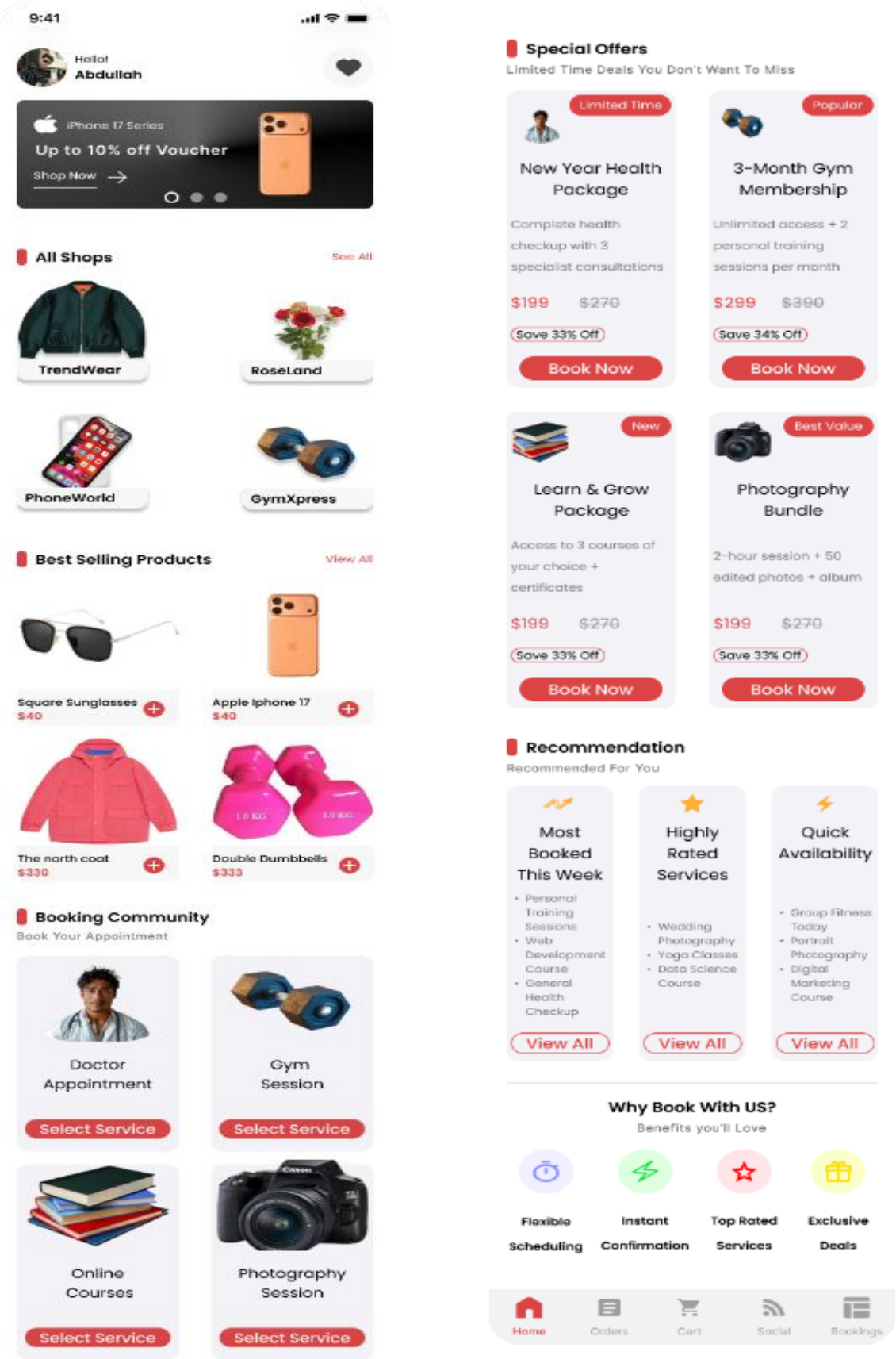


➢ **User Interface design for Password recovery and otp**

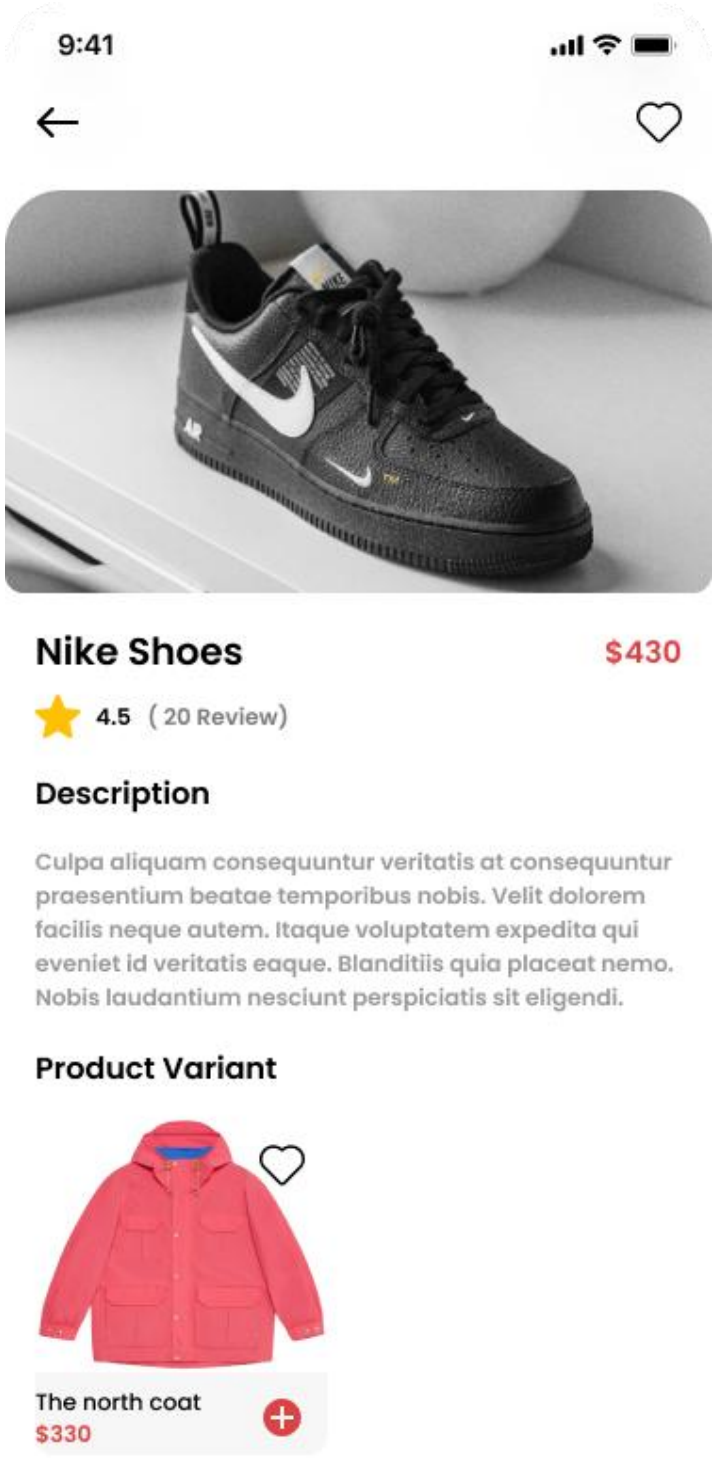


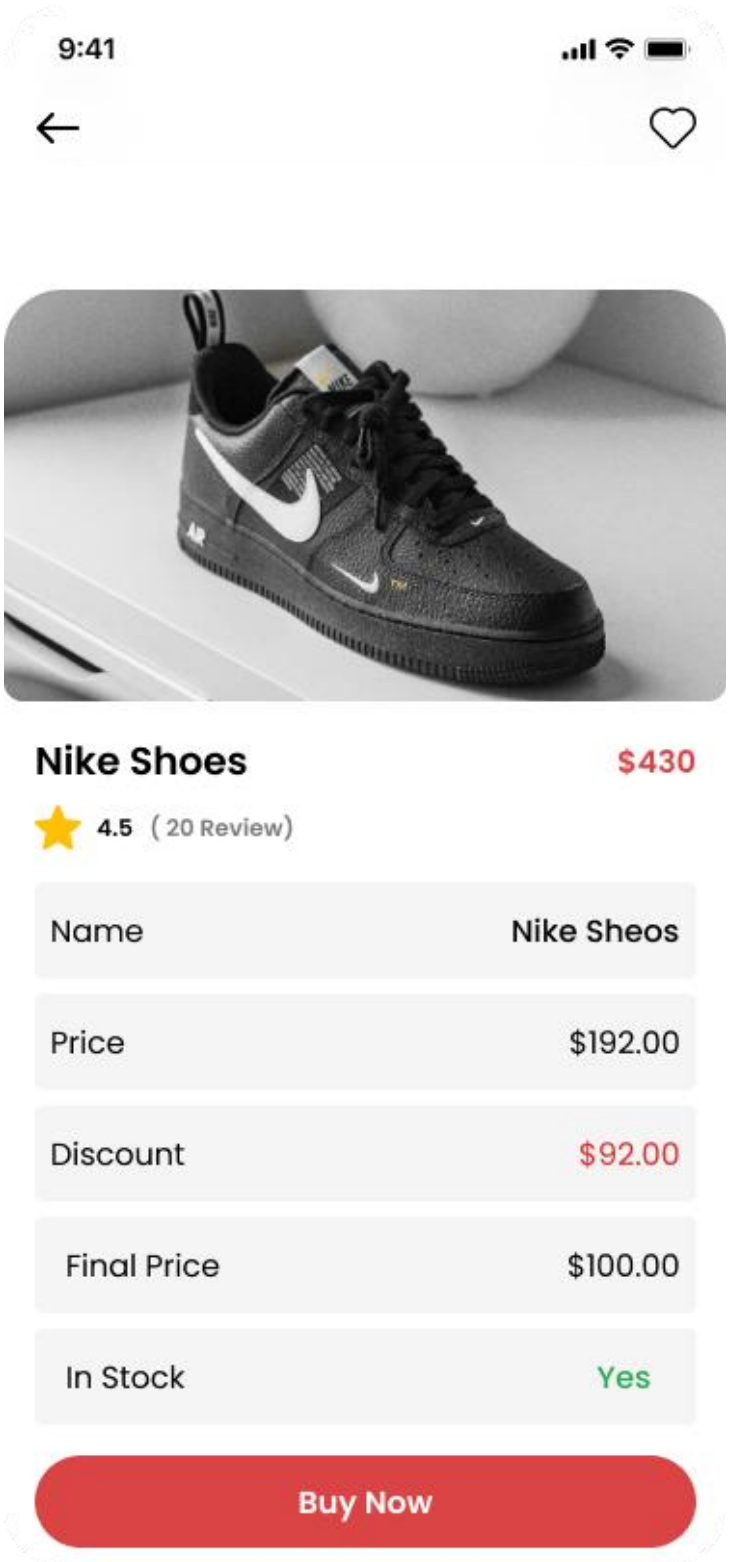


➢ **User Interface design Home page**

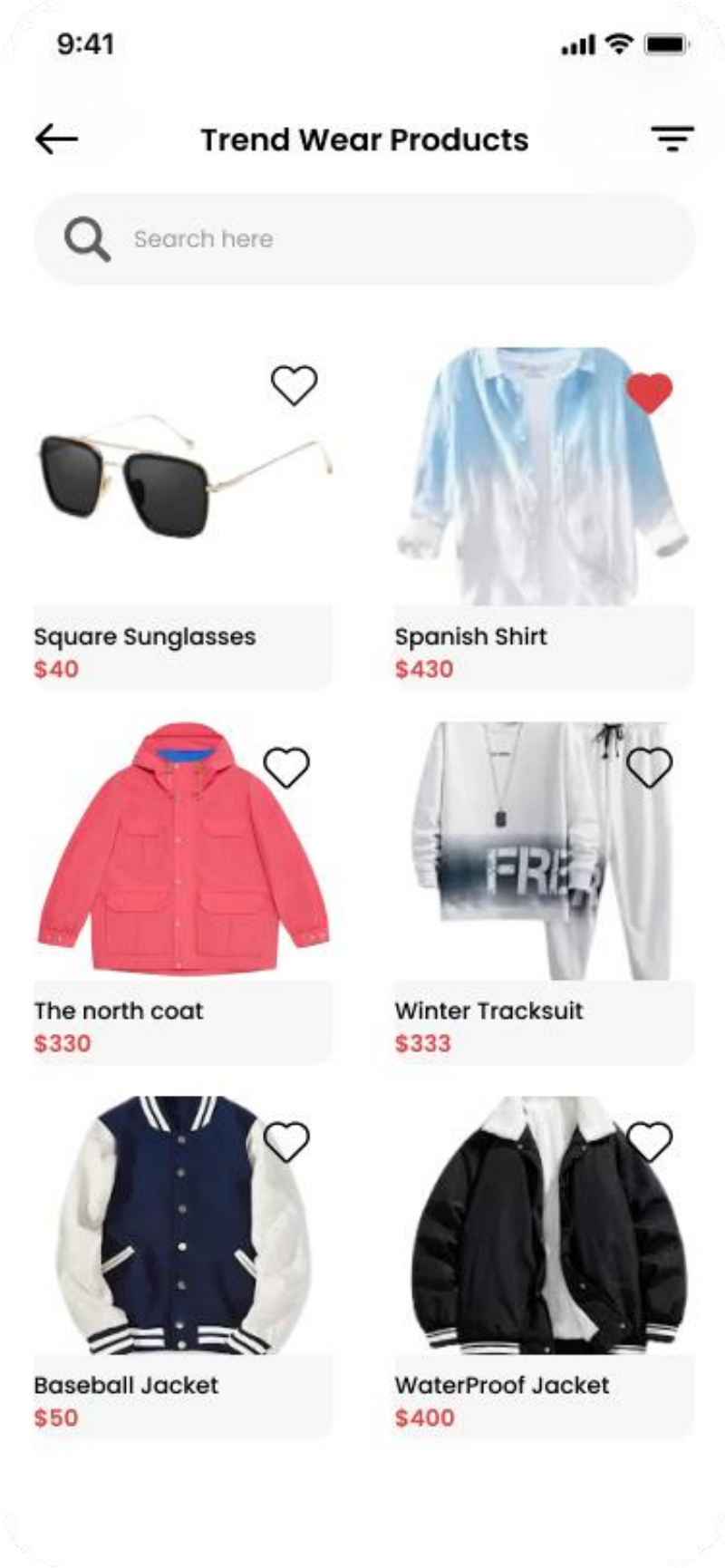


- **User Interface design for product description**

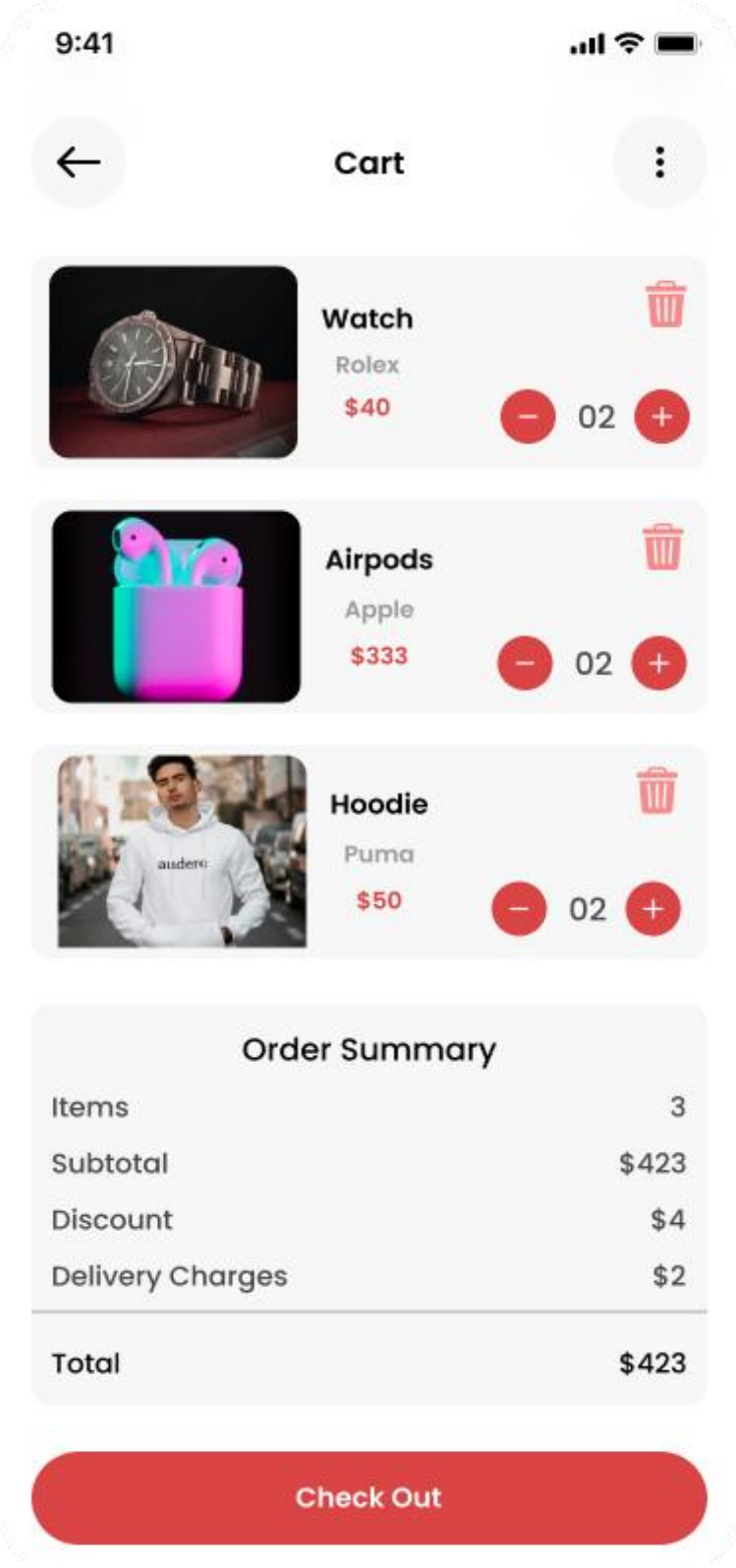




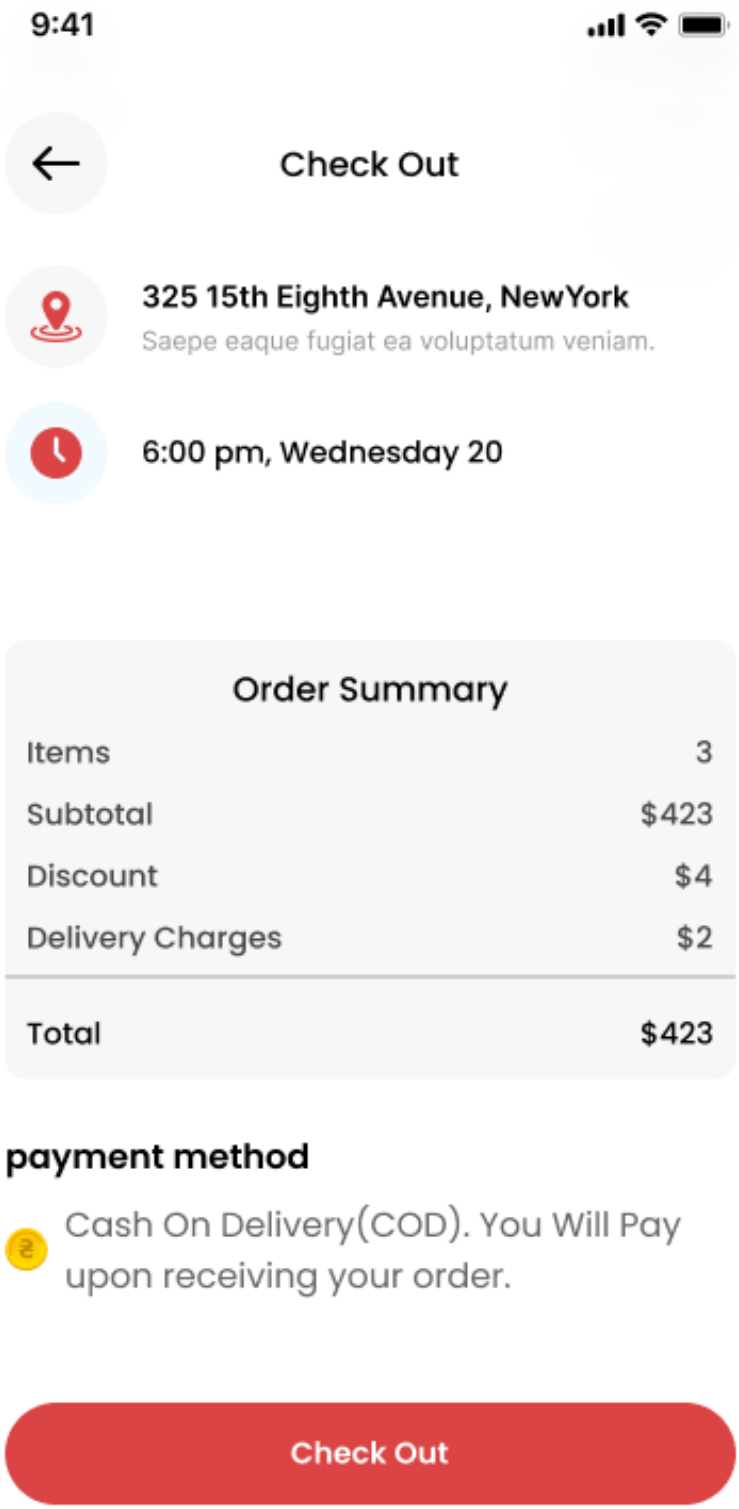
- **User Interface design for shop products**



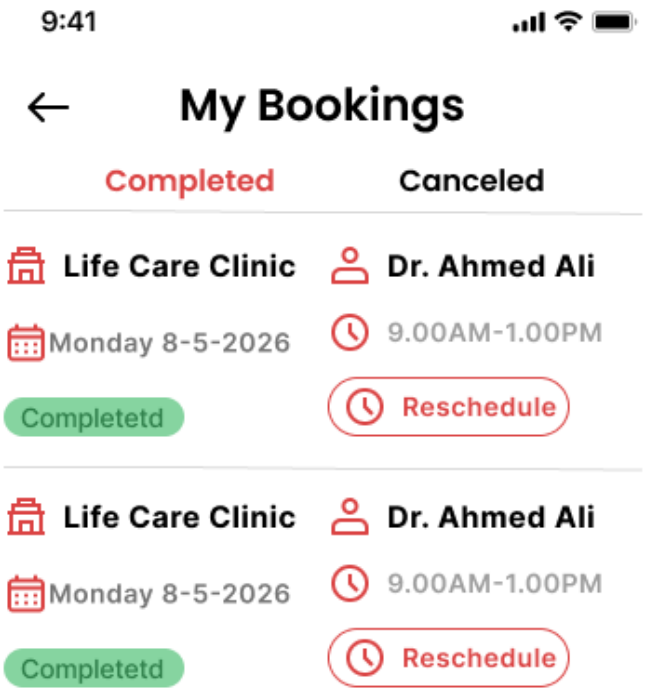
- **User Interface design for cart**

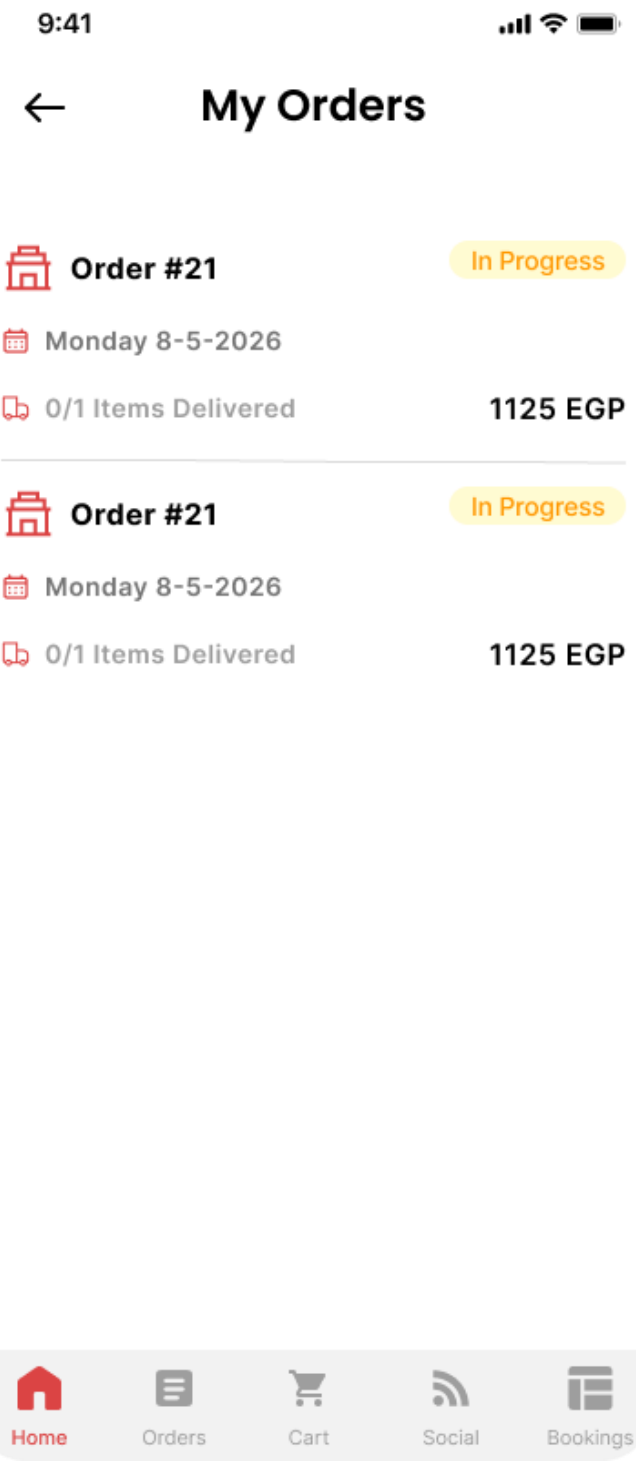


➢ **User Interface design for check out**

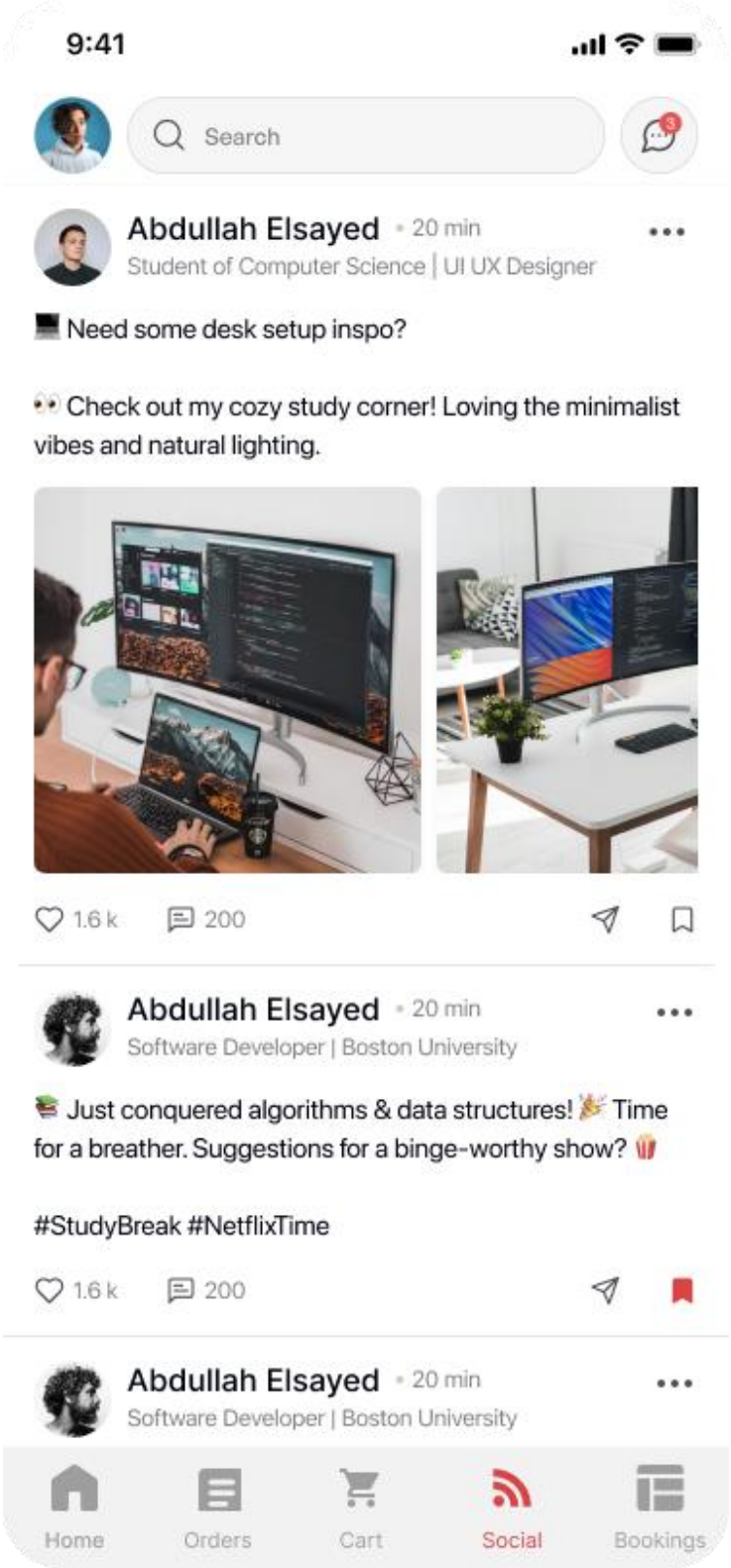


➢ **User Interface design for orders & bookings**

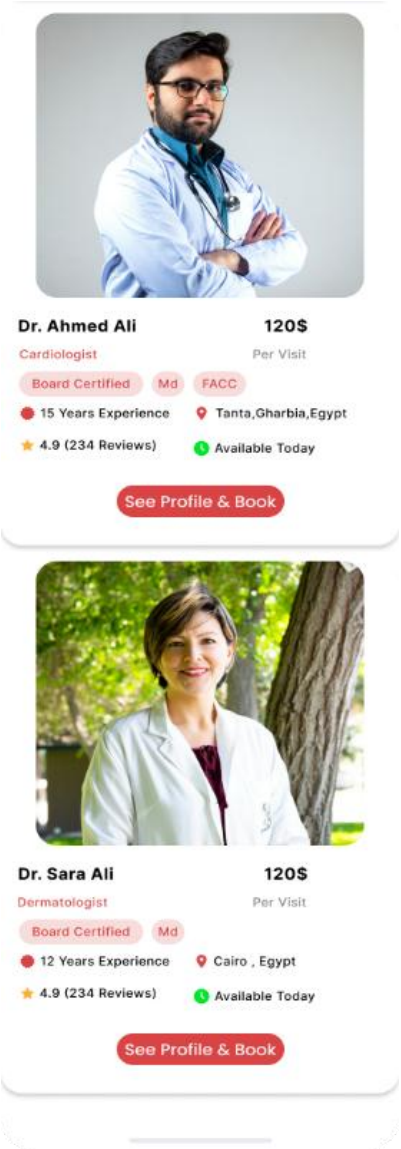




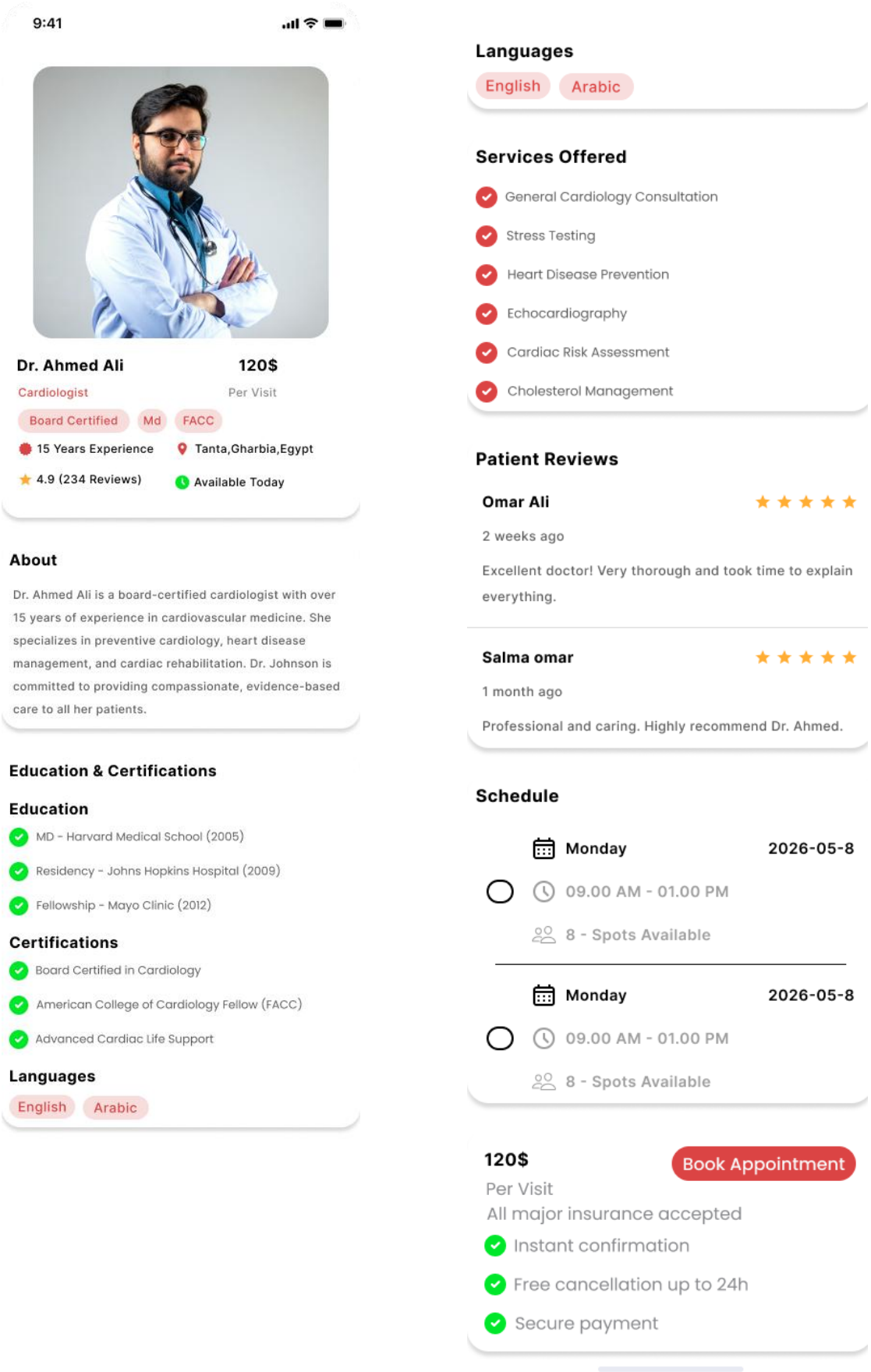
➢ **User Interface design for social**



User Interface design for doctors booking service



➢ User Interface design for Doctor Details page



CHAPTER 6— IMPLEMETATION

6.1 Introduction

The System Implementation chapter describes the process of converting the design and system specifications of the Trabot community platform into a fully functioning and user-ready web and mobile application. This phase includes the development, integration, and deployment of all system modules based on the defined requirements.

6.2 Tools and technologies

6.2.1 Backend Development

The backend of the system was developed using modern Microsoft technologies and architectural patterns to ensure scalability, maintainability, performance, and clean separation of concerns.

6.2.1.1 Backend Technologies

Technologies Used:

- **ASP.NET** Core

ASP.NET Core is the primary framework used to build the backend RESTful API. It is a cross-platform, open-source framework developed by Microsoft, known for its high performance, security, and scalability.
Key features utilized include:

  - REST API Development
  - Dependency Injection (DI)
  - Authentication and Authorization
  - Middleware Pipeline
  - Configuration Management
  - Security Features

ASP.NET Core serves as the backbone of the platform, handling business operations, data processing, integrations, and communication between system components.

- **SQL Server**

Microsoft SQL Server was selected as the primary relational database management system due to its reliability, security, and seamless integration with ASP.NET Core and Entity Framework Core.
The database stores all system data, including users, products, services, bookings, transactions, reviews, chats, notifications, and administrative records.
Key benefits include:

  - Data Integrity and Consistency
  - High Performance Query Processing
  - Scalability for Growing Data Volumes
  - Advanced Indexing and Optimization Features
  - Secure Data Management

- **Entity Framework Core**

Entity Framework Core (EF Core) was used as the Object Relational Mapper (ORM) for database operations.

The project follows the **Code First Approach**, where database entities and relationships are defined through C# classes, and the database schema is generated automatically.

Entity Framework Core provides:

  - Database Access Abstraction
  - LINQ Query Support
  - Change Tracking
  - Relationship Management
  - Automatic Data Persistence

- **Entity Framework Core Migrations**

EF Core Migrations were used to manage database schema changes throughout the development lifecycle.
This approach allows:

  - Version Control for Database Structure
  - Safe Schema Updates
  - Automated Database Creation and Modification
  - Easier Team Collaboration

- **AutoMapper**

AutoMapper was implemented to simplify object-to-object mapping between:

- Entities
- DTOs (Data Transfer Objects)
- Request Models
- Response Models

This reduces repetitive mapping code and improves maintainability and readability.

- **Swagger (OpenAPI)**

Swagger was used for API documentation, visualization, and testing.

It provides:

- Interactive API Documentation
- Endpoint Testing Interface
- Request and Response Visualization
- Faster Backend and Frontend Integration

6.2.1.2 Backend Architecture

The backend follows the **Clean Architecture** pattern to achieve a clear separation of responsibilities, improve maintainability, and support future scalability.

Architecture Layers:

- **Presentation Layer (Trabot API)**

Responsible for exposing RESTful API endpoints, receiving client requests, validating input, and returning standardized responses, This layer acts as the entry point to the system.

- **Core Layer (Trabot.Core)**

Contains:

- Application Features
- DTOs
- Request and Response Models
- Business Rules
- validation behaviors
- localization resources
- CQRS Commands and Queries

    - Mapping Profiles
    - pagination wrappers

- **Service Layer (Trabot.Service)**

Contains the application services responsible for executing business logic, enforcing business rules, and coordinating operations across different system layers.

- **Infrastructure Layer (Trabot.Infrastructure)**

Responsible for external integrations and technical implementations, including:

    - Authentication Services
    - File Management
    - Repository Implementations
    - External Service Integrations
    - Third-Party Dependencies

- **Data Layer (Trabot. Data)**

Handles data persistence and contains:

    - Database Entities
    - Entity Configurations
    - DbContext
    - Migrations
    - Fluent API Configurations

- **Real-Time Service Layer (Trabot.RealTimeService)**

Provides real-time communication features using SignalR, including:

    - Real-Time Chat

6.2.1.3 Design Patterns and Architectural Practices

Several software design patterns and architectural practices were adopted to improve code quality and maintainability.

- **CQRS (Command Query Responsibility Segregation)**

The application follows the CQRS pattern by separating:

    - Commands (Write Operations)
    - Queries (Read Operations)

This separation improves maintainability, readability, and scalability.

- **MediatR**

MediatR was used to implement CQRS and decouple application components.

Benefits include:

  - Reduced Dependencies Between Layers
  - Cleaner Request Handling
  - Improved Maintainability
  - Better Separation of Concerns

- **Repository Pattern**

The Repository Pattern was implemented to abstract data access logic from business logic, providing a clean interface for interacting with the database.

- **Dependency Injection**

ASP.NET Core's built-in Dependency Injection container was used to manage service lifetimes and dependencies across the application.

- **Validation Pipeline**

Validation Pipelines were implemented to validate requests before executing business logic, ensuring data integrity and reducing duplicated validation code.

- **Global Exception Handling Middleware**

A centralized exception handling middleware was implemented to:

  - Capture Unhandled Exceptions
  - Log System Errors
  - Return Consistent Error Responses
  - Improve System Reliability

- **Response Wrapper**

A standardized response wrapper was used across all API endpoints to provide consistent response structures for:

  - Successful Operations
  - Validation Errors

  - Business Errors

- Server Exceptions

- **AutoMapper**

AutoMapper was used to simplify object mapping between entities, Data Transfer Objects (DTOs), request models, and response models. This approach reduces repetitive mapping code, improves maintainability, and promotes a clear separation between the data access layer and API contracts.

- **Resource-Based Localization**

Resource-based localization was implemented to support multiple languages and provide localized system messages and responses. This enhances accessibility and improves the overall user experience for users with different language preferences.

- **Swagger API Documentation**

Swagger was utilized for API documentation, testing, and visualization. It provides an interactive interface that allows developers to explore available endpoints, inspect request and response models, and test API functionality efficiently, simplifying both development and integration processes.

6.2.1.4 Database Design and Performance Optimization

The database was designed using Entity Framework Core with the Code First approach, where entities and their relationships are defined through C# classes and automatically translated into the database schema.

Entity relationships, including:

- One-to-One
- One-to-Many
- Many-to-Many

were configured using Entity Framework Core and Fluent API to ensure data consistency, integrity, and proper relational structure.

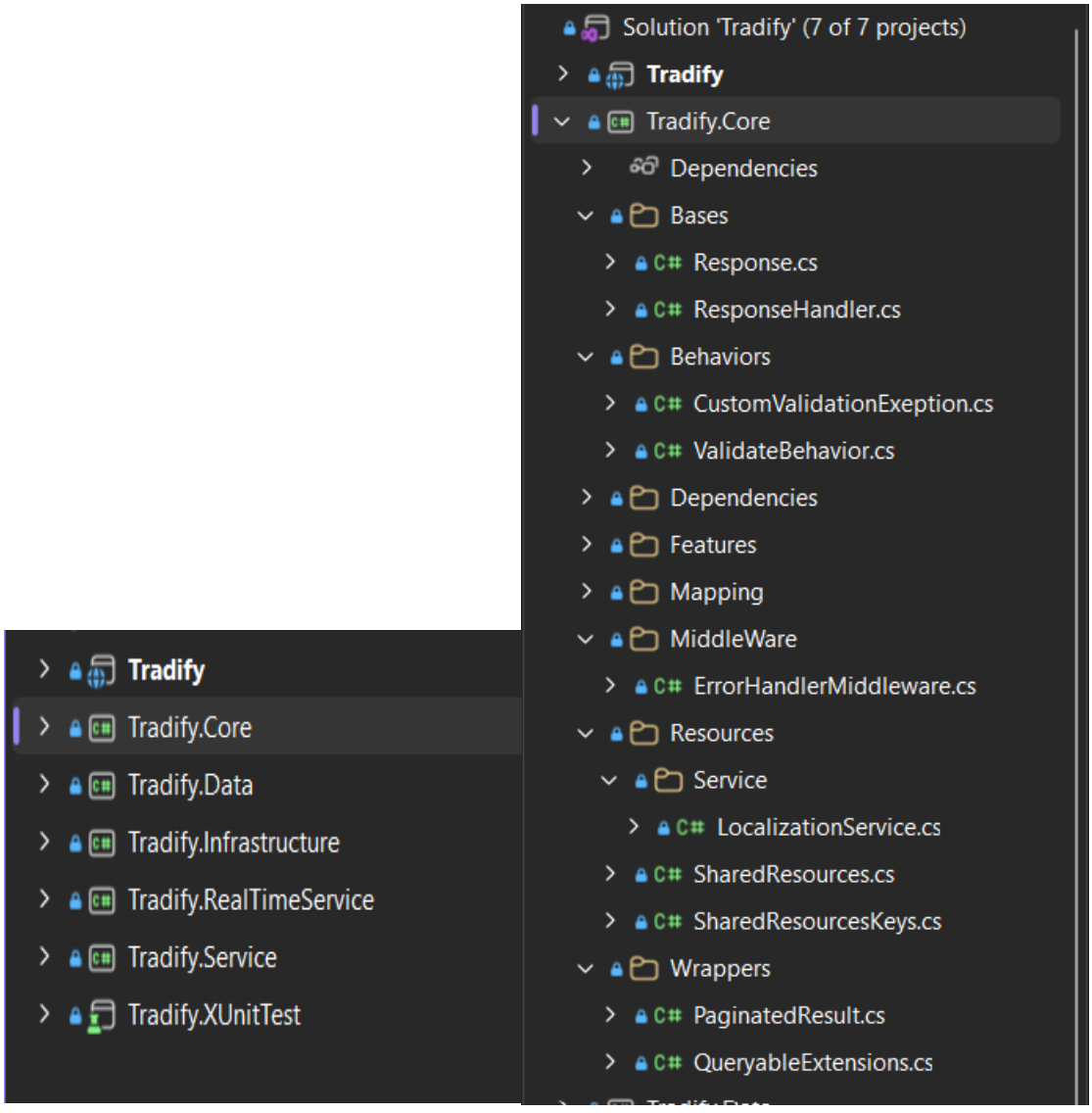
Entity Framework Core Migrations were used to manage database schema changes throughout the development lifecycle in a controlled and maintainable manner.

To optimize performance when retrieving large datasets, pagination techniques were implemented across multiple API endpoints. This reduces response size, minimizes data transfer, and improves response times.

Additionally, SQL Server indexing and Entity Framework Core query optimization techniques were utilized to enhance data retrieval efficiency and support system scalability.

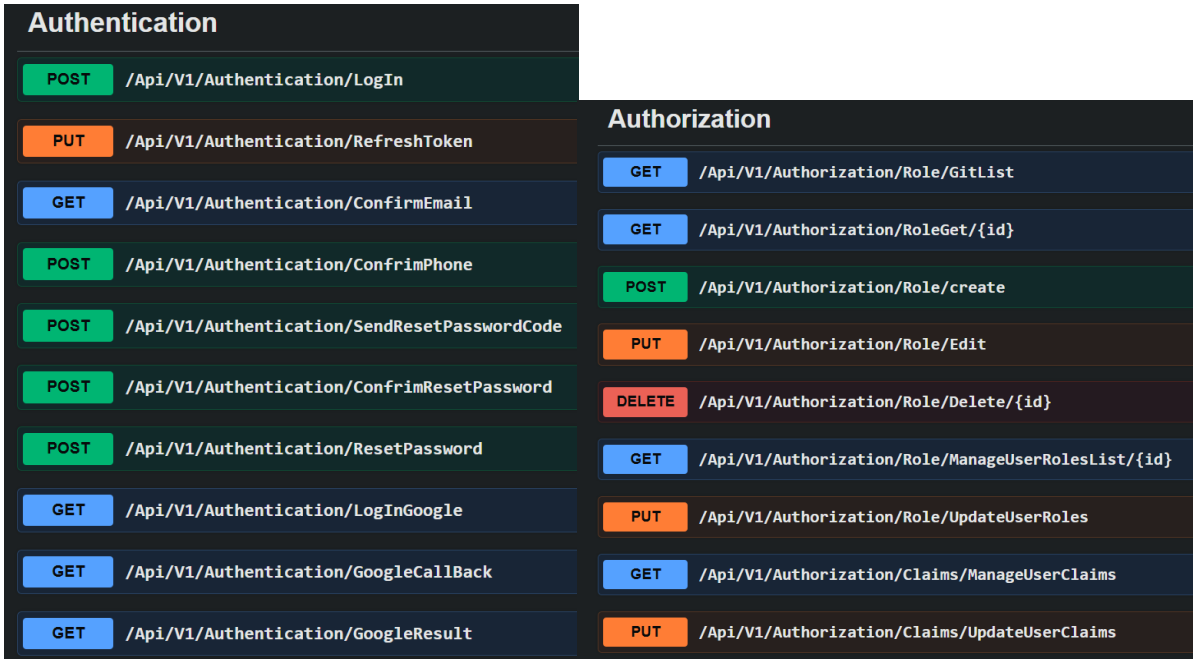
Combined with these optimization strategies, the system is capable of handling large volumes of data while maintaining reliable performance and responsiveness.

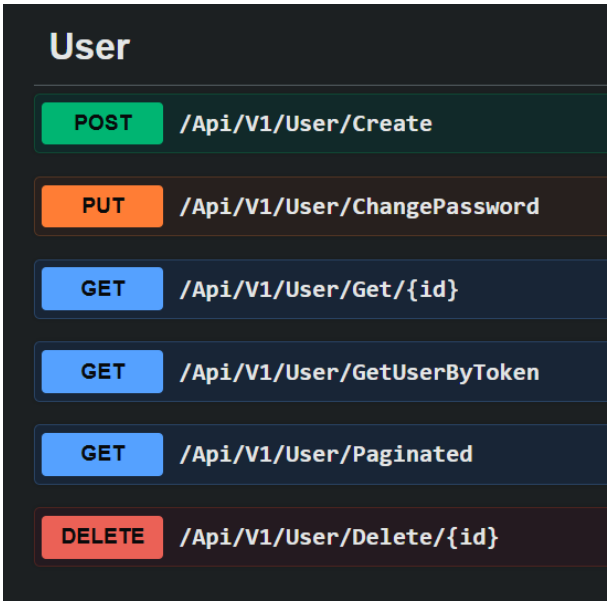
6.2.1.5 Clean Architecture Structure of the Trabot Backend System.



6.2.1.6 Endpoints overview

- **Authentication, Authorization APIs**





- **Product Management API Endpoints**

SellerProduct

POST	/Api/V1/Product/AddProduct
POST	/Api/V1/Product/AddWithImage
PUT	/Api/V1/Product/Update
DELETE	/Api/V1/Product/Delete/{id}
PUT	/Api/V1/Product/restore/{id}
GET	/Api/V1/Product/MyProducts

Product

GET	/Api/V1/Product/Paginated
GET	/Api/V1/Product/BestSelling
GET	/Api/V1/Product/List
GET	/Api/V1/Product/Search
GET	/Api/V1/Product/Store
GET	/Api/V1/Product/Get/{id}
GET	/Api/V1/Product/Discount

Discount

PUT	/Api/V1/ProductVariant/AddDiscount
DELETE	/Api/V1/ProductVariant/DeleteDiscount
PUT	/Api/V1/Product/AddDiscount
DELETE	/Api/V1/Product/DeleteDiscount
PUT	/Api/V1/Instructor/AddDiscount
DELETE	/Api/V1/Instructor/DeleteDiscount
PUT	/Api/V1/Store/AddStoreServiceDiscount
DELETE	/Api/V1/Store/DeleteStoreServiceDiscount

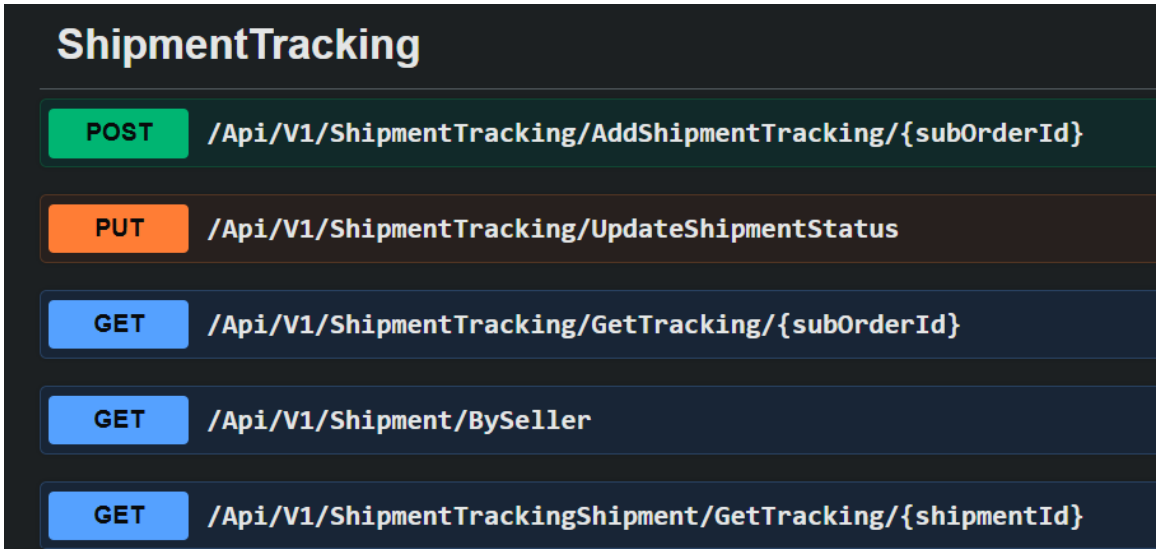
- **Order, Suborder and shipment tracking API Endpoints**

SupOrder

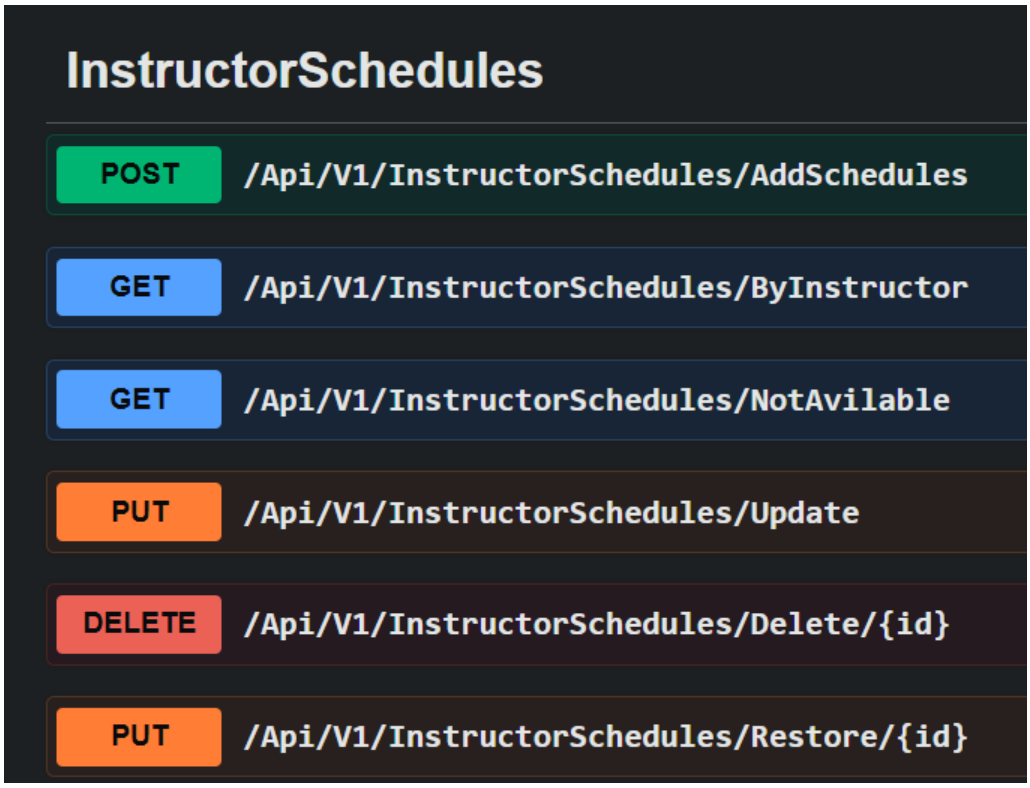
GET	/Api/V1/SupOrder/ByOrder
GET	/Api/V1/SupOrder/BySeller

Order

POST	/Api/V1/Order/Create
PUT	/Api/V1/Order/Update
GET	/Api/V1/Order/{id}
GET	/Api/V1/Order/Paginated
DELETE	/Api/V1/Order/Delete/{id}



- **Instructor and booking API Endpoints**



InstructorService

POST	/Api/V1/InstructorService/AddService
GET	/Api/V1/InstructorService/ByInstructor
PUT	/Api/V1/InstructorService/Update
DELETE	/Api/V1/InstructorService/Delete/{id}

Booking

POST	/Api/V1/Booking/AddBooking
GET	/Api/V1/Booking/GetUserBookings
GET	/Api/V1/Booking/GetInstructorBooking
PUT	/Api/V1/Booking/CanceId
PUT	/Api/V1/Booking/Reschedule

- **Social Interaction API Endpoints**

Post

POST	/Api/V1/Post/AddPost
GET	/Api/V1/Post/GetPostsOfUserByID/{id}
GET	/Api/V1/Post/GetPostByID/{id}
GET	/Api/V1/Post/GetPosts
DELETE	/Api/V1/Post/Delete
POST	/Api/V1/Post/Update

Interaction

POST	/Api/V1/Interaction/Add
PUT	/Api/V1/Interaction/Update
DELETE	/Api/V1/Interaction/Delete/{id}
GET	/Api/V1/Interaction/GetInteractions/post/{id}
GET	/Api/V1/Interaction/GetInteractions/{id}
GET	/Api/V1/InteractionGetUserInteractionPost

Comments

POST	/Api/V1/Comments/Add
PUT	/Api/V1/Comments/update
DELETE	/Api/V1/Comments/Delete
POST	/Api/V1/ReplayOnComments/Add
PUT	/Api/V1/ReplayOnComments/update
DELETE	/Api/V1/ReplayOnComments/Delete
GET	/Api/V1/Comments/Get/ByPostId
GET	/Api/V1/Comments/Get/ByCommentId
GET	/Api/V1/ReplayOnComments/Get/ByCommentId
GET	/Api/V1/ReplayOnComments/Get/ByReplayId

Messages

POST	/Api/V1/Message/AddMessage
DELETE	/Api/V1/Message/Delete
PUT	/Api/V1/Message/Update
PATCH	/Api/V1/Message/IsRead
GET	/Api/V1/Message/Conversation
GET	/Api/V1/Message/UnReadMessages/{id}
GET	/Api/V1/Message/{id}

- **Dashboards API Endpoints**

AdminDashpoard

GET	/Api/V1/Dashpoard/admin
GET	/Api/V1/Dashpoard/admin/orders-chart
GET	/Api/V1/Dashpoard/admin/booking-chart
GET	/Api/V1/Dashpoard/admin/RevenueChart
GET	/Api/V1/Dashpoard/admin/top-stores

SellerServiceDashpoard

GET	/Api/V1/Dashpoard/sellerService
GET	/Api/V1/Dashpoard/sellerService/BookingChart
GET	/Api/V1/Dashpoard/sellerService/top-Instructor

SellerProductDashpoard

GET	/Api/V1/Dashpoard/sellerProduct
GET	/Api/V1/Dashpoard/sellerProduct/orders-chart
GET	/Api/V1/Dashpoard/sellerProduct/RevenueChart
GET	/Api/V1/Dashpoard/sellerProduct/top-products-Selling
GET	/Api/V1/Dashpoard/sellerProduct/top-Rated-Products

InstructorDashpoard

GET	/Api/V1/Dashpoard/instructor
GET	/Api/V1/Dashpoard/instructor/SessionsChart
GET	/Api/V1/Dashpoard/instructor/upcoming-appointments

- **Recommendations**

Recommendation

GET	/Api/V1/Recommendation/Instructors
GET	/Api/V1/Recommendation/TopRated/Product
GET	/Api/V1/Recommendation/TopRated/Instructor

6.2.2 Flutter Mobile Application

6.2.2.1 Mobile Application Technologies

The mobile application was developed using modern Flutter technologies and architectural practices to ensure maintainability, scalability, performance, and a responsive user experience across multiple platforms.

Technologies Used:

- **Flutter**

Flutter was selected as the primary framework for mobile application development due to its cross-platform capabilities, high performance, and rich widget ecosystem.

Key features utilized include:

  - Cross-Platform Development
  - Reactive User Interface
  - Custom Widgets
  - State Management
  - Responsive Design
  - Hot Reload

  Flutter serves as the presentation layer of the system, providing users with a seamless and interactive experience while communicating with backend services through RESTful APIs.

- **Dart**

Dart is the programming language used for developing the Flutter application.

Key benefits include:

  - Object-Oriented Programming
  - Strong Type Safety
  - High Performance
  - Asynchronous Programming Support
  - Clean and Readable Syntax

- **Dio**

Dio was used as the primary HTTP client for communication with backend APIs.

It provides:

- API Requests Handling
- Request and Response Interceptors
- Error Handling
- File Upload and Download Support
- Request Configuration Management

- **Flutter Bloc (Cubit)**

Flutter Bloc was implemented for state management using the Cubit pattern.

Benefits include:

- Predictable State Management
- Separation of Business Logic from UI
- Improved Maintainability
- Better Code Organization
- Easier Testing

6.2.2.2 Mobile Application Architecture

The mobile application follows a Feature-Based Clean Architecture approach to ensure scalability, maintainability, and separation of concerns.

Architecture Layers:

- **Presentation Layer**

Responsible for:

- Screens
- Widgets
- Cubits
- states

This layer communicates with repositories and displays data to users.

- **Data Layer**

Responsible for:

- API Integration
- Data Models
- Remote Data Sources
- Local Data Sources
- Repository Implementations

This layer handles communication with backend services and local storage.

- **Core Layer**

Contains reusable components shared across the application:

  - Constants
  - Routing
  - Network Configuration
  - Utility Functions
  - Shared Widgets

6.2.2.3 Design Patterns and Architectural Practices

Several architectural practices were adopted to improve code quality and maintainability.

- **Repository Pattern**

The Repository Pattern was implemented to separate data access logic from presentation and business logic.

Benefits include:

  - Better Maintainability
  - Improved Testability
  - Cleaner Code Structure

- **State Management with Cubit**

Cubit was adopted to manage application states efficiently.

Benefits include:

  - Clear State Flow
  - Reduced UI Complexity
  - Better Separation of Concerns

- **API Response Handling**

A unified response handling mechanism was implemented to process:

  - Successful Responses
  - Validation Errors
  - Business Errors
  - Server Exceptions

6.2.2.4 Performance Optimization

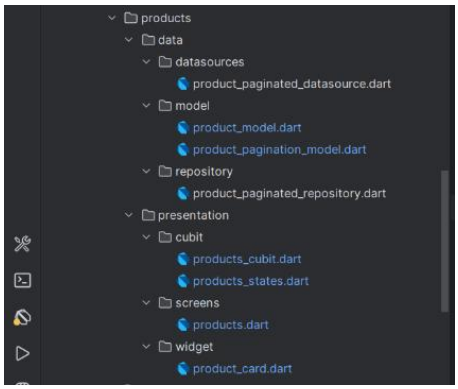
Several techniques were implemented to optimize application performance.

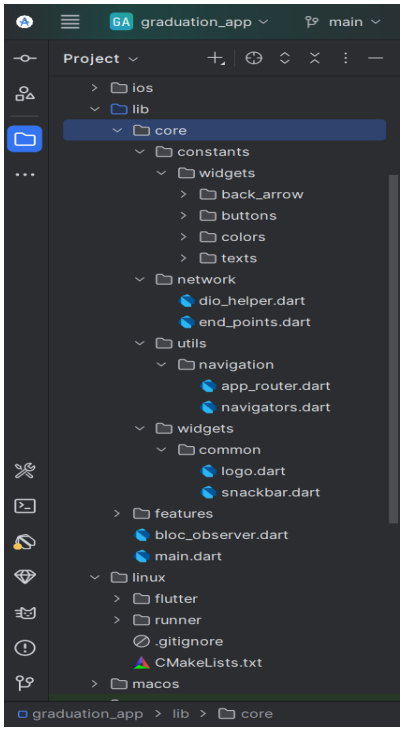
These include:

- Lazy Loading
- Pagination for Large Datasets
- Efficient State Management
- API Request Optimization
- Reusable Widgets
- Minimized Widget Rebuilds

These optimizations help maintain a smooth user experience while handling large amounts of data and real-time interactions.

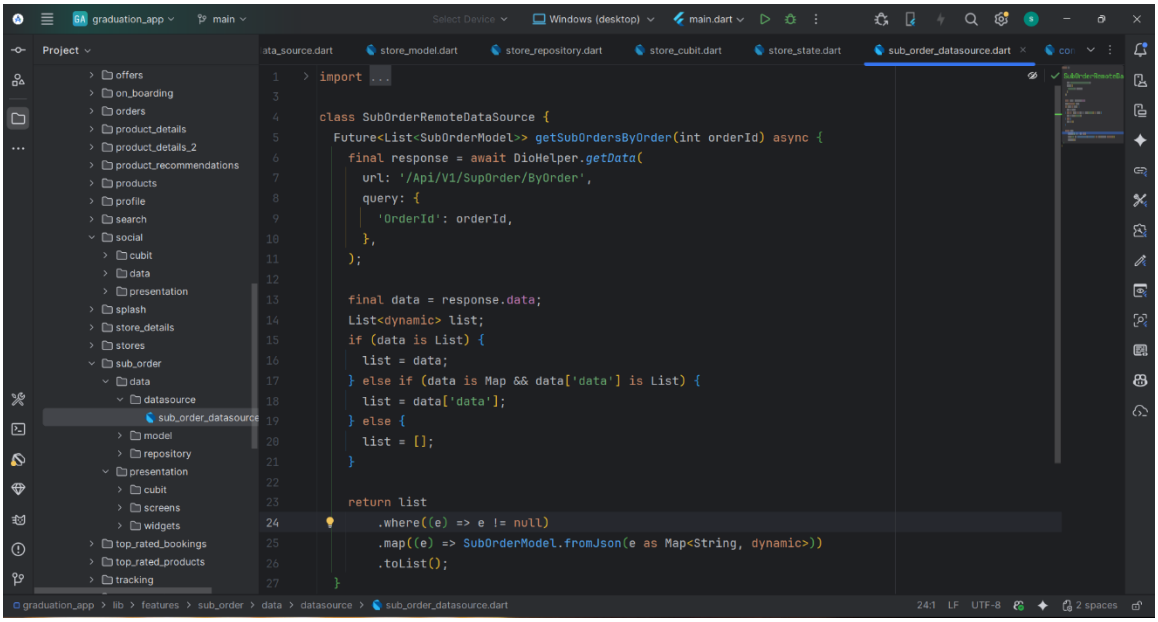
6.2.2.5 flutter folders clean Architecture





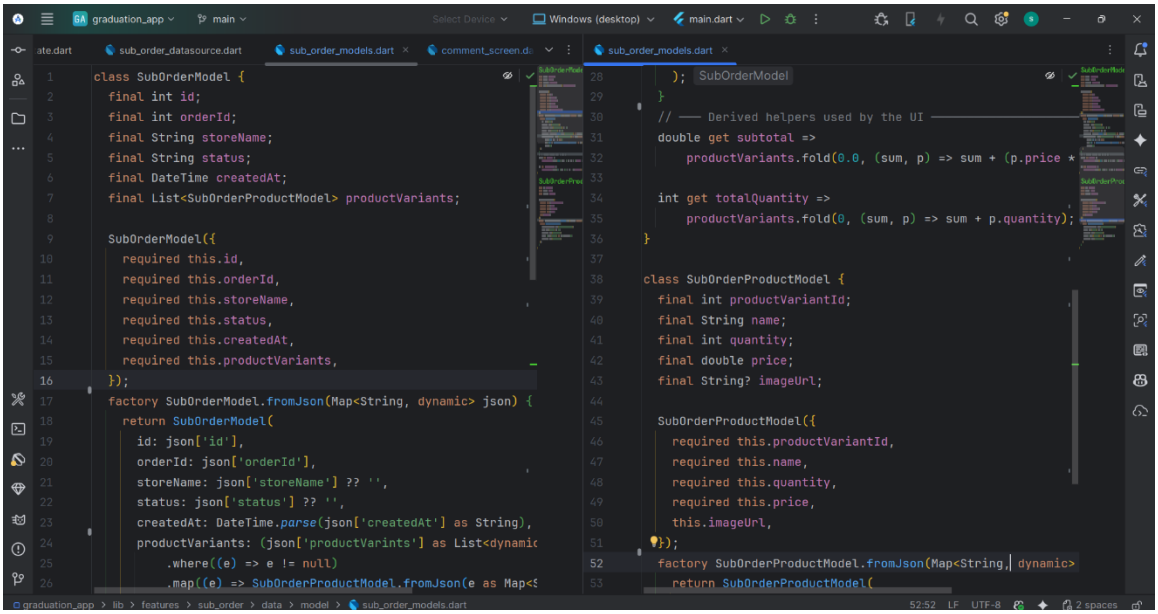
6.2.2.5 flutter codes overview

- **Data source implementation**



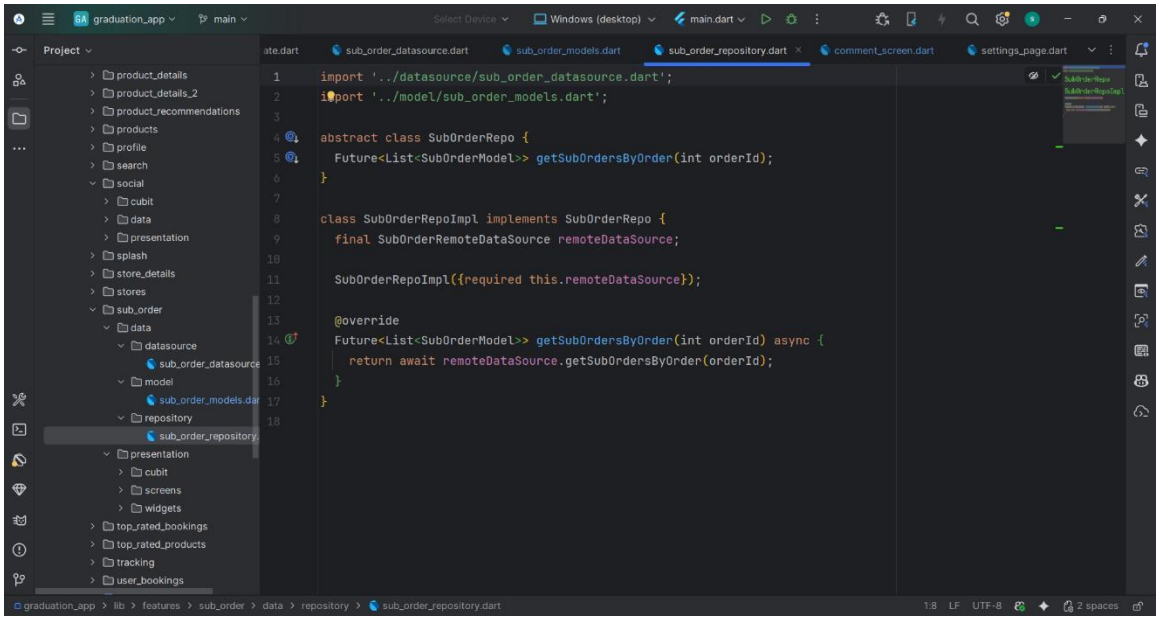
```dart
import ...

class SubOrderRemoteDataSource {
  Future<List<SubOrderModel>> getSubOrdersByOrder(int orderId) async {
    final response = await DioHelper.getData(
      url: '/Api/V1/SupOrder/ByOrder',
      query: {
        'OrderId': orderId,
      },
    );

    final data = response.data;
    List<dynamic> list;
    if (data is List) {
      list = data;
    } else if (data is Map && data['data'] is List) {
      list = data['data'];
    } else {
      list = [];
    }

    return list
        .where((e) => e != null)
        .map((e) => SubOrderModel.fromJson(e as Map<String, dynamic>))
        .toList();
  }
```

- **Model implementation**

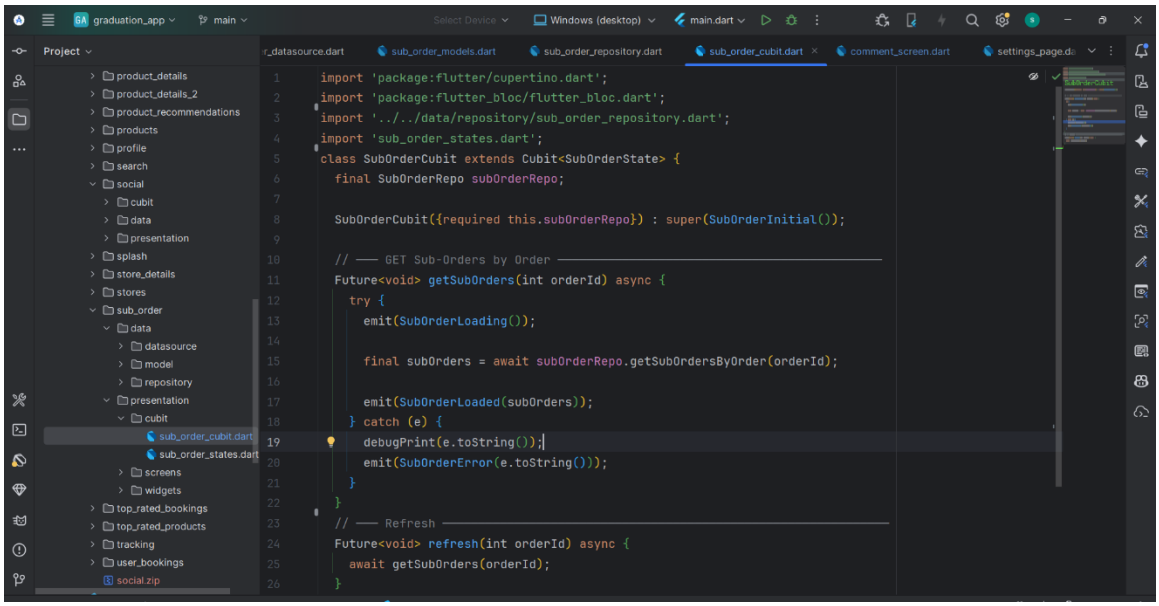


```dart
class SubOrderModel {
  final int id;
  final int orderId;
  final String storeName;
  final String status;
  final DateTime createdAt;
  final List<SubOrderProductModel> productVariants;

  SubOrderModel({
    required this.id,
    required this.orderId,
    required this.storeName,
    required this.status,
    required this.createdAt,
    required this.productVariants,
  });
  factory SubOrderModel.fromJson(Map<String, dynamic> json) {
    return SubOrderModel(
      id: json['id'],
      orderId: json['orderId'],
      storeName: json['storeName'] ?? '',
      status: json['status'] ?? '',
      createdAt: DateTime.parse(json['createdAt'] as String),
      productVariants: (json['productVarints'] as List<dynamic
          .where((e) => e != null)
          .map((e) => SubOrderProductModel.fromJson(e as Map<
  );
  }
  // ── Derived helpers used by the UI ──────────────────────
  double get subtotal =>
      productVariants.fold(0.0, (sum, p) => sum + (p.price *

  int get totalQuantity =>
      productVariants.fold(0, (sum, p) => sum + p.quantity);
}

class SubOrderProductModel {
  final int productVariantId;
  final String name;
  final int quantity;
  final double price;
  final String? imageUrl;

  SubOrderProductModel({
    required this.productVariantId,
    required this.name,
    required this.quantity,
    required this.price,
    this.imageUrl,
});
  factory SubOrderProductModel.fromJson(Map<String, dynamic>
    return SubOrderProductModel(
```

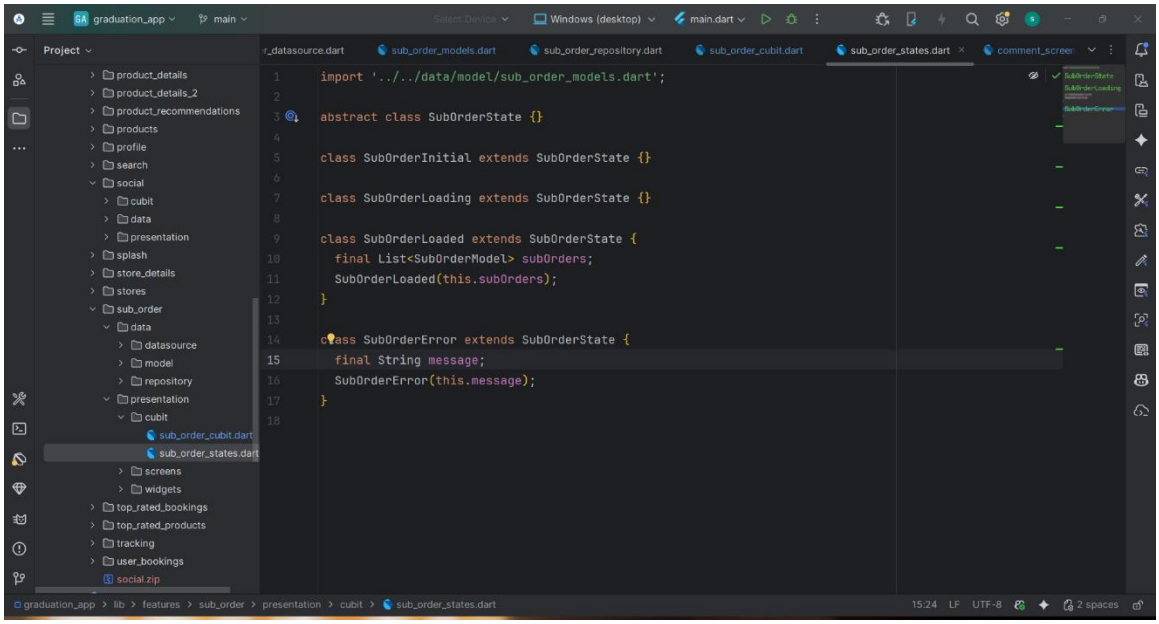
- **Repository implementation**



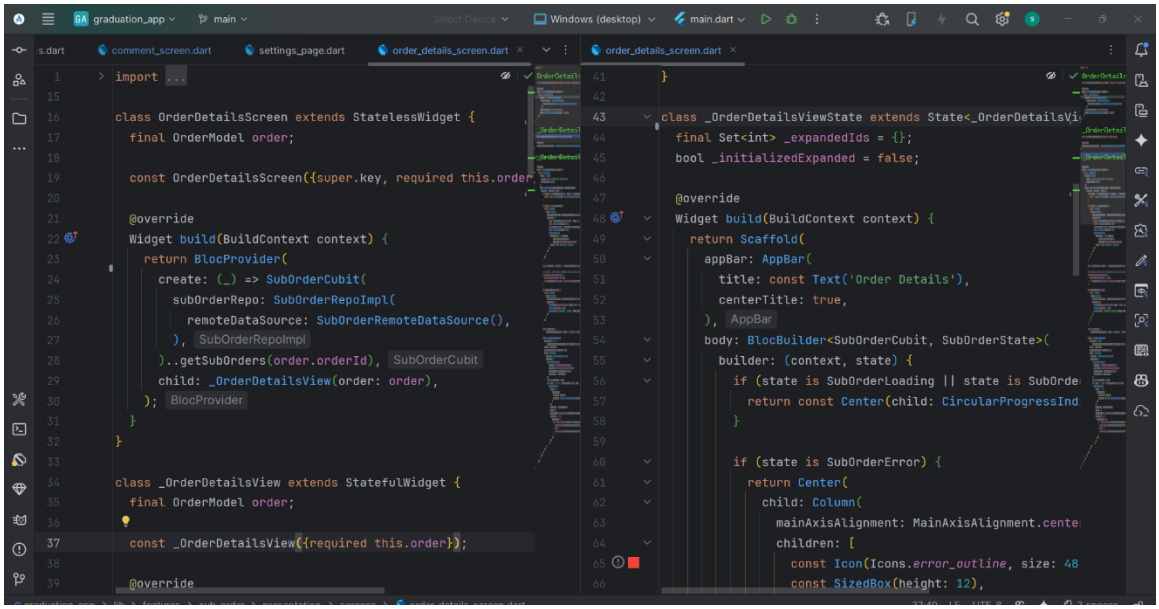
- **Cubit implementation**



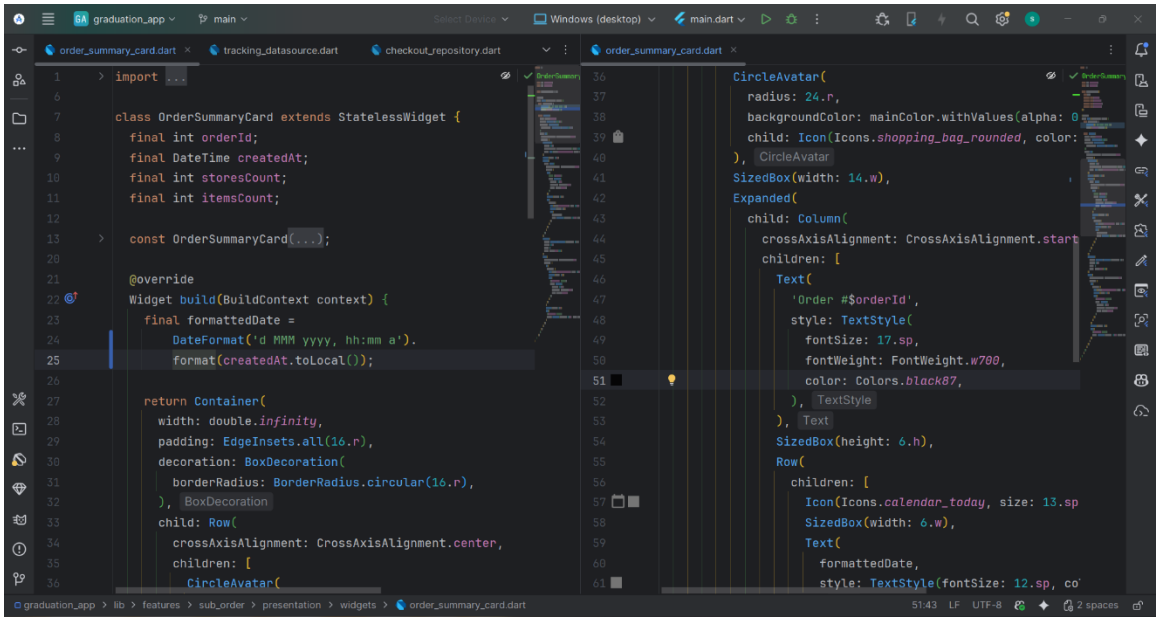
- **states implementation**



- **screen implementation**



- **widget implementation**



6.2.3 Web Application

6.2.3.1 Web Application Technologies

The web application was developed using modern front-end technologies to provide a responsive, scalable, and user-friendly experience across different devices and browsers.

- **React**

Used as the core library for building reusable user interface components and managing application views.

Key features include:

- **React DOM**

Handled entry-point rendering of the React component tree into the browser DOM.

- **Vite**

Utilized as the build tool and development server to achieve near-instant hot module replacement (HMR).

- **Vite React SWC Plugin (v3.11.0):** Integrated to speed up compilation times during development using the SWC compiler.

- **JavaScript (ES6+)**

  The primary programming language used to develop the application logic and markup structure.

6.2.3.2 User Interface & Architecture

The web application follows a component-based architecture that promotes modularity, maintainability, and code reusability.

- **Main Views:**
    - Doctors List Page (Dynamic catalog of available physicians)
    - Doctor Details Page (Comprehensive profile view)
- **Layout & Core Components:**
    - Navigation Bar & Sidebar Filters (For seamless browsing and category selection)
    - Booking Section (Interactive interface for reserving slots)
    - Footer Section (Standard platform links and metadata)

6.2.3.3 Styling & Responsive Design

The visual layer is styled using **Tailwind CSS** combined with custom CSS variables for global configurations (such as the primary color theme), ensuring a clean and modern interface that adapts seamlessly to all screen sizes.

- **Layout Technique:** Built primarily with Flexbox and CSS Grid to ensure flexible, fluid card layouts and structure.
- **Responsive Breakpoint:** Managed via tailwind utilities and native media queries @media (max-width: 768px) to handle:
    - Collapsing standard navigation links into a mobile-friendly menu.
    - Adjusting the sidebar filters to occupy full-width on smaller touch screens.

- Fluid image scaling and optimized touch targets for better mobile UX.

Color Palette

The application uses a centralized design system that maintains visual consistency throughout the platform.

Primary colors include:

- Primary Color
- Secondary Color
- Success Color
- Warning Color
- Error Color
- Neutral Gray Shades

6.2.3.4 Project Directory Structure

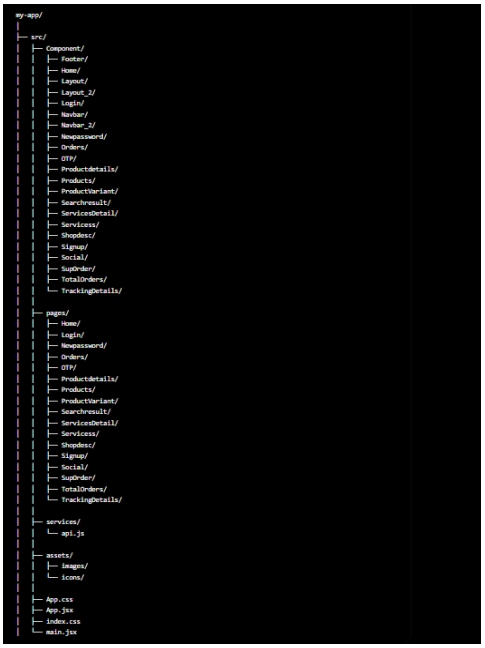
The files are structured logically to separate business logic, UI definitions, and static assets:

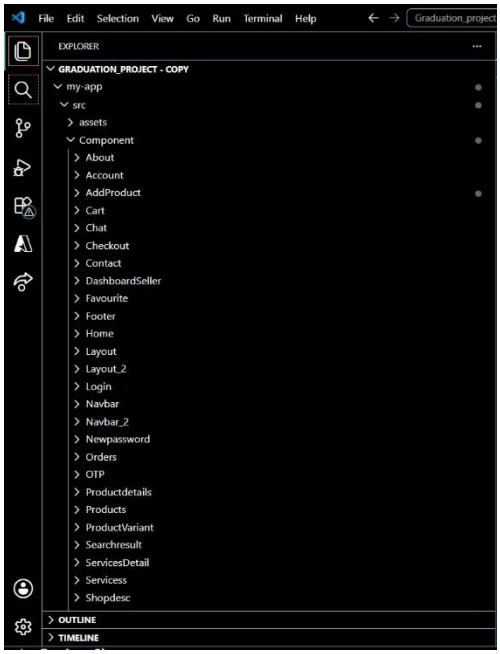
```
├── src/
│   ├── assets/        # Project images and static media resources
│   ├── components/    # Global, reusable UI components (Buttons, Cards, Inputs)
│   ├── pages/         # Main page views
│   ├── hooks/         # Custom React hooks for isolated logic
│   ├── lib/   │   ├── ui/            # Pre-styled foundational UI components
│   ├── index.css      # Global CSS rules and Tailwind directives
│   └── main.jsx       # Application entry point and state mounting
```

6.2.3.5 Build Tools & Quality Assurance

- **npm:** Used as the package manager for managing dependencies and project scripts.
- **ESLint:** Configured to enforce code quality, uniform syntax structure, and catch potential errors early.
- **PostCSS & Autoprefixer:** Integrated within the Vite pipeline to automatically handle browser vendor prefixes and compile Tailwind CSS directives into standard production-ready CSS.

6.2.3.6 React Architecture





6.2.3.7 React code overview

Component implementation

```
306                 <div className="products-grid">
                      Complexity is 9 It's time to do something...
307                   {products.map((p) => (
308                     <div key={p.id} className="product-card" style={{ position: 'relative' }}>
309                       <span className="heart">♡</span>
310                       <img
311                         src={p.mainImage?.mediaPath || imageee}
312                         alt={p.name}
313                       />
314                       <h4>{p.name}</h4>
315                       <div className="price">
316                         ${p.finalPrice?.toFixed(2)}
317                       </div>
318                       <div className="stars">
319                         {renderStars(p.rating)} ({p.reviewsCount || 0})
320                       </div>
321                     </div>
322                   ))}
323                 </div>
324
```

```
my-app > src > Component > Servicess > Servicess.jsx > ...
1   import React, { useState, useEffect } from "react";  7.9k (gzipped: 3.1k)
2   import { useNavigate, useParams } from "react-router-dom";  208.2k (gzipped: 65.1k)
3   import axios from "axios";  70.9k (gzipped: 26.2k)
4   import "bootstrap/dist/css/bootstrap.min.css";
5
      Complexity is 112 Bloody hell...
6   export default function Servicess() {
7     const [services, setServices] = useState([]);
8     const [categories, setCategories] = useState([]);
9     const [loading, setLoading] = useState(true);
10    const [error, setError] = useState(null);
11
12
```

- **css implementation**

```css
@media (max-width: 576px) {
  .shops-grid,
  .products-grid,
  .appointments,
  .deals,
  .recommend,
  .why-grid,
  .features,
  .stats {
    grid-template-columns: 1fr;
  }

  .hero {
    flex-direction: column;
    text-align: center;
    padding: 24px;
  }

  .hero img {
    width: 100%;
    max-width: 250px;
    margin-top: 20px;
  }

  .hero h1 {
    font-size: 24px;
  }
}
```

- **Fetch products implementation**

```jsx
{/* ================ RECOMMENDED ================ */}
<section className="section" ref={recommendRef}>
  <span className="section-tag">Recommendation</span>
  <div className="section-head">
    <h2>Recommended For You</h2>
  </div>
  <div className="recommend">
    <div className="rec-card">
      <span style={{ fontSize: 24 }}>⚡</span>
      <h3>Most Booked This Week</h3>
      <ul>
        <li>Personal Training Sessions</li>
        <li>Web Development Course</li>
        <li>General Health Checkup</li>
      </ul>
    </div>
    <div className="rec-card">
      <span style={{ fontSize: 24, color: "#f5a623" }}>★</span>
      <h3>Highly Rated Services</h3>
      <ul>
        <li>Wedding Photography</li>
        <li>Yoga Classes</li>
        <li>Data Science Course</li>
      </ul>
    </div>
    <div className="rec-card">
      <span style={{ fontSize: 24, color: "#f5a623" }}>⚡</span>
      <h3>Quick Availability</h3>
      <ul>
        <li>Group Fitness Today</li>
        <li>Portrait Photography</li>
        <li>Digital Marketing Course</li>
      </ul>
    </div>
  </div>
</section>
```

- **recommended products implementation**

```
/* ================ FETCH HERO PRODUCT ================ */
Complexity is 7 It's time to do something...
const fetchHeroProduct = async () => {
  try {
    const res = await fetch(`/Api/V1/Product/Discount?PageNumber=1&PageSize=10`);
    const data = await res.json();
    const products = extract(data);

    const heroItem = products[0] || {};
    setHeroProduct({
      name: heroItem.name || "Featured Product",
      image: heroItem.mainImage || { mediaPath: image },
      discount: heroItem.discount || 10
    });
  } catch (error) {
    console.error("Hero fetch error:", error);
  }
};

/* ================ FETCH SHOPS (PAGINATION) ================ */
Complexity is 3 Everything is cool!
const fetchShops = async (page = 1) => {
  const res = await fetch(
    `/Api/V1/Store/Paginated?PageNumber=${page}&PageSize=4&Type=1`
  );
  const data = await res.json();
  const items = extract(data);

  setShops(items);
  setShopPage(page);
  setShopTotalPages(
    data.totalPages || Math.ceil((data.totalCount || 7) / 4)
  );
};

```

6.2.4 Artificial Intelligence Module

6.2.4.1 AI Technologies

The AI recommendation module was developed using modern Python-based technologies and machine learning libraries to provide personalized product recommendations based on user demographic information and purchasing behavior.

Technologies Used:

- **Python**

Python was used as the primary programming language for developing the recommendation engine due to its simplicity, flexibility, and extensive ecosystem of data science and machine learning libraries.

- **FastAPI**

FastAPI was used to build and expose the recommendation engine as a RESTful API service.

Key features include:

- High Performance API Development
- Automatic API Documentation
- Request Validation
- Easy Integration with Backend Systems

- **Scikit-Learn**

Scikit-Learn was used to implement machine learning and recommendation algorithms.

It provides:

- Nearest Neighbors Algorithm
- Data Preprocessing Utilities
- Feature Engineering Tools
- Similarity Calculations

- **Pandas**

Pandas was used for data manipulation, processing, and analysis of user and product datasets.

- **NumPy**

NumPy was utilized for numerical computations and efficient handling of multidimensional data structures.

- **Joblib**

Joblib was used for model serialization and persistence, allowing the trained recommendation model to be saved and loaded efficiently without retraining.

- **Pydantic**

Pydantic was used for request and response validation, ensuring data consistency and reliability within the recommendation API.

- **Uvicorn**

Uvicorn was used as the ASGI server responsible for hosting and running the FastAPI application.

6.2.4.2 Recommendation System Architecture

The recommendation module follows a demographic-based recommendation approach that generates personalized product suggestions based on similarities between users.

The system consists of the following components:

- **Data Processing Layer**

Responsible for processing user demographic information and transforming raw data into machine-learning-ready features.

- **Feature Engineering Layer**

Extracts and prepares relevant user attributes, including:

  - Age (calculated from Date of Birth)
  - Gender
  - City
  - Country

Categorical features are encoded using One-Hot Encoding, while numerical features are normalized using Standard Scaling.

- **Recommendation Engine**

The recommendation engine identifies users with similar demographic characteristics and analyzes their purchasing behavior to generate personalized recommendations.

- **API Layer**

Exposes recommendation services through RESTful API endpoints that allow the main platform to request personalized product recommendations.

6.2.4.3 Recommendation Algorithm

The recommendation system implements a Demographic-Based Recommendation approach using the K-Nearest Neighbors (KNN) algorithm.

The recommendation process consists of the following steps:

1. Collecting user demographic information.

2. Calculating user age from the provided birthdate.
3. Transforming user attributes into numerical feature vectors.
4. Finding the most similar users using the Nearest Neighbors algorithm.
5. Calculating similarity scores using Cosine Similarity.
6. Analyzing products purchased by similar users.
7. Ranking products according to similarity-based scores.
8. Returning the highest-ranked products as recommendations.

When insufficient similarity information is available, the system falls back to recommending globally popular products.

6.2.4.4 Model Persistence and Deployment

The trained recommendation model is stored using Joblib and loaded automatically when the recommendation service starts.

This approach provides:

- Faster Startup Time
- Reduced Computational Cost
- Consistent Recommendation Results
- Efficient Production Deployment

The recommendation service is deployed as a standalone FastAPI application and integrated with the main platform through REST API communication.

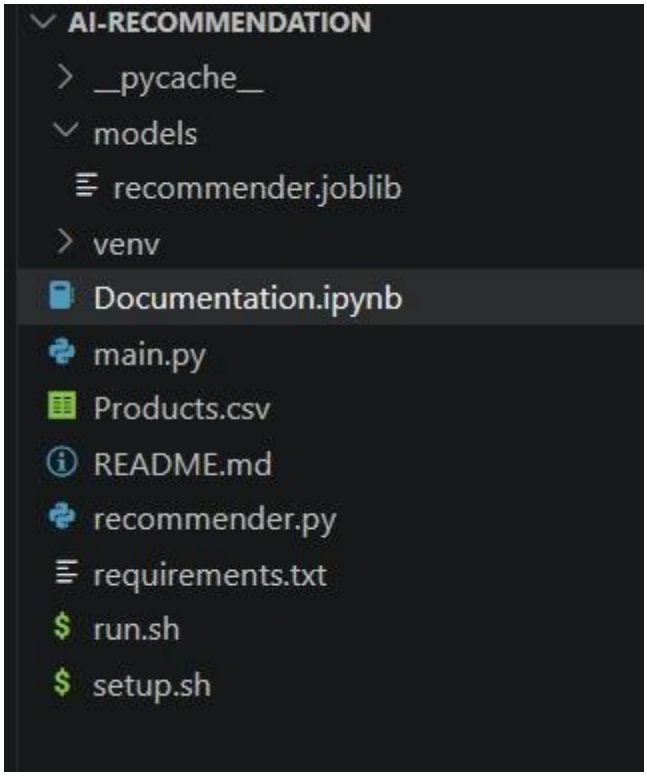
6.2.4.5 Benefits of the Recommendation System

The recommendation module provides several advantages:

- Personalized Product Recommendations
- Improved Product Discovery
- Enhanced User Experience
- Increased User Engagement
- Better Product Visibility
- Faster Access to Relevant Products

By leveraging demographic similarities and purchasing behavior, the system helps users discover products that are more relevant to their interests and characteristics.

6.2.4.6 Artificial intelligence code overview




```
        self._user_features = self._build_feature_matrix(self.users)
        n_neighbors = min(self.n_neighbors + 1, len(self.users))
        self._nn = NearestNeighbors(
            metric="cosine",
            algorithm="brute",
            n_neighbors=n_neighbors,
        )
        self._nn.fit(self._user_features)

        self._user_purchases = (
            transactions.groupby("user_id")["product_id"]
            .apply(lambda s: set(s.astype(int).tolist()))
            .to_dict()
        )
        return self
```

```python
def _build_feature_matrix(self, df: pd.DataFrame) -> np.ndarray:
    df = df.copy()
    df["birthdate"] = pd.to_datetime(df["birthdate"]).dt.date
    df["age"] = df["birthdate"].apply(_age_years)

    if self._preprocessor is None:
        self._preprocessor = ColumnTransformer(
            transformers=[
                ("cat", OneHotEncoder(handle_unknown="ignore"), ["gender", "city", "country"]),
                ("num", StandardScaler(), ["age"]),
            ]
        )
        return self._preprocessor.fit_transform(df)

    return self._preprocessor.transform(df)
```

6.2.4.7 Artificial intelligence Endpoint

6.2.5 UI/UX

The UI/UX implementation focused on designing an:

- intuitive,
- user-friendly,
- visually consistent interface

that enhances the overall user experience.

The design process began with analyzing user requirements and project objectives to understand user needs and system functionality.

Wireframes were then created to define the layout and navigation structure of the application before developing high-fidelity user interface designs.

Attention was given to selecting appropriate colors, typography, icons, and reusable components to ensure consistency across all screens. Additionally, user flows and

interactive elements were carefully designed to simplify navigation and improve usability.

A comprehensive design system was established to maintain a unified visual identity throughout the application.

Using Figma, interactive prototypes and complete screen designs were developed, reviewed, and refined continuously to achieve a professional, attractive, and efficient user experience that aligns with modern UI/UX design principles

---

6.2.6 Development & version control

- **Git**

was used throughout the development lifecycle to manage source code versions, track changes, and support collaborative development among team members. It enabled efficient branch management and facilitated the integration of new features while maintaining project stability.

- **GitHub**

GitHub served as the central repository for hosting the project's source code. It was used for collaboration, branch management, pull requests, issue tracking, and maintaining a structured workflow between the frontend and backend development teams.

- **Visual Studio Code**

Visual Studio Code was used as the primary development environment for the frontend applications, including both the Flutter mobile application and the React web application. Its extensive extension ecosystem, integrated debugging tools, and support for modern development frameworks improved productivity and streamlined the development process.

- **Microsoft Visual Studio**

Microsoft Visual Studio was used as the primary integrated development environment (IDE) for backend development using ASP.NET Core. It provided advanced debugging tools, project management capabilities, integrated package management, and seamless support for developing, testing, and deploying RESTful APIs and backend services.

- **Postman**

Postman was used extensively to test and validate API endpoints during development. It helped ensure accurate request and response handling, verify authentication and authorization processes, and facilitate smooth integration between frontend applications and backend services.

CHAPTER 7— CONCLUSION

7.1 conclusion

In conclusion, the developed platform successfully provides an integrated solution that combines e-commerce, service booking, instructor management, social interaction, and real-time communication within a single system. The project was designed and implemented using modern technologies and architectural practices to ensure scalability, maintainability, performance, and a seamless user experience. Through its multiple modules, including product management, order and booking management, chat functionality, administrative dashboards, and recommendation features, the platform effectively addresses the needs of customers, sellers, instructors, and administrators. The achieved results demonstrate the effectiveness of the proposed solution and establish a strong foundation for future enhancements and expansion.

7.2 future features

To further improve the platform and enhance user engagement, several advanced features can be implemented in future releases.

1. **Community-Based Personalization**

During registration, users will be able to select their community, residential area, or location (e.g., 6th of October, New Cairo, Sheikh Zayed, etc.). Based on the selected community, the platform will provide personalized content, services, and social interactions.

Objectives:

- Deliver a more personalized user experience.
- Increase local engagement among community members.
- Improve accessibility to nearby services and businesses.

Expected Benefits:

- Display community-specific stores and service providers.
- Connect users with people from the same area.
- Provide community-focused social feeds for local discussions and announcements.

- Recommend products, services, and events based on location.
- Support local business promotion and community activities.
- Enable personalized notifications related to the user's community.

---

2. AI Chatbot Assistant

An intelligent chatbot can be integrated into the platform to provide instant support and assistance to users.

Objectives:

- Improve user experience and reduce response time.
- Assist users in navigating platform features efficiently.

Expected Benefits:

- Answer frequently asked questions.
- Recommend suitable products and services.
- Assist users with booking processes.
- Track orders and reservations.
- Provide personalized suggestions based on user preferences.

---

3. AI Content Moderation

Artificial Intelligence can be utilized to automatically monitor and moderate user-generated content.

Objectives:

- Maintain a safe and professional community environment.
- Reduce harmful or inappropriate content.

Expected Benefits:

- Detect spam messages and fake content.
- Identify offensive or abusive language.
- Filter inappropriate posts before publication.
- Improve overall trust and user satisfaction.

---

4. Smart Search Engine

A semantic search engine can be implemented to understand user intent rather than relying solely on keywords.

Objectives

- Enhance search accuracy and efficiency.
- Help users find relevant content faster.

Expected Benefits

- Understand natural language queries.
- Provide intelligent recommendations based on user requests.
- Improve discovery of products, services, and community discussions.

Example:
A search query such as "I need a mathematics tutor near me" would directly suggest the most suitable tutors based on location, ratings, and availability.

---

5. Loyalty and Rewards System

A reward mechanism can be introduced to encourage user participation and platform engagement.

Objectives:

- Increase user retention and activity.
- Encourage continuous interaction within the platform.

Expected Benefits:

- Award points for purchases, bookings, and community participation.
- Offer discounts and exclusive benefits.
- Recognize active users through badges and achievements.

---

6. Push Notifications

A real-time notification system can be implemented to keep users informed about important activities.

Objectives:

- Improve communication between users and the platform.
- Increase user engagement and responsiveness.

Expected Benefits:

- Instant notifications for bookings and reservations.
- Updates on messages and chat activities.
- Alerts for order status changes.
- Notifications for community announcements and events.

---

7. Community Groups

The platform can support dedicated groups that allow users to interact based on common interests or locations.

Objectives:

- Strengthen social connections within the platform.
- Encourage knowledge sharing and collaboration.

Expected Benefits:

- Create groups based on communities, universities, professions, or interests.
- Facilitate discussions and information exchange.
- Support local events and collaborative activities.
- Build stronger and more active communities.

---

8. Voice Search

Voice-based search functionality can be integrated to simplify the search process and improve accessibility.

Objectives:

- Provide a faster and more convenient search experience.
- Enhance accessibility for different types of users.

Expected Benefits:

- Allow users to search using voice commands.
- Reduce typing effort and improve usability.
- Support mobile users and hands-free interaction.
- Improve overall accessibility and user experience.

7.3 Project links

1. https://trabot.runasp.net/swagger/index.html
2. https://www.figma.com/design/SrlV2wCE75jQzpKYxnW46N/Final-proj?node-id=130-6839&t=8CXJpFBr042u8K4R-1
3. https://github.com/shahd-nabil/Graduation-project
4. https://product-recommendation-hcez.vercel.app/docs#/default/recommend_recommend_post
5. https://github.com/SalspilAmin/Product-Recommendation
6. https://github.com/SamyMo7amed/Tradify
7.

7.4 References

[1] Microsoft, “ASP.NET Core Documentation.” Available:

https://learn.microsoft.com/en-us/aspnet/core/

[2] Microsoft, “Entity Framework Core Documentation.” Available:

https://learn.microsoft.com/en-us/ef/core/

[3] Microsoft, “SQL Server Documentation.” Available:

https://learn.microsoft.com/en-us/sql/sql-server/

[4] Flutter Team, “Flutter Documentation.” Available:

https://docs.flutter.dev/

[5] “Dart Documentation.” Available:

https://dart.dev/

[6] Flutter Bloc Library Documentation. Available:

https://bloclibrary.dev/

[7] Dio Package Documentation. Available:

https://pub.dev/packages/dio

[8] React Documentation. Available:

https://react.dev/

[9] Tailwind CSS Documentation. Available:

https://tailwindcss.com/docs

[10] FastAPI Documentation. Available:

https://fastapi.tiangolo.com/

[11] Scikit-Learn Documentation. Available:

https://scikit-learn.org/

[12] Figma Documentation. Available:

https://help.figma.com/

[13] Git Documentation. Available:

https://git-scm.com/doc

[14] GitHub Documentation. Available:

https://docs.github.com/